

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

Курсова робота з освітнього компоненту
«Комп’ютерне моделювання дискретно-подійних систем.
Курсова робота»

Тема: Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування

Керівник:
д.т.н., професор
Стеценко Інна Вячеславівна

Виконавець:
Понзель Тетяна Юріївна
студентка групи ІП-31
залікова книжка №

«Допущено до захисту»
_____

«___» _____ 2026 р.

_____

«11» листопада 2026 р.

Захищено з оцінкою
_____

Члени комісії:

_____ Інна СТЕЦЕНКО

_____ Дмитро КОРИНЕЦЬКИЙ

Київ – 2026

ЗАВДАННЯ

Виконати дослідження для такої системи:

Експериментальна роботизована гнучка виробнича система має два верстати із числовим пультом керування, три роботи, пункт прибуття і склад оброблених деталей. Деталі прибувають на пункт прибуття кожні 40 секунд згідно з експоненціальним законом розподілу, захоплюються одним з вільних роботів і переміщуються ним до першого верстата, після чого робот звільняється. Після завершення обробки на першому верстаті деталь захоплюється одним з роботів і переміщується на другий верстат, а після обробки на другому верстаті – одним з роботів переміщується на склад оброблених деталей. Кожний з верстатів може одночасно обробляти до трьох деталей.

Час переміщення робота між пунктом прибуття і першим верстатом, першим і другим верстатом, другим верстатом і пунктом зберігання оброблених деталей складає відповідно 6, 7, і 5 секунд незалежно від того, холостий це хід, чи ні. Роботу потрібний час 8±1 секунд на захоплення або вивільнення деталей. Час обробки на першому верстаті розподілений за нормальним законом із середнім значенням 60 секунд і має стандартне відхилення 10 секунд. Середній час обробки на другому верстаті дорівнює 100 секунд і має експоненціальний закон розподілу.

Визначте найкращий (з точки зору підвищення пропускної здатності гнучкої виробничої системи) спосіб закріплення роботів до операцій. Можливі варіанти закріплення:

1) по одному роботу на кожний з трьох шляхів переміщення деталей (пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат – склад);

2) кожний робот може використовуватися на кожному шляху переміщення деталей (при цьому повинен займатися найближчий з роботів).

Знайдіть розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів, максимальну місткість місця зберігання деталей на ділянці прибуття.

Для вищезазначеної системи:

1) розробити опис концептуальної моделі системи;

2) виконати формалізацію опису об’єкта моделювання в термінах визначеного у завданні формалізму (мережа масового обслуговування);

3) розробити алгоритм імітації моделі дискретно-подійної системи у відповідності до побудованого формального опису;

4) для доведення коректності побудованого алгоритму виконати верифікацію алгоритму імітації;

5) визначити статистичні оцінки заданих характеристик моделі, що є метою моделювання;

6) провести експериментальне дослідження моделі;

7) інтерпретувати результати моделювання та сформулювати пропозиції щодо поліпшення функціонування системи;

8) зробити висновки щодо складності розробки моделі та алгоритму імітації на основі використаного формалізму, отриманих результатів моделювання та їх корисності.

АНОТАЦІЯ

Курсова робота на тему «Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування».

Текстова частина курсової роботи складається із вступу, 5 розділів, висновків, списку використаних джерел з 10 найменувань та 3 додатків, які викладено на 113 сторінках. В тексті роботи оформлено 15 рисунків, 25 таблиць.

У роботі поставлено завдання визначити найкращий з погляду пропускної здатності спосіб закріплення роботів до операцій переміщення деталей в експериментальній роботизованій гнучкій виробничій системі з двох верстатів і трьох роботів. Порівнюються закріплення по одному роботу за кожним шляхом переміщення і призначення найближчого вільного робота на будь-який шлях. Для кожного способу потрібно знайти розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів і максимальну місткість місця зберігання на ділянці прибуття.

Імітаційна модель розроблена на основі формального опису мережею масового обслуговування з блокуванням маршрутів. Алгоритм імітації власний, з просуванням часу до найближчої події, реалізований мовою Python. Для статистичної обробки результатів використано бібліотеки NumPy і SciPy, для побудови рисунків – Matplotlib. Коректність моделі підтверджено верифікацією, автоматичними тестами і протоколом подій.

Проведено експериментальне дослідження моделі: визначено тривалість перехідного періоду і кількість прогонів, способи закріплення роботів порівняно методом дисперсійного аналізу, досліджено межу стійкості системи, закон розподілу часу проходження ідентифіковано за критерієм χ^2.

Результати моделювання свідчать, що найкращим є закріплення по одному роботу за кожним шляхом: його пропускна здатність становить 83,1 дет./год проти 76,4 дет./год при призначенні найближчого робота. За умови завдання, 90 дет./год, обидва варіанти перевантажені, а вузьким місцем є другий верстат. Час проходження деталей розподілений за гамма-законом зі зсувом.

Зроблено висновок, що для обробки всього потоку деталей доцільно додати четверте місце обробки на другому верстаті і зберегти закріплення роботів за шляхами. Пробні запуски показали, що після цієї зміни система обробляє 90 дет./год із середнім часом проходження 411 с.

Ключові слова: імітаційна модель, алгоритм імітації, мережа масового обслуговування, гнучка виробнича система, закріплення роботів, дисперсійний аналіз, Python.

ЗМІСТ

ВСТУП 9

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ 11

1.1 Мета моделювання 11
1.2 Складові елементи системи 11
1.3 Опис процесу функціонування системи 13
1.4 Параметри моделі 16
1.5 Вхідні змінні 16
1.6 Вихідні змінні 17
1.7 Висновки до розділу 18

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ 19

2.1 Відповідність елементів системи елементам мережі МО 19
2.2 Мережа МО для варіанта 1 21
2.3 Мережа МО для варіанта 2 23
2.4 Часові параметри пристроїв 28
2.5 Числові параметри та генератори випадкових величин 29
2.6 Стан мережі, правила функціонування та початковий стан 30
2.7 Обчислення вихідних характеристик та вимоги до точності 31
2.8 Висновки до розділу 32

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ 33

3.1 Обґрунтування вибору мови програмування 33
3.2 Структура програми 33
3.3 Алгоритм імітації 34
3.4 Події та зміна стану елементів мережі 36
3.5 Блокування маршрутів і керування роботами 38
3.6 Генератори випадкових величин 39
3.7 Збір статистики 40

3.8 Опис класів та UML-діаграма класів 41
3.9 Верифікація моделі 43
3.10 Висновки до розділу 47
4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ 48
4.1 Мета і план експериментального дослідження 48
4.2 Тривалість перехідного періоду та кількість прогонів 49
4.3 Порівняння способів закріплення роботів 54
4.4 Пропускна здатність залежно від інтенсивності надходження деталей 57
4.5 Коефіцієнти використання роботів і верстатів 58
4.6 Розподіл часу проходження деталей 60
4.7 Максимальна місткість місця зберігання на ділянці прибуття 64
4.8 Програмна реалізація експериментів 65
4.9 Висновки до розділу 66
5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ СИСТЕМИ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ 68
5.1 Інтерпретація результатів моделювання 68
5.2 Пропозиції щодо поліпшення функціонування системи 71
5.3 Пропозиції щодо подальшого дослідження 73
5.4 Висновки до розділу 74
ВИСНОВКИ 75
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 78
ДОДАТОК А. ПРОТОКОЛИ ПОДІЙ 79
А.1 Протокол подій для варіанта 1 80
А.2 Протокол подій для варіанта 2 83
ДОДАТОК Б. ЛІСТИНГ КОДУ АЛГОРИТМУ ІМІТАЦІЇ 87
Б.1 Модуль rng.py – генератори випадкових величин 88
Б.2 Модуль params.py – вхідні змінні та параметри моделі 90

Б.3 Модуль elements.py – елементи мережі масового обслуговування.. 91
Б.4 Модуль model.py – алгоритм імітації 95
ДОДАТОК В. ЛІСТИНГ КОДУ ЕКСПЕРИМЕНТАЛЬНОГО ДОСЛІДЖЕННЯ 102
В.1 Модуль common.py – спільні засоби для серій прогонів 103
В.2 Модуль verification.py – верифікація моделі 105
В.3 Модуль warmup.py – тривалість перехідного періоду 106
В.4 Модуль replications.py – кількість прогонів 108
В.5 Модуль compare.py – порівняння способів закріплення роботів ... 109
В.6 Модуль capacity.py – межа стійкості 110
В.7 Модуль distribution.py – розподіл часу проходження деталей 111
В.8 Модуль improvement.py – пробні запуски пропозицій 113

ВСТУП

Гнучкі виробничі системи (ГВС) з верстатами з числовим програмним керуванням і промисловими роботами дають змогу автоматизувати обробку деталей і швидко переналаштовувати виробництво. Продуктивність такої системи залежить не лише від швидкодії верстатів, а й від організації транспортування деталей між ними. Одних і тих самих роботів можна закріпити за окремими шляхами переміщення або дозволити кожному роботу виконувати будь-яке перевезення. Який спосіб кращий, заздалегідь не очевидно. Перевіряти це на реальному обладнанні дорого, а аналітичні методи теорії масового обслуговування погано враховують блокування верстатів, очікування роботів із деталлю і залежність вибору робота від його положення. Тому для розв’язання такого завдання застосовують імітаційне моделювання. Воно відтворює роботу системи з урахуванням випадкових тривалостей операцій і дає змогу порівняти варіанти організації роботи без змін у реальній системі [1; 2].

Тема курсової роботи – «Імітаційна модель роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування». Об’єктом моделювання є експериментальна роботизована ГВС із двох верстатів, трьох роботів, пункту прибуття деталей і складу оброблених деталей.

Метою роботи є визначення найкращого з погляду пропускної здатності способу закріплення роботів до операцій переміщення деталей. Порівнюються два способи: по одному роботу на кожному з трьох шляхів переміщення і призначення на будь-який шлях найближчого вільного робота. Для кожного способу потрібно знайти розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів та максимальну місткість місця зберігання деталей на ділянці прибуття.

Для досягнення мети розв’язано такі задачі:

– розроблено концептуальну модель системи з BPMN-діаграмами процесу;

– побудовано формалізовані моделі обох варіантів у вигляді мереж масового обслуговування з блокуванням маршрутів, а для варіанта 2 – з окремим фрагментом мережі для руху роботів;

– розроблено власний алгоритм імітації з просуванням часу до найближчої події і реалізовано його мовою Python [4];

– виконано верифікацію моделі варіюванням вхідних змінних і порівнянням з ручним розрахунком;

– визначено тривалість перехідного періоду і кількість прогонів, порівняно варіанти методом дисперсійного аналізу, ідентифіковано закон розподілу часу проходження за критерієм χ^2;

– інтерпретовано результати моделювання і перевірено пробними запусками пропозиції щодо поліпшення системи.

Статистичну обробку результатів експериментів виконано з використанням бібліотек NumPy і SciPy, рисунки побудовано бібліотекою Matplotlib.

Пояснювальна записка складається з п’яти розділів. У розділі 1 описано концептуальну модель системи, у розділі 2 – формалізовані моделі обох варіантів, у розділі 3 – алгоритм імітації, його програмну реалізацію і верифікацію. Розділ 4 містить експериментальне дослідження моделі, розділ 5 – інтерпретацію результатів і пропозиції щодо поліпшення системи. Протоколи подій і код програми наведено в додатках.

1 КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ

Концептуальна модель системи – це деталізований опис складових елементів системи, правил їх взаємодії та процесу, який моделюється, разом із зазначенням мети моделювання [3, с. 49]. За умовою завдання потрібно порівняти два способи закріплення роботів. Якщо завдання передбачає альтернативні моделі, концептуальний опис розробляють для кожної з них [3, с. 50]. Тому спільні елементи і процес описано один раз, а відмінності – окремо для кожного варіанта.

1.1 Мета моделювання

Об’єктом моделювання є експериментальна роботизована гнучка виробнича система (ГВС), що складається з двох верстатів із числовим програмним керуванням, трьох роботів, пункту прибуття деталей і складу оброблених деталей.

Метою моделювання є визначення найкращого з погляду пропускної здатності ГВС способу закріплення роботів до операцій переміщення деталей. Розглядаються два способи:

1) варіант 1 – по одному роботу на кожному з трьох шляхів переміщення деталей: пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат – склад;

2) варіант 2 – кожен робот може виконувати переміщення на будь-якому шляху, при цьому на перевезення призначається найближчий з вільних роботів.

Для кожного варіанта потрібно визначити:

– пропускну здатність ГВС, тобто кількість деталей, що надходять на склад за одиницю часу;

– розподіл часу проходження деталей через систему;

– коефіцієнти використання роботів і верстатів;

– максимальну місткість місця зберігання деталей на ділянці прибуття.

1.2 Складові елементи системи

За текстом завдання виділено такі елементи ГВС:

1) деталі – об’єкти обробки; вони надходять на пункт прибуття в середньому кожні 40 с, інтервали між надходженнями розподілені за експоненціальним законом;

2) пункт прибуття A з місцем зберігання (площадкою), на якому деталі очікують на робота; кількість місць на площадці не обмежена, а її найбільше заповнення є однією з шуканих характеристик;

3) верстати M1 і M2 з числовим програмним керуванням; кожен із них одночасно обробляє до трьох деталей, тобто має три місця обробки; час обробки на M1 розподілений за нормальним законом із середнім 60 с і стандартним відхиленням 10 с, на M2 – за експоненціальним законом із середнім 100 с;

4) три роботи R1, R2, R3, які захоплюють деталь, переміщують її між пунктами і вивільняють; захоплення і вивільнення триває 8 ± 1 с;

5) склад оброблених деталей W, на якому деталь залишає систему.

Час переміщення робота між пунктом прибуття і першим верстатом, першим і другим верстатом, другим верстатом і складом становить відповідно 6, 7 і 5 с і однаковий для руху з деталлю і порожнім. Схему розташування обладнання та закріплення роботів для обох варіантів наведено на рисунку 1.1.

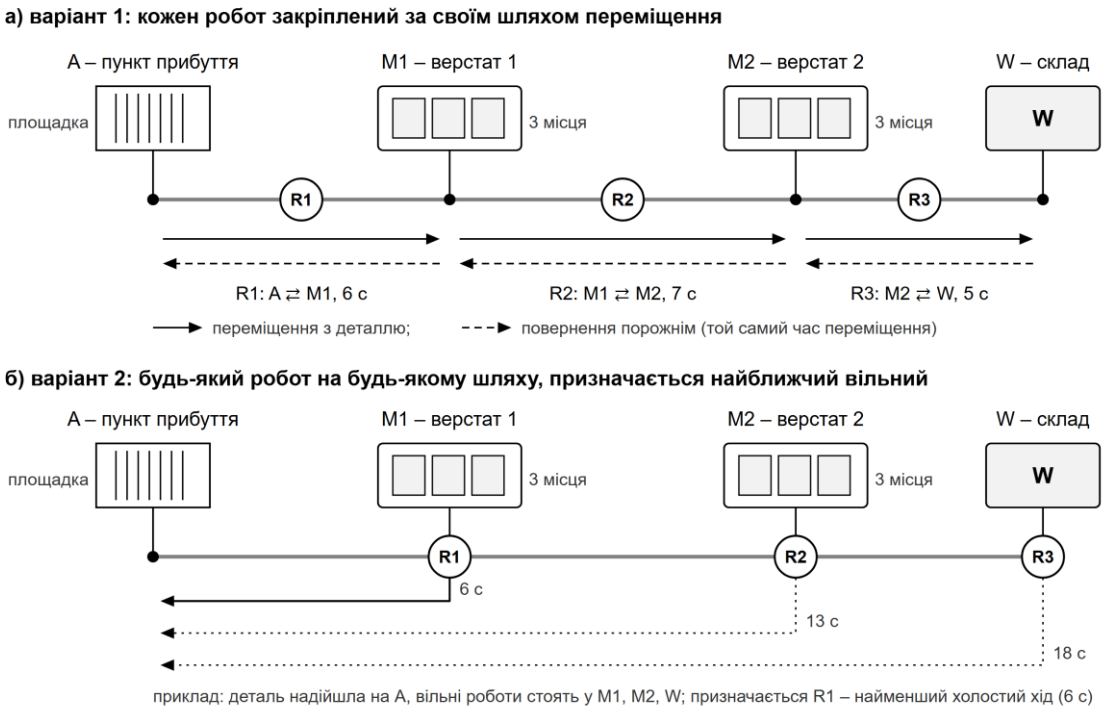


Рисунок 1.1 – Схема розташування обладнання ГВС і закріплення роботів

1.3 Опис процесу функціонування системи

Деталь, що надійшла на пункт прибуття, залишається на площадці, доки її не захопить робот. Перевезення деталі між двома пунктами складається з таких дій робота:

1) захоплення деталі; місце, яке займала деталь, звільняється після завершення захоплення;

2) переміщення з деталлю до пункту призначення;

3) очікування з деталлю, якщо на верстаті-приймачі немає вільного місця; робот тримає деталь і не може виконувати інших перевезень, доки місце не звільниться;

4) вивільнення деталі; місце верстата зайняте вже під час вивільнення, а після його завершення на верстаті починається обробка.

Оброблена на верстаті деталь залишається на своєму місці, доки її не захопить робот. Поки деталь чекає на робота, місце верстата зайняте і нова деталь на нього потрапити не може. Після обробки на M1 деталь перевозиться на M2, після обробки на M2 – на склад, де вона залишає систему.

Варіанти відрізняються правилом призначення робота на перевезення і тим, що робот робить після вивільнення деталі:

– у варіанті 1 кожен робот обслуговує лише свій шлях: R1 – A ⇄ M1, R2 – M1 ⇄ M2, R3 – M2 ⇄ W. Робот везе деталь у прямому напрямку, а після вивільнення одразу повертається порожнім до стартової точки свого шляху і стає вільним лише там. Тому наступну деталь він захоплює без холостого ходу, якщо вона надійшла після його повернення;

– у варіанті 2 будь-який вільний робот може виконати будь-яке перевезення, і на перевезення призначається найближчий з вільних роботів, тобто той, холостий хід якого до точки завантаження найменший. Робот рухається порожнім до деталі, коли його призначено, а після вивільнення залишається в пункті вивантаження. Якщо однаково близько стоять кілька вільних роботів, вони стоять в одній точці і рівноцінні, тому обирається будь-який з них.

У варіанті 2 правило призначення найближчого робота доповнено умовою, без якої система зупиняється. Робот не бере деталь для верстата, до якого вже везуть деталі на одного робота більше, ніж там вільних місць. Кількість роботів не змінюється, їх три за умовою: умова лише визначає, коли вільний робот може взяти деталь. Без неї всі три роботи можуть узяти деталі для одного заповненого верстата й чекати біля нього, а оброблені деталі на цьому верстаті не матимуть вільного робота, і система зупиниться (підрозділ 2.3). У варіанті 1 до кожного верстата їздить лише один закріплений робот, тож така ситуація неможлива.

Під час розробки моделі прийнято такі припущення:

1) пункти розташовані на одній лінії A – M1 – M2 – W, тому час переміщення між несусідніми пунктами дорівнює сумі часів між сусідніми: A – M2 становить 13 с, M1 – W 12 с, A – W 18 с;

2) у варіанті 2 після вивільнення деталі робот залишається в пункті вивантаження;

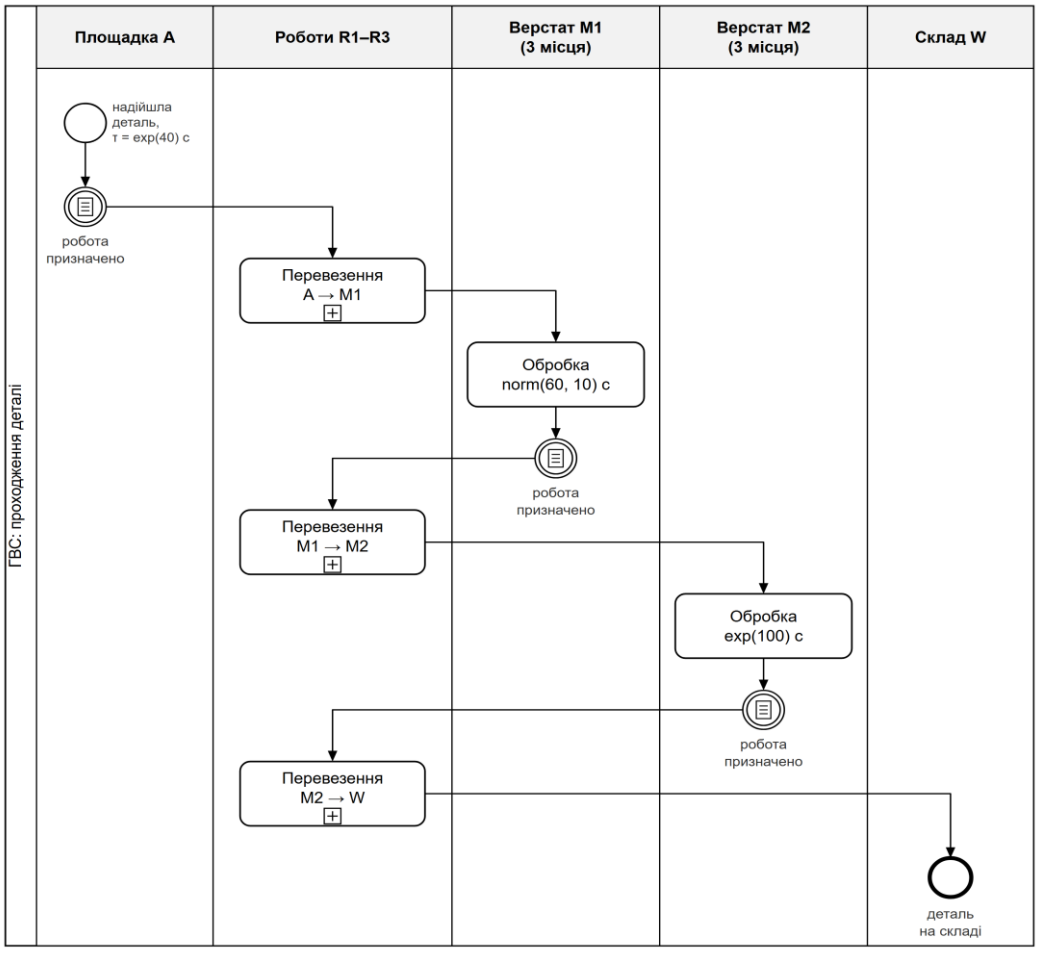
3) час обробки на M1 не може бути від'ємним, тому від'ємні значення нормального закону відкидаються; ймовірність такого значення менша за 10^{-9};

4) якщо на площадці A або на верстаті робота чекають кілька деталей, першою перевозиться та, що стала готовою до перевезення раніше;

5) події, що відбуваються одночасно, виконуються у випадковому порядку;

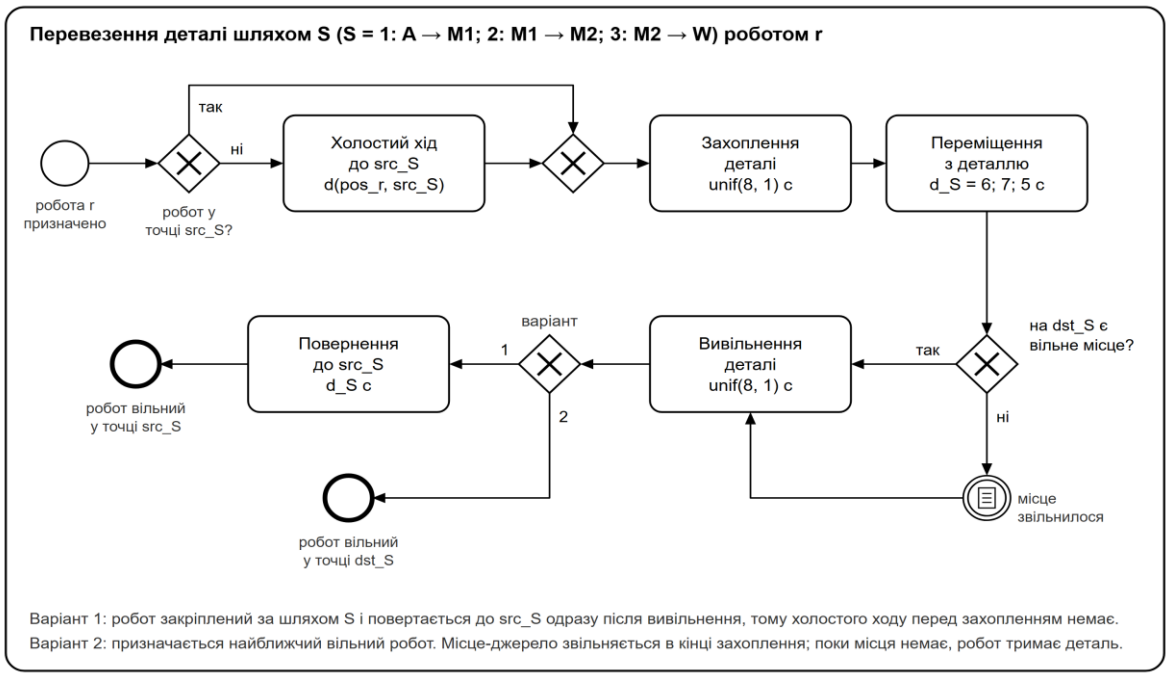
6) у початковий момент система порожня, а роботи стоять у точках завантаження своїх шляхів варіанта 1: R1 – в A, R2 – в M1, R3 – в M2; для варіанта 2 прийнято той самий початковий стан, щоб варіанти порівнювались за рівних умов.

Процес проходження деталі через ГВС у нотації BPMN [5] наведено на рисунку 1.2. Перевезення деталі на діаграмі є підпроцесом, однаковим для всіх трьох шляхів. Його розгорнуто на рисунку 1.3.



◎ – проміжна подія очікування: деталь чекає на робота, займаючи місце на площадці A або на верстаті;
[+] – підпроцес перевезення (рис. 1.3); робота призначає правило варіанта 1 (закріплений робот) або варіанта 2 (найближчий вільний).

Рисунок 1.2 – BPMN-діаграма процесу проходження деталі через ГВС



Варіант 1: робот закріплений за шляхом S і повертається до src_S одразу після вивільнення, тому холостого ходу перед захопленням немає.
Варіант 2: призначається найближчий вільний робот. Місце-джерело звільняється в кінці захоплення; поки місця немає, робот тримає деталь.

Рисунок 1.3 – BPMN-діаграма підпроцесу перевезення деталі роботом

1.4 Параметри моделі

Параметри моделі – це числові характеристики системи, які не змінюються під час експериментів з базовою моделлю. Перелік параметрів наведено в таблиці 1.1.

Таблиця 1.1 – Параметри моделі

Позначення	Параметр	Значення
m	кількість місць обробки на кожному верстаті	3
N_R	кількість роботів	3
$d_1,\ d_2,\ d_3$	час переміщення робота шляхами A – M1, M1 – M2, M2 – W, с	6; 7; 5
—	час переміщення між несусідніми пунктами A – M2, M1 – W, A – W, с	13; 12; 18
max	кількість місць на площадці A	не обмежена
—	закріплення роботів у варіанті 1	R1: A → M1; R2: M1 → M2; R3: M2 → W

1.5 Вхідні змінні

Вхідні змінні моделі описують зовнішній потік деталей і тривалості операцій. Для кожної змінної в таблиці 1.2 наведено закон розподілу з числовими значеннями, початкове значення і діапазон варіювання під час верифікації. Закони розподілу позначено за методичними рекомендаціями [3, с. 51]: $\exp(d)$ – експоненціальний із середнім d; $\mathrm{norm}(d, s)$ – нормальний із середнім d і середнім квадратичним відхиленням s; $\mathrm{unif}(d, s)$ – рівномірний на інтервалі $(d - s;\ d + s)$, тобто $\mathrm{unif}(8,1)$ означає 8 ± 1 с.

Таблиця 1.2 – Вхідні змінні моделі

Позначення	Вхідна змінна	Початкове значення	Діапазон варіювання
τ_A	інтервал між надходженнями деталей, с	$\exp(40)$	від $\exp(30)$ до $\exp(80)$
$t_{\text{обр1}}$	час обробки на верстаті M1, с	$\mathrm{norm}(60,10)$	від $\mathrm{norm}(60,10)$ до $\mathrm{norm}(120,10)$
$t_{\text{обр2}}$	час обробки на верстаті M2, с	$\exp(100)$	від $\exp(50)$ до $\exp(100)$
$t_{\text{зх}}$	час захоплення деталі роботом, с	$\mathrm{unif}(8,1)$	від $\mathrm{unif}(2;\ 0{,}5)$ до $\mathrm{unif}(8,1)$
$t_{\text{вив}}$	час вивільнення деталі роботом, с	$\mathrm{unif}(8,1)$	від $\mathrm{unif}(2;\ 0{,}5)$ до $\mathrm{unif}(8,1)$

Під час верифікації змінювались також час переміщення роботів (до трикратного) і кількість місць на верстаті М2 (до шести), а для пропозицій щодо поліпшення системи – кількість місць на М2 і кількість роботів (підрозділи 3.9 і 5.2).

1.6 Вихідні змінні

Вихідні змінні визначаються метою моделювання. Їх перелік і величини, які спостерігаються під час імітації для їх обчислення, наведено в таблиці 1.3. Формули обчислення наведено в підрозділі 2.7.

Таблиця 1.3 – Вихідні змінні моделі

Позначення	Вихідна змінна	Що спостерігається під час імітації
Λ	пропускна здатність ГВС, дет./год	кількість деталей, вивільнених на складі за час спостереження
τ	час проходження деталі, с	момент надходження деталі на площадку А і момент завершення її вивільнення на складі; за масивом значень обчислюються середнє, середнє квадратичне відхилення, гістограма і закон розподілу
$\rho_{\mathrm{R}k}$	коефіцієнт використання робота Rk	тривалість станів пристроїв, що відповідають роботу: зайнятий (обслуговування деталі, затримка звільнення), блокований (робот тримає деталь), а у варіанті 2 також холостий хід
$\rho_{\mathrm{M}i}$	коефіцієнт використання верстата Mi	тривалість станів кожного місця верстата: вільне, зайняте (вивільнення, обробка, захоплення), блоковане (оброблена деталь чекає на робота)
L_{max}	максимальна місткість місця зберігання на ділянці прибуття, шт.	максимальна кількість деталей на площадці А
$\bar{L}_A$	середня кількість деталей на площадці А, шт.	кількість деталей на площадці А з урахуванням тривалості інтервалів між подіями

Для оцінювання стійкості режиму додатково фіксується кількість деталей, що перебувають у системі в кінці прогону. Якщо вона зростає разом із часом моделювання, черга на площадці накопичується необмежено, і стаціонарного режиму немає.

1.7 Висновки до розділу

У розділі описано концептуальну модель роботизованої ГВС: визначено мету моделювання, складові елементи системи, процес проходження деталі і дії робота під час перевезення. Процес для обох способів закріплення роботів однаковий, а відрізняються правило призначення робота на перевезення і те, де робот чекає на наступну деталь: у варіанті 1 – у стартовій точці свого шляху, у варіанті 2 – там, де вивільнив попередню. Визначено параметри, вхідні та вихідні змінні моделі. Процес представлено BPMN-діаграмами, які є основою для формалізованого опису мережею масового обслуговування в розділі 2.

2 ФОРМАЛІЗОВАНА МОДЕЛЬ СИСТЕМИ

Формалізацію виконано засобами мережі масового обслуговування (МО). Мережа МО складається із систем масового обслуговування (СМО), з'єднаних дугами, що представляють маршрут слідування запитів, і зовнішнього потоку надходження запитів [3, с. 50]. Графічні позначення елементів і їх числові параметри прийнято за таблицею 3.1 методичних рекомендацій [3, с. 52-54], правила функціонування мережі і блокування маршрутів – за навчальним посібником [2, с. 62-67].

2.1 Відповідність елементів системи елементам мережі МО

Запито Запитом мережі є деталь. Мережа розімкнена: деталі надходять ззовні на площадку А і залишають мережу після вивільнення на складі. Під час обслуговування деталь не знищується, не з'єднується з іншими деталями і не розпадається, тому процес можна описати мережею МО [2, с. 65].

Кожне місце, де може перебувати деталь, представлено пристроєм обслуговування: місце на площадці А, місце на верстаті, захват робота, місце на складі. Деталь переходить з одного пристрою в інший дугою мережі. Умови «немає вільного робота» і «на верстаті немає місця» задано блокуванням маршрутів: якщо маршрут закритий, запит залишається в пристрої, з якого намагається вийти, і пристрій переходить у стан «блокований» [3, с. 54]. Блокування стоїть на дузі між СМО. Між чергою і пристроями СМО його розмістити не можна, бо черга є частиною СМО і з'єднується з пристроями відрізком, а не дугою [3, с. 53]. Тому деталь, що чекає на робота, не стоїть у черзі, а займає місце площадки чи верстата як заблокований пристрій.

Стан кожного пристрою набуває трьох значень: 0 – вільний, 1 – зайнятий, 2 – блокований [3, с. 68]. Окрім часу обслуговування t, для пристрою задано затримку звільнення $t_{зв}$ – час, протягом якого пристрій залишається зайнятим після виходу запиту. Вона потрібна з двох причин. По-перше, захоплення і вивільнення деталі робот виконує разом із місцем: під час захоплення деталь ще лежить на місці-джерелі, а під час вивільнення вже займає місце-приймач. По-друге, після вивільнення робот звільняється не одразу: у варіанті 1 він

повертається порожнім до точки завантаження свого шляху. Тривалості захоплення $t_{\text{зх}}$ і вивільнення $t_{\text{вив}}$ генеруються в момент переходу деталі між місцем і роботом і зараховуються обом пристроям: пристрою, у який переходить деталь, – як частина часу обслуговування, а пристрою, з якого вона вийшла, – як затримка звільнення. Відповідність елементів системи елементам мережі МО наведено в таблиці 2.1.

Таблиця 2.1 – Відповідність елементів системи елементам мережі МО

Елемент системи	Варіант 1	Варіант 2
деталь	запит	те саме
надходження деталей	зовнішнє надходження, $t = \exp(40)$	те саме
площадка A	СМО A з необмеженою кількістю пристроїв (місць)	те саме
роботи	три одноканальні СМО R1, R2, R3, кожна на своєму шляху	три триканальні СМО T1, T2, T3, по одній на кожному шляху; пристрій r кожної з них – робот Rr
рух робота без деталі	затримка звільнення пристрою робота (повернення до точки завантаження)	фрагмент руху роботів: СМО П і Х, запити – роботи
верстат M1, M2	СМО з трьох пристроїв (місць) без черги	те саме
склад	СМО W з необмеженою кількістю пристроїв, далі вихід з мережі	те саме
«немає вільного робота», «на верстаті немає місця»	блокування маршрутів B_0-B_4	блокування маршрутів B_0-B_4, B_{X} і керування роботами К

Маршрути переміщення деталей наведено в таблиці 2.2. Тут і далі src_S і dst_S – точки завантаження і вивантаження маршруту S, d_S – час переміщення з деталлю.

Таблиця 2.2 – Маршрути переміщення деталей

Маршрут S	Звідки, src_S	Куди, dst_S	d_S, с	Робот у варіанті 1	Де деталь чекає на робота
1	A	M1	6	R1: A ⇄ M1	на місці площадки A
2	M1	M2	7	R2: M1 ⇄ M2	на місці верстата M1
3	M2	W	5	R3: M2 ⇄ W	на місці верстата M2

2.2 Мережа МО для варіанта 1

Мережу МО, що представляє формалізовану модель ГВС для варіанта 1, наведено на рисунку 2.1. Кожен робот закріплений за одним шляхом, тому йому відповідає окрема одноканальна СМО. Верстат, що одночасно обробляє до трьох деталей, є СМО з трьох ідентичних пристроїв (місць) без черги. Площадка А і склад W – СМО з необмеженою кількістю пристроїв, бо кількість місць на них не обмежена.

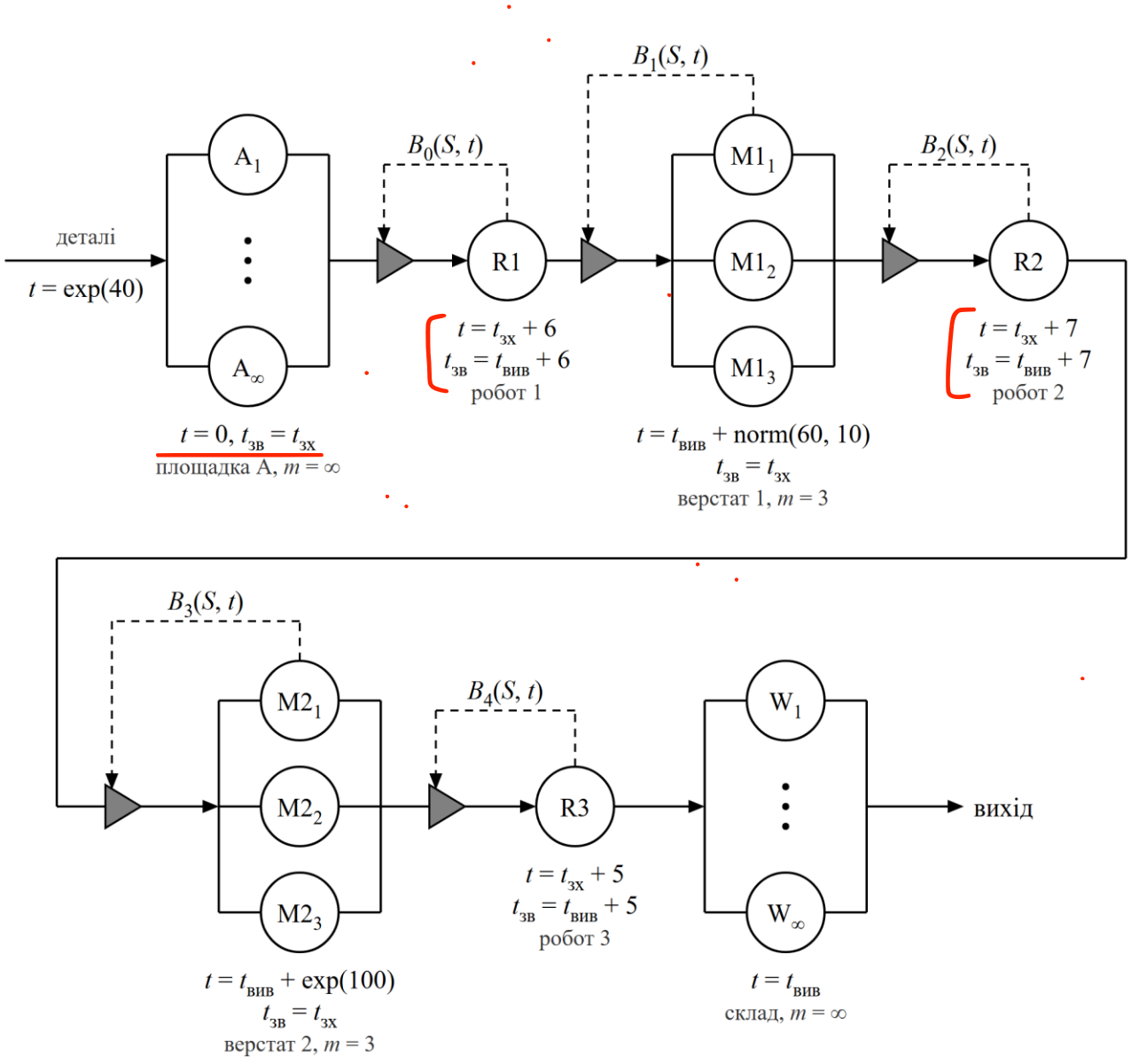


Рисунок 2.1 – Мережа масового обслуговування, що представляє формалізовану модель ГВС (варіант 1)

Дещо відходите від формалізму мережі МО, тому не найліпший варіант формалізації.

Деталь проходить мережу послідовно: A → R1 → M1 → R2 → M2 → R3 → W → вихід. На площадці A час обслуговування дорівнює нулю: деталь одразу намагається перейти до робота R1 і, якщо він зайнятий, чекає на своєму місці. Обслуговування деталі роботом R1 триває $t_{\text{зх}} + 6$: робот захоплює деталь і везе її до M1. Місце площадки звільняється в кінці захоплення, тому його затримка звільнення дорівнює $t_{\text{зх}}$. Коли деталь переходить на місце верстата M1, починається її вивільнення, тож час обслуговування на M1 дорівнює $t_{\text{вив}} + t_{\text{обр1}}$. Робот R1 після переходу деталі ще вивільняє її, а потім за 6 с повертається порожнім до A. Тому пристрій R1 звільняється із затримкою $t_{\text{зв}} = t_{\text{вив}} + 6$, і лише після цього може взяти наступну деталь. Так само працюють R2 і R3. Оброблена деталь переходить з місця верстата до робота, а місце звільняється в кінці захоплення, тобто із затримкою $t_{\text{зх}}$. На складі деталь вивільняється ($t = t_{\text{вив}}$) і залишає мережу.

Трикутники на рисунку 2.1 позначають блокування маршрутів, пунктирні лінії – керуючі зв’язки від елемента, стан якого визначає умову блокування. Зміст блокувань наведено в таблиці 2.3.

Таблиця 2.3 – Блокування маршрутів у варіанті 1

Умова	Маршрут	Маршрут закритий, якщо	Де чекає деталь
B_0	A → R1	робот R1 зайнятий	на місці площадки A
B_1	R1 → M1	усі 3 місця M1 зайняті	у пристрої R1: робот тримає деталь біля M1
B_2	M1 → R2	робот R2 зайнятий	на місці верстата M1
B_3	R2 → M2	усі 3 місця M2 зайняті	у пристрої R2: робот тримає деталь біля M2
B_4	M2 → R3	робот R3 зайнятий	на місці верстата M2

Блокування задано предикатами, які залежать від стану мережі S у момент t [3, с. 54]. Позначимо $K_{E,j}(t)$ стан j-го пристрою СМО E: 0 – вільний, 1 – зайнятий, 2 – блокований. Тоді:

логічну операцію ТА тут ліпше використати

$$B_0(S,t) = \mathbf{1}\{K_{\mathrm{R1}}(t) \neq 0\}, \quad B_1(S,t) = \prod_{j=1}^{3} \mathbf{1}\{K_{\mathrm{M1},j}(t) \neq 0\},$$

$$B_2(S,t) = \mathbf{1}\{K_{\mathrm{R2}}(t) \neq 0\}, \quad B_3(S,t) = \prod_{j=1}^{3} \mathbf{1}\{K_{\mathrm{M2},j}(t) \neq 0\},$$

$$B_4(S,t) = \mathbf{1}\{K_{\mathrm{R3}}(t) \neq 0\}, \qquad (2.1)$$

де $\mathbf{1}\{P\}$ дорівнює 1, якщо умова P виконується, і 0, якщо ні, тож B_1 і B_3 дорівнюють 1, лише коли всі три пристрої верстата не вільні. Маршрут закритий, якщо $B(S,t) = 1$. Маршрути надходження, R3 → W і W → вихід не блокуються.

Блокування B_1 і B_3 перевіряються в момент, коли робот закінчив переміщення з деталлю. Якщо вільного місця немає, деталь залишається в пристрої робота, тобто робот тримає її біля верстата [2, с. 67], і переходить на верстат, щойно там звільниться місце. Блокування B_0, B_2, B_4 тримають деталь на її місці, доки робот не звільниться, тобто не вивільнить попередню деталь і не повернеться до точки завантаження. Якщо на відкриття маршруту чекають кілька пристроїв, першим виходить запит, що був заблокований раніше.

2.3 Мережа МО для варіанта 2

У варіанті 2 будь-який вільний робот може перевезти деталь будь-яким шляхом. Тому на кожному шляху S роботам відповідає триканальна СМО TS, а пристрій r кожної з них – робот Rr (рисунок 2.2). Деталь проходить мережу тим самим маршрутом, що й у варіанті 1: A → T1 → M1 → T2 → M2 → T3 → W → вихід. Отже, обов'язковий маршрут деталі A – M1 – M2 – W зберігається, а від способу закріплення роботів залежить лише те, який пристрій СМО перевезення обслуговує деталь.

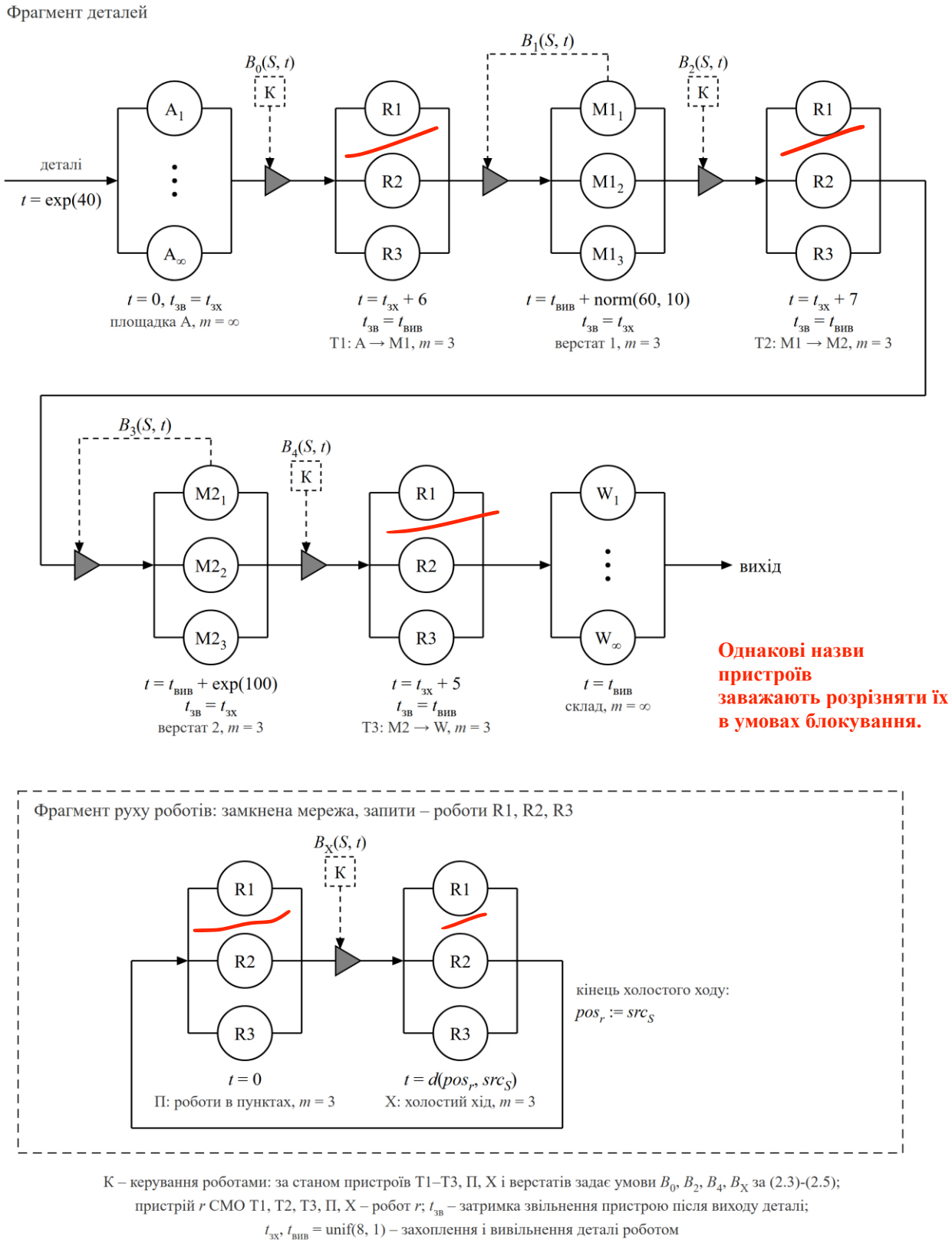


Рисунок 2.2 – Мережа масового обслуговування, що представляє формалізовану модель ГВС (варіант 2)

Пристрої r трьох СМО Т1, Т2, Т3 відповідають одному роботу, тому зайнятим може бути лише один з них. Крім того, робот може захопити деталь лише в тій точці, де він стоїть, а після вивільнення залишається в точці вивантаження. Тому маршрути роботів відстежує окремий фрагмент мережі – фрагмент руху роботів (нижня частина рисунка 2.2). Це замкнена мережа з двох СМО, у якій циркулюють $N_R = 3$ запити – роботи:

– П – роботи в пунктах: запит робота Rr перебуває в r-му пристрої П, поки робот не їде порожнім, тобто стоїть у точці pos_r або працює з деталлю як пристрій СМО Т1-Т3; $t = 0$;

– Х – холостий хід: обслуговування запиту робота Rr триває $d(pos_r, src_S)$ – час переїзду до точки завантаження шляху S; після цього $pos_r := src_S$, і запит повертається в П.

Після вивільнення деталі на шляху S робот залишається в точці вивантаження: $pos_r := dst_S$. Взаємодія фрагментів відбувається через блокування маршрутів. Умови B_1 і B_3 мають той самий зміст, що й у варіанті 1. Умови B_0, B_2, B_4 на маршрутах до роботів і B_X на маршруті П → Х задає керування роботами К – правило вибору найближчого вільного робота. Його керуючі зв'язки на рисунку 2.2 позначено квадратами «К». Зміст блокувань наведено в таблиці 2.4.

Таблиця 2.4 – Блокування маршрутів у варіанті 2

Умова	Маршрут	Маршрут закритий, якщо	Де чекає запит
B_0	А → Т1	у точці А немає вільного робота або до М1 уже везуть деталі (чи їдуть по них) більше роботів, ніж там вільних місць	деталь на місці площадки А
B_1	Т1 → М1	усі 3 місця М1 зайняті	деталь у пристрої Т1: робот тримає її біля М1
B_2	М1 → Т2	у точці М1 немає вільного робота або до М2 уже везуть деталі (чи їдуть по них) більше роботів, ніж там вільних місць	деталь на місці верстата М1
B_3	Т2 → М2	усі 3 місця М2 зайняті	деталь у пристрої Т2: робот тримає її біля М2
B_4	М2 → Т3	у точці М2 немає вільного робота	деталь на місці верстата М2
B_X	П → Х	робота не обрано для холостого ходу за правилом (2.5)	робот у пристрої П

Для запису предикатів через стан елементів мережі введемо величини:

$$F_r(t) = \prod_{E\in\{\mathrm{T1,T2,T3,X}\}} \mathbf{1}\{K_{E,r}(t) = 0\},$$

$$V_P(t) = \sum_r F_r(t)\,\mathbf{1}\{pos_r(t) = P\},$$

$$h_P(t) = \sum_r \mathbf{1}\{K_{\mathrm{X},r}(t) \neq 0 \wedge target_r = P\},$$

$$f_i(t) = \sum_{j=1}^{3} \mathbf{1}\{K_{\mathrm{M}i,j}(t) = 0\},$$

$$u_S(t) = N_{\mathrm{T}S}(t) + h_{src_S}(t),$$

$$D_S(t) = \mathbf{1}\{u_S(t) > f_S(t)\},\ S = 1,2, \qquad D_3(t) = 0, \qquad (2.2)$$

операція ТА дала б такий самий результат .

Усе це можна записати компактно.

не визначені

де $F_r(t)$ дорівнює 1, якщо робот Rr вільний: не обслуговує і не тримає деталь та не їде порожнім; $V_P(t)$ – кількість вільних роботів у точці P; $h_P(t)$ – кількість роботів, що їдуть порожнім до точки P ($target_r$ – точка призначення холостого ходу); $f_i(t)$ – кількість вільних місць верстата Mi; $N_{\mathrm{T}S}(t)$ – кількість деталей у пристроях СМО TS (обслуговуваних або заблокованих); $u_S(t)$ – кількість роботів, які везуть деталь до верстата MS або їдуть по таку деталь. Тоді

$$B_0(S,t) = \mathbf{1}\{V_{\mathrm{A}}(t) = 0 \vee D_1(t) = 1\},$$
$$B_2(S,t) = \mathbf{1}\{V_{\mathrm{M1}}(t) = 0 \vee D_2(t) = 1\},$$
$$B_4(S,t) = \mathbf{1}\{V_{\mathrm{M2}}(t) = 0\}, \qquad (2.3)$$

а B_1 і B_3 – як у формулі (2.1). Якщо в точці src_S стоять кілька вільних роботів, деталь переходить до будь-якого з них, бо роботи в одній точці рівноцінні.

Умова $D_S(t) = 0$ визначає, коли вільний робот може взяти деталь до верстата: не можна брати деталь для верстата, до якого вже везуть деталі (чи їдуть по них) на одного робота більше, ніж там вільних місць. Кількість роботів при цьому не змінюється, їх три за умовою. Правило лише не дає трьом роботам одночасно стати з деталями біля заповненого верстата. Робот, який привіз деталь до заповненого верстата, тримає її і не може виконувати інших перевезень. Якщо всі три роботи візьмуть деталі для заповненого M2, оброблені деталі на M2

нікому вивезти на склад, місця на M2 не звільняються, і система зупиняється (взаємне блокування). У пробних прогонах без цього правила таке блокування в усіх п’яти прогонах наставало на 320-1172-й секунді, після чого жодна деталь на склад не надходила. Оскільки умова перевіряється перед тим, як деталь перейде до робота, завжди $u_S(t) \le f_S(t) + 1$, тобто біля кожного верстата з деталлю чекає не більше одного робота. Тоді заблокованими можуть бути щонайбільше два роботи, біля M1 і M2, а третій завжди може вивезти деталь з M2 на склад, бо цей маршрут не блокується. Місце на M2 звільняється, і рух відновлюється. У варіанті 1 до кожного верстата їздить лише свій робот, тому така умова виконується автоматично.

Керування роботами K після обробки всіх подій поточного моменту визначає, який вільний робот і до якої деталі рушає. Множина допустимих пар «робот – шлях»:

$$P(t) = \{(r, S):\ F_r = 1,\ pos_r \neq src_S,\ q_S > 0,\ D_S = 0\},$$
$$q_S(t) = w_S(t) - h_{src_S}(t) - V_{src_S}(t), \qquad (2.4)$$

де умови множини $P(t)$ перевіряються в момент t; $w_S(t)$ – кількість деталей, що чекають на робота в точці src_S, тобто заблокованих пристроїв СМО A, M1 або M2; $q_S(t)$ – кількість таких деталей, по які ще не їде і біля яких не стоїть вільний робот. З множини $P(t)$ обирається пара з найменшим холостим ходом:

$$(r^*, S^*) = \arg\min_{(r,S)\in P(t)} d(pos_r, src_S), \qquad (2.5)$$

де $d(pos_r, src_S)$ – час холостого ходу робота Rr із точки, де він стоїть, до точки завантаження src_S. Якщо однаково близьких пар кілька, обирається випадкова з них. Для обраного робота $B_X = 0$: його запит переходить з П у X з $target_{r^*} = src_{S^*}$. Вибір повторюється, доки множина $P(t)$ не стане порожньою. Робот, який уже стоїть у точці src_S, холостого ходу не виконує: деталь переходить до нього маршрутом до СМО TS, щойно блокування B_0, B_2 чи B_4 знімається. Отже, кожну деталь перевозить найближчий з вільних роботів, як вимагає умова завдання.

2.4 Часові параметри пристроїв

Час обслуговування і затримку звільнення пристроїв усіх СМО наведено в таблиці 2.5.

Таблиця 2.5 – Час обслуговування t і затримка звільнення $t_{\text{зв}}$ пристроїв мережі

СМО	t	$t_{\text{зв}}$	Що відбувається з деталлю і роботом
A	0	$t_{\text{зх}}$	деталь займає місце площадки від надходження до кінця захоплення
RS (варіант 1)	$t_{\text{зх}} + d_S$	$t_{\text{вив}} + d_S$	захоплення і переміщення з деталлю; після переходу деталі – вивільнення і повернення робота до src_S
TS (варіант 2)	$t_{\text{зх}} + d_S$	$t_{\text{вив}}$	захоплення і переміщення з деталлю; після переходу деталі – вивільнення, робот залишається в dst_S
M1	$t_{\text{вив}} + t_{\text{обр1}}$	$t_{\text{зх}}$	вивільнення і обробка; після переходу деталі до робота – захоплення
M2	$t_{\text{вив}} + t_{\text{обр2}}$	$t_{\text{зх}}$	те саме
W	$t_{\text{вив}}$	0	вивільнення на складі, після нього деталь залишає мережу
П	0	0	робот стоїть у точці pos_r або працює з деталлю
X	$d(pos_r, src_S)$	0	холостий хід робота до точки завантаження

Час обслуговування в СМО робота не включає блокування: якщо на верстаті немає місця, пристрій робота блокований, і деталь чекає в ньому. Отже, у варіанті 1 робот зайнятий однією деталлю протягом

$$t_{\text{R}S} + \tau_{\text{бл}} = t_{\text{зх}} + d_S + \tau_{\text{бл}} + t_{\text{вив}} + d_S,$$
$$\bar{t}_{\text{R1}} = 28 \text{ с}, \quad \bar{t}_{\text{R2}} = 30 \text{ с}, \quad \bar{t}_{\text{R3}} = 26 \text{ с}, \tag{2.6}$$

де $\tau_{\text{бл}} \geq 0$ – час блокування, $\bar{t}$ – середнє значення без блокування. Холостий хід у варіанті 1 виконується після вивільнення, тож деталь чекає на робота лише тоді, коли надійшла раніше, ніж він повернувся. У варіанті 2 до часу роботи з деталлю додається холостий хід із точки pos_r, де стоїть робот, до точки завантаження. Час переміщення між пунктами задано матрицею

$$D = \begin{array}{c} \\ \text{A} \\ \text{M1} \\ \text{M2} \\ \text{W} \end{array} \begin{array}{cccc} \text{A} & \text{M1} & \text{M2} & \text{W} \\ 0 & 6 & 13 & 18 \\ 6 & 0 & 7 & 12, \\ 13 & 7 & 0 & 5 \\ 18 & 12 & 5 & 0 \end{array} \tag{2.7}$$

з якої для кожного шляху отримуємо межі часу, протягом якого робот зайнятий деталлю без блокування:

$$\begin{cases} d(pos_r, \text{A}) + t_{\text{зх}} + 6 + t_{\text{вив}}, & 20 < t < 42 \text{ с}, \\ d(pos_r, \text{M1}) + t_{\text{зх}} + 7 + t_{\text{вив}}, & 21 < t < 37 \text{ с}, \\ d(pos_r, \text{M2}) + t_{\text{зх}} + 5 + t_{\text{вив}}, & 19 < t < 36 \text{ с}. \end{cases} \quad (2.8)$$

Найменші значення досягаються, коли робот уже стоїть у точці завантаження: наприклад, робот, що вивільнив деталь на M2, одразу забирає з M2 оброблену деталь на склад. Середній холостий хід у варіанті 2 заздалегідь невідомий, бо залежить від того, де стоять вільні роботи в момент вибору, тому його оцінюють за результатами імітації.

2.5 Числові параметри та генератори випадкових величин

Числові значення параметрів і вхідних змінних мережі наведено в таблицях 1.1 і 1.2 та на рисунках 2.1 і 2.2: інтервал надходження $\exp(40)$, час обробки на M1 $\text{norm}(60,10)$, на M2 $\exp(100)$, час захоплення і вивільнення $\text{unif}(8,1)$, по три місця на верстатах, переміщення 6, 7 і 5 с, необмежена черга площадки A. Усі випадкові величини незалежні, $t_{\text{зх}}$ і $t_{\text{вив}}$ генеруються окремо для кожної дії робота. Значення випадкових величин отримуються з рівномірно розподіленої на $(0;\ 1)$ величини ζ за формулами [2, с. 157-161]:

$$\exp(d)\colon\ x = -d\ln\zeta; \qquad \text{unif}(d,s)\colon\ x = d - s + 2s\zeta; \qquad \text{norm}(d,s)\colon\ x = d + s\,z, \quad (2.9)$$

де z – стандартна нормальна величина, отримана полярним методом Марсальї – Брея:

$$v_1 = 2\zeta_1 - 1, \quad v_2 = 2\zeta_2 - 1, \quad w = v_1^2 + v_2^2,$$
$$z_{1,2} = v_{1,2}\sqrt{\frac{-2\ln w}{w}}. \quad (2.10)$$

Пара ζ_1, ζ_2 генерується знову, доки не виконається умова $0 < w \leq 1$, а один розрахунок дає два значення z. Якщо $t_{\text{обр1}} \leq 0$, значення генерується повторно.

2.6 Стан мережі, правила функціонування та початковий стан

Стан мережі $S(t)$ у момент t описують змінні, визначені для елементів мережі МО в методичних рекомендаціях [3, с. 67-70]:

– t_0 – момент наступного надходження деталі;

– $K_{E,j}(t)$ і $t_{E,j}$ – стан j-го пристрою СМО E і момент завершення його обслуговування або затримки звільнення ($t_{E,j} = \infty$, якщо подія не очікується); під час обслуговування і блокування в пристрої є запит, під час затримки звільнення запиту немає;

– для кожного маршруту – заблоковані пристрої, що чекають на його відкриття, у порядку блокування;

– у варіанті 2 – $pos_r(t)$ і $target_r$: точка, де стоїть робот Rr, і точка призначення його холостого ходу.

Мережа функціонує за такими правилами:

1) деталь, що надійшла, займає вільне місце площадки А і одразу намагається перейти до робота;

2) після завершення обслуговування запит переходить у наступну СМО, якщо маршрут відкритий: займає вільний пристрій (на верстаті – вільне місце з найменшим номером, у СМО TS – пристрій вільного робота, що стоїть у точці src_S), а пристрій, з якого він вийшов, звільняється одразу або після затримки $t_{зв}$;

3) якщо маршрут закритий, пристрій переходить у стан 2, запит залишається в ньому; блокування знімається, щойно $B(S,t) = 0$, і першим виходить запит, заблокований раніше;

4) у варіанті 2 після обробки всіх подій поточного моменту керування роботами за правилом (2.5) відправляє вільних роботів у холостий хід;

5) події, які настають в один момент часу, обробляються у випадковому порядку, причому завершення обслуговування обробляються раніше за надходження нової деталі [3, с. 66].

У початковий момент $t = 0$ усі пристрої вільні, а роботи стоять у точках завантаження шляхів варіанта 1: R1 – в A, R2 – в M1, R3 – в M2. Для варіанта 2

прийнято той самий стан: $pos_1 = \text{A}$, $pos_2 = \text{M1}$, $pos_3 = \text{M2}$, запити роботів у СМО П. Перша деталь надходить у момент $\tau_{A,1}$.

2.7 Обчислення вихідних характеристик та вимоги до точності

Статистика збирається на інтервалі $[T_\text{п};\ T_\text{mod}]$ тривалістю $T = T_\text{mod} - T_\text{п}$, де T_mod – час моделювання, $T_\text{п}$ – тривалість перехідного періоду. Стан мережі змінюється лише в моменти подій t_k і між сусідніми подіями сталий. Тому середні за часом величини обчислюються як суми за інтервалами $\Delta t_k = t_{k+1} - t_k$ між послідовними подіями [3, с. 59]. Формули обчислення вихідних характеристик наведено в таблиці 2.6, де $L_A(t) = \sum_j \mathbf{1}\{K_{\text{A},j}(t) \neq 0\}$ – кількість зайнятих місць площадки А.

Таблиця 2.6 – Формули обчислення вихідних характеристик

Характеристика	Формула обчислення
Пропускна здатність, дет./год	$\Lambda = 3600\, N_W/T$, де N_W – кількість деталей, що залишили мережу за час T
Час проходження i-ї деталі, с	$\tau_i = t_i^W - t_i^A$ – від надходження на площадку А до кінця вивільнення на складі
Середній час проходження і його середнє квадратичне відхилення, с	$\bar{\tau} = \frac{1}{N_W}\sum_i \tau_i\,, \qquad s_\tau = \sqrt{\frac{1}{N_W - 1}\sum_i (\tau_i - \bar{\tau})^2}$
Розподіл часу проходження	гістограма відносних частот $h_k = N_k/N$, де N_k – кількість значень τ_i у k-му інтервалі; відповідність закону розподілу перевіряється за критерієм χ^2
Коефіцієнт використання робота Rr	$\rho_r = \frac{1}{T}\sum_k (1 - F_r(t_k))\ \Delta t_k$ – частка часу, коли хоча б один пристрій робота не вільний: робот працює з деталлю, тримає її, вивільняє, повертається або їде порожнім
Коефіцієнт використання верстата Mi	$\rho_{\text{M}i} = \frac{1}{3T}\sum_k \sum_{j=1}^{3} \mathbf{1}\{K_{\text{M}i,j}(t_k) \neq 0\}\,\Delta t_k$
Частка блокування верстата Mi	$\rho_{\text{M}i}^{\text{бл}} = \frac{1}{3T}\sum_k \sum_{j=1}^{3} \mathbf{1}\{K_{\text{M}i,j}(t_k) = 2\}\,\Delta t_k$
Максимальна місткість місця зберігання на ділянці прибуття, шт.	$L_{max} = \max_{t_k \in [T_\text{п};\ T_\text{mod}]} L_A(t_k)$
Середня кількість деталей на площадці А, од.	$\bar{L}_A = \frac{1}{T}\sum_k L_A\,(t_k)\,\Delta t_k$

У варіанті 1 $F_r(t) = \mathbf{1}\{K_{\text{R}r}(t) = 0\}$. Кожна вихідна характеристика оцінюється як середнє $\bar{x}$ за серією з p незалежних прогонів моделі з різними

початковими значеннями генераторів. Кількість прогонів визначається за нерівністю Чебишова [3, с. 81]:

$$p = \frac{\sigma^2}{\varepsilon^2(1-\beta)}, \qquad \beta = 0{,}95 \;\Rightarrow\; p = 20\left(\frac{\sigma}{\varepsilon}\right)^2, \tag{2.11}$$

де σ – середнє квадратичне відхилення характеристики за пілотною серією з 20 прогонів; ε – допустима похибка; β – довірча ймовірність. Для пропускної здатності, часу проходження і коефіцієнтів використання прийнято $\varepsilon = 0{,}02\,\bar{x}$, тобто 2% від середнього значення. Для максимальної місткості площадки, яка є цілим числом, точність задано абсолютною – 1-2 деталі. Тривалість перехідного періоду і кількість прогонів визначено в підрозділі 4.2.

2.8 Висновки до розділу

У розділі побудовано формалізовані моделі ГВС для обох способів закріплення роботів у термінах мережі масового обслуговування. Місця площадки, верстатів і складу та роботи представлено пристроями СМО зі станами «вільний», «зайнятий», «блокований». Захоплення і вивільнення деталі, повернення робота у варіанті 1 враховано затримкою звільнення пристроїв. У варіанті 1 кожен робот є окремою одноканальною СМО. У варіанті 2 на кожному шляху є триканальна СМО роботів, а положення роботів і їх холості ходи відстежує окремий фрагмент мережі. Умови «немає вільного робота» і «на верстаті немає місця» описано предикатами блокування маршрутів через стан елементів мережі. Для варіанта 2 сформульовано правило вибору найближчого вільного робота й умову, що виключає взаємне блокування роботів. Визначено змінні стану, правила функціонування, початковий стан, формули обчислення вихідних характеристик і вимоги до точності їх оцінювання.

3 АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ ІМІТАЦІЙНОЇ МОДЕЛІ СИСТЕМИ

У розділі описано алгоритм імітації, побудований за формалізованою моделлю розділу 2, і його програмну реалізацію: просування часу, просування стану мережі МО в часі, перевірку умов блокування, керування роботами, генерування випадкових величин і збір статистики. Алгоритм імітації власний: спеціалізовані бібліотеки імітації не використовуються. Повний код наведено в додатках Б і В, а також у репозиторії проєкту.

3.1 Обґрунтування вибору мови програмування

Для програмної реалізації обрано мову Python [4]. Вона дає змогу компактно описати елементи мережі МО класами, а модуль `dataclasses` – задати параметри і структури даних без службового коду. Python також має розвинені бібліотеки для статистичної обробки результатів. Бібліотеки NumPy [6] і SciPy [7] використано лише на етапі експериментального дослідження: для обчислення квантилів розподілів Стьюдента, Фішера і χ^2, оцінювання параметрів законів розподілу і дисперсійного аналізу. Графіки побудовано бібліотекою Matplotlib [8]. Сам алгоритм імітації використовує лише стандартну бібліотеку Python.

Недоліком Python є менша швидкодія порівняно з компільованими мовами C++ і Java. Для цієї моделі вона достатня: один прогін тривалістю 300 000 с модельного часу виконується приблизно за 1 с у варіанті 1 і за 2 с у варіанті 2. Серії незалежних прогонів виконуються паралельно в окремих процесах за допомогою модуля `concurrent.futures`. Коректність окремих частин алгоритму перевіряється автоматичними тестами (бібліотека pytest).

3.2 Структура програми

Програма є пакетом `fms` із модулями, наведеними в таблиці 3.1.

Таблиця 3.1 – Модулі програми

Модуль	Призначення
`rng.py`	генератори випадкових величин: окремі потоки чисел для кожного джерела випадковості, закони exp, unif, norm

Модуль	Призначення
`params.py`	вхідні змінні та параметри моделі, спосіб закріплення роботів
`elements.py`	елементи мережі МО: надходження `Create`, СМО `QS` з пристроями `Channel`, дуга з блокуванням `Route`, вихід `Exit`, а також запити мережі: деталь `Part` і робот `Robot`
`model.py`	мережа МО обох варіантів, предикати блокування B_0-B_4, керування роботами, алгоритм імітації, збір і обчислення вихідних характеристик
`experiments/`	експериментальне дослідження: верифікація, перехідний період, кількість прогонів, порівняння варіантів, межа стійкості, розподіл часу проходження, пробні запуски пропозицій
`cli.py`	командний інтерфейс для запуску прогонів та експериментів
`report_plots.py`	побудова рисунків за результатами експериментів

Варіант закріплення роботів задається параметром `strategy`: `dedicated` – варіант 1, `nearest` – варіант 2. Від нього залежать склад мережі (СМО R1-R3 або СМО Т1-Т3 і фрагмент руху роботів П, Х) і предикати блокування. Алгоритм імітації для обох варіантів однаковий, бо він працює з елементами мережі МО, а не з конкретними об’єктами системи.

3.3 Алгоритм імітації

Алгоритм імітації побудовано за алгоритмами 3.1 і 3.2 методичних рекомендацій [3, с. 65-72] з просуванням часу до найближчої події. Як і в мережі МО, подій лише дві: надходження деталі ззовні і завершення обслуговування в пристрої. Затримка звільнення теж завершується подією «завершення обслуговування», лише в пристрої вже немає запиту. Перехід деталі між СМО, блокування і розблокування маршрутів, а у варіанті 2 також вибір робота є змінами стану, які ці події спричиняють. Псевдокод алгоритму імітації має вигляд:

```
Алгоритм 3.1 Імітаційне моделювання мережі МО ГВС
вхід: вхідні змінні X, параметри P, поточний час t = 0, час
моделювання Tmod,
      перехідний період Tп
використовує перетворення стану мережі E(S, t) (алгоритм 3.2)
ініціалізувати стан мережі S: усі пристрої вільні (K = 0, t_E,j =
∞),
      t0 := τA,1, роботи в A, M1, M2
поки t < Tmod
    знайти момент найближчої події tnext := min(t0, min t_E,j)
    якщо Tп ∈ (t; tnext], оновити статистику на [t; Tп] і обнулити
статистику
```

```
    оновити статистику вихідних змінних Y на інтервалі [t; tnext]
    перемістити поточний час t := tnext
    викликати перетворення стану S := E(S, t)
кінець поки
обчислити статистику вихідних змінних Y за формулами таблиці 2.6
вивести вихідні змінні Y
```

Алгоритм 3.2 Перетворення стану мережі МО E(S, t)

```
вхід: стан мережі S, поточний момент t
для кожного пристрою (E, j), для якого t_E,j = t, у випадковому
порядку
    t_E,j := ∞
    якщо в пристрої немає запиту
        K_E,j := 0
    інакше якщо B(S, t) = 1 для маршруту з СМО E
        K_E,j := 2, запам'ятати пристрій у черзі блокування
маршруту
    інакше ПЕРЕХІД(E, j)
якщо t0 = t
    зайняти вільне місце j площадки A, t0 := t + τA
    якщо B0(S, t) = 1, то K_A,j := 2, інакше ПЕРЕХІД(A, j)
повторювати, доки стан мережі змінюється
    для кожного маршруту, для якого B(S, t) = 0
        ПЕРЕХІД для заблокованих пристроїв у порядку блокування
    варіант 2: поки P(t) ≠ ∅, обрати (r*, S*) за правилом (2.5),
        запит робота r* переходить з П у X:
        K_X,r* := 1, t_X,r* := t + d(pos_r*, src_S*)
вивести стан мережі S

ПЕРЕХІД(E, j): запит з пристрою (E, j) переходить у наступну СМО
E'
    h := t_зх або t_вив, якщо перехід супроводжує дія робота,
інакше h := 0
    обрати вільний пристрій j' СМО E': K_E',j' := 1, t_E',j' := t
+ t_E'(h)
    якщо t_зв,E(h) > 0: K_E,j := 1, t_E,j := t + t_зв,E(h), інакше
K_E,j := 0
```

Статистика оновлюється перед просуванням часу, тому кожен інтервал між подіями враховується зі станом, який був на ньому. Події, що настають одночасно, виконуються у випадковому порядку, бо це найбільш правильний підхід за відсутності явного пріоритету [3, с. 66]. Завершення обслуговування виконуються раніше за надходження нової деталі. Умови блокування перевіряються після обробки всіх подій моменту, коли відомий стан усіх пристроїв. Програмна реалізація головного циклу (метод `Model.run`, спрощено: прибрано виклики спостерігача, що використовуються для аналізу перехідного періоду):

```
def run(self, t_end, t_warmup=0.0):
    pending_reset = t_warmup > 0
    self.update_blockings(0.0)
    while True:
        t_next = self._next_event_time()        # найближча подія
        if pending_reset and t_warmup <= min(t_next, t_end):
            self._advance_stats(t_warmup)       # кінець перехідного періоду:
            self.reset_stats()                  # статистика обнуляється
            pending_reset = False
        if t_next > t_end:
            self._advance_stats(t_end)
            break
        self._advance_stats(t_next)             # статистика на [tcurr; t_next]
        self.transition(self.tcurr)             # перетворення стану E(S, t)
    return self
```

Моменти завершення обслуговування пристроїв зберігаються в календарі подій – купі, з якої найближча подія вибирається без перегляду всіх пристроїв. Перетворення стану за алгоритмом 3.2 виконує метод `Model.transition`:

```
def transition(self, t):
    due = []
    while self.calendar and self.calendar[0][0] == t:
        _, seq, ch = heapq.heappop(self.calendar)
        if ch.seq == seq and ch.tnext == t:
            due.append(ch)
    self.streams.ties.shuffle(due)              # одночасні події – випадково
    for ch in due:
        ch.qs.end_of_service(ch, t)             # завершення обслуговування
    if self.create.tnext == t:
        self.create.arrival(t)                  # надходження – останнім
    self.update_blockings(t)

def update_blockings(self, t):
    changed = True
    while changed:
        changed = False
        for route in self.routes:               # зняття блокувань, B(S, t) = 0
            changed |= route.unblock(t)
        if self.robot_control:                  # варіант 2: правило (2.5)
            changed |= self.robot_control(t)
```

Наприкінці перехідного періоду обнуляються лише накопичені статистики. Стан мережі (деталі на місцях площадки і верстатів, у пристроях роботів) при цьому не змінюється.

3.4 Події та зміна стану елементів мережі

Зміни стану елементів мережі під час подій наведено в таблиці 3.2 у вигляді, прийнятому в методичних рекомендаціях [3, с. 68-70].

Таблиця 3.2 – Зміна стану елементів мережі МО

Елемент	Змінні стану	Подія «надходження деталі», $t_0 = t$	Подія «завершення обслуговування в пристрої (E,j)», $t_{E,j} = t$
надходження ззовні `Create`	t_0	деталь займає вільне місце площадки A; $t_0 := t + \tau_A$	без змін
пристрій (E,j) СМО `QS`	$K_{E,j}$, $t_{E,j}$, запит у пристрої	для місця площадки: $K := 1$, $t := t$ (нульова затримка)	якщо запиту немає (кінець затримки звільнення): $K := 0$, $t := \infty$; якщо маршрут закритий: $K := 2$, $t := \infty$; якщо відкритий – перехід запиту в наступну СМО, $K := 1$, $t := t + t_{зв}$ або $K := 0$, $t := \infty$
пристрій СМО-приймача	$K_{E',j'}$, $t_{E',j'}$	без змін	при переході запиту: $K := 1$, $t := t + t_{E'}(h)$
блокування маршруту `Route`	$B(S,t)$, черга заблокованих пристроїв	оновлення $B(S,t)$	оновлення $B(S,t)$; якщо $B = 0$, заблоковані пристрої в порядку блокування переводять запит у наступну СМО в момент t
керування роботами (варіант 2)	pos_r, $target_r$	вибір пар за правилом (2.5), перехід запитів роботів з П у Х	те саме; у кінці холостого ходу $pos_r := target_r$, у кінці вивільнення $pos_r := dst_S$
вихід `Exit`	N_W, масив τ_i	без змін	кінець вивільнення на складі: $N_W := N_W + 1$, $\tau_i := t - t_i^A$

немає такого елементу у фармалізмі мережі МО, усі управління прописані блокуваннями маршрути

Подію «завершення обслуговування» обробляє метод `QS.end_of_service`. Якщо маршрут з СМО закритий, пристрій переходить у стан 2, а запит залишається в ньому. Перехід запиту виконує метод `Route.transfer`. Він генерує тривалість захоплення або вивільнення h і передає її обом СМО: пристрій-джерело звільняється із затримкою $t_{зв}(h)$, пристрій-приймач починає обслуговування тривалістю $t(h)$ (таблиця 2.5):

```
def end_of_service(self, ch, t):                # QS: подія в пристрої ch
    ch.tnext = INF
    if ch.request is None:                      # кінець затримки звільнення
        self._vacated(ch, t)                    # K := 0
        return
    if self.route.blocked():                    # B(S, t) = 1
        self.set_state(ch, BLOCKED, t)          # K := 2, запит лишається
        self.route.waiting.append(ch)
    else:
        self.route.transfer(ch, t)

def transfer(self, ch, t):                      # Route: перехід src -> dst
    req = ch.request
    h = self.model.handling_time() if self.handling else 0.0   # t_зх, t_вив
    dst_ch = self.select(req) if self.select else None
    self.src.vacate(ch, t, h)                   # K := 1 на t_зв(h) або K := 0
    self.dst.accept(req, t, h, dst_ch)          # K := 1, t := t + t(h)
```

```
def unblock(self, t):                          # Route: зняття блокування
    moved = False
    while self.waiting and not self.blocked():
        self.transfer(self.waiting.popleft(), t)
        moved = True
    return moved
```

3.5 Блокування маршрутів і керування роботами

Кожен маршрут мережі (об'єкт `Route`) містить предикат блокування, який перевіряється в момент завершення обслуговування і під час оновлення блокувань. Предикати реалізовано окремими методами моделі, названими так само, як у формулах (2.1) і (2.3). СМО зберігає кількість своїх пристроїв у кожному стані `n[K]`, тому умова «усі пристрої СМО не вільні» перевіряється як `n[FREE] == 0`:

```
# варіант 1, (2.1)
def B0(self): return self.T[1].n[FREE] == 0    # K_R1 != 0
def B2(self): return self.T[2].n[FREE] == 0    # K_R2 != 0
def B4(self): return self.T[3].n[FREE] == 0    # K_R3 != 0

# обидва варіанти, (2.1): усі місця верстата не вільні
def B1(self): return self.M1.n[FREE] == 0
def B3(self): return self.M2.n[FREE] == 0

# варіант 2, (2.3): у точці src_S немає вільного робота або D_S = 1
def B0_n(self): return not self.free_at("A") or self.D(1)
def B2_n(self): return not self.free_at("M1") or self.D(2)
def B4_n(self): return not self.free_at("M2")

def is_free(self, robot):                      # F_r, (2.2)
    return all(ch.K == FREE for ch in self.robot_channels[robot.id])

def u(self, S):                                # u_S, (2.2)
    return self.T[S].requests + self.heading(SRC[S])

def D(self, S):                                # D_S, (2.2)
    return S != 3 and self.u(S) > self.M[S].n[FREE]
```

Тут `robot_channels[r]` – пристрої робота Rr у СМО T1, T2, T3 і X, `free_at(P)` – вільні роботи в точці P, `heading(P)` – кількість роботів, що їдуть порожнім до точки P. Маршрут з'єднує СМО так само, як дуга на рисунках 2.1 і 2.2, наприклад:

```
Route(self, self.T[1], self.M1, self.B1, "B1", handling="вивільнення")
Route(self, self.M1, self.T[2], self.B2, "B2", handling="захоплення",
      select=self._robot_for(2))
```

Параметр `select` задає правило вибору пристрою СМО-приймача. У СМО TS варіанта 2 це пристрій вільного робота, що стоїть у точці src_S; якщо таких

роботів кілька, обирається випадковий. Керування роботами за формулами (2.4) і (2.5) реалізує метод `Model.control`. Він викликається після обробки всіх подій моменту і відправляє найближчого вільного робота до деталі, по яку ще ніхто не їде (код наведено без рядків запису протоколу і статистики):

Представлено фрагментом мережі МО і мало бути реалізовано як мережа МО

```
def control(self, t):
    moved = False
    while True:
        best = None
        for r in self.robots:
            if not self.is_free(r):            # F_r = 0
                continue
            for S in (1, 2, 3):
                src = SRC[S]
                q = (self.waiting(S) - self.heading(src)
                     - len(self.free_at(src)))          # q_S(t)
                if r.pos == src or q <= 0 or self.D(S):  # (r, S) не в P(t)
                    continue
                key = (self.params.distance(r.pos, src),
                       self.streams.ties.random())
                if best is None or key < best[0]:        # (2.5)
                    best = (key, r, S)
        if best is None:
            return moved
        (d, _), r, S = best
        r.target = SRC[S]                      # B_X = 0 для обраного робота
        ch = self.P.channels[r.id - 1]
        self.route_PX.waiting.remove(ch)
        self.route_PX.transfer(ch, t)          # запит робота: П -> Х
        moved = True
```

У ключі вибору друге значення – рівномірно розподілена випадкова величина: вона обирає випадкову пару, якщо кілька пар мають однаковий холостий хід.

3.6 Генератори випадкових величин

Базову рівномірно розподілену на (0; 1] величину ζ отримує генератор Mersenne Twister з модуля `random` стандартної бібліотеки. Усі інші закони обчислюються власними формулами (2.9) і (2.10). Кожне джерело випадковості моделі має окремий потік чисел із власним зерном: інтервали надходження, обробка на М1, обробка на М2, захоплення і вивільнення, а також вибір серед одночасних подій. Завдяки цьому варіанти 1 і 2 з однаковим зерном отримують однакові послідовності надходжень і часів обробки (спільні випадкові числа). Різниця результатів тоді пояснюється способом закріплення роботів, а не випадковістю. Фрагмент класу потоку `Stream`:

```
def zeta(self):                          # рівномірна на (0; 1]
    return 1.0 - self._rnd.random()

def exponential(self, mean):             # exp(d): x = -d ln ζ
    return -mean * math.log(self.zeta())

def uniform(self, center, half_width):   # unif(d, s): x = d - s + 2sζ
    return center - half_width + 2.0 * half_width * self._rnd.random()

def standard_normal(self):               # полярний метод Марсальї – Брея
    if self._spare_normal is not None:   # друге значення попередньої пари
        value, self._spare_normal = self._spare_normal, None
        return value
    while True:
        a1 = 2.0 * self._rnd.random() - 1.0
        a2 = 2.0 * self._rnd.random() - 1.0
        s = a1 * a1 + a2 * a2
        if 0.0 < s <= 1.0:
            break
    beta = math.sqrt(-2.0 * math.log(s) / s)
    self._spare_normal = a2 * beta
    return a1 * beta
```

Час обробки на M1 генерується як $\mathrm{norm}(60{,}10)$ з повторним генеруванням від'ємних значень. Правильність генераторів перевірено автоматичними тестами: для вибірки з 200 000 значень середнє і середнє квадратичне відхилення збігаються з теоретичними з похибкою до 1 і 2% відповідно.

3.7 Збір статистики

Статистика збирається методом `update_stats(t1, t2)`, який викликається перед кожним просуванням часу з моменту t_1 у момент t_2. Стан мережі на інтервалі $[t_1;\ t_2)$ сталий, тому до накопичених сум додаються добутки на тривалість інтервалу, як у формулах таблиці 2.6:

– СМО верстатів і роботів для кожного пристрою накопичує тривалість станів $K = 0$, 1 і 2; у стані 1 окремо враховується частина, що припадає на захоплення чи вивільнення, бо момент його завершення відомий;

– для площадки А накопичуються сума $\sum_k L_A\,(t_k)\,\Delta t_k$, розподіл часу за кількістю деталей і найбільша кількість деталей L_{max};

– для коефіцієнта використання робота підсумовується час, коли не вільний хоча б один його пристрій у СМО перевезень і холостого ходу.

За станами пристроїв робота визначається і структура його часу: стан 1 з деталлю – захоплення і переміщення з деталлю, стан 2 – очікування з деталлю

біля заповненого верстата, затримка звільнення – вивільнення і повернення (варіант 1), пристрій СМО X – холостий хід (варіант 2).

Для часу проходження кожна деталь зберігає момент надходження і момент завершення вивільнення на складі, а також моменти готовності до перевезення і початку захоплення на кожному шляху. Наприкінці прогону функція `collect_results` за формулами таблиці 2.6 обчислює всі вихідні характеристики за період $[T_{\text{п}};\ T_{\text{mod}}]$: пропускну здатність, середнє, середнє квадратичне відхилення і квантилі часу проходження, коефіцієнти використання та частки станів верстатів і роботів, а також L_{max}, $\bar{L}_A$ і кількість деталей у системі в кінці прогону. Масив часів проходження використовується для побудови гістограми та перевірки закону розподілу.

3.8 Опис класів та UML-діаграма класів

Класи програми та їх призначення наведено в таблиці 3.3, UML-діаграму [9] класів – на рисунку 3.1.

Таблиця 3.3 – Класи програми

Клас	Призначення	Вміст класу
`Params`	вхідні змінні та параметри моделі	закони розподілу, кількість місць і роботів, розташування пунктів, спосіб закріплення; `distance`, `robot_routes`
`Dist`	закон розподілу в позначеннях методичних рекомендацій	вид закону і параметри; `sample(stream)`
`Stream`, `RandomStreams`	потоки випадкових чисел, окремий для кожного джерела випадковості	`exponential`, `uniform`, `normal`, `shuffle`, `choice`
`Model`	мережа МО і алгоритм імітації	СМО, маршрути, календар подій, роботи; предикати `B0`-`B4`, керування роботами `control`; `run`, `transition`, `update_blockings`, `results`
`Create`	надходження деталей ззовні	момент наступного надходження; `arrival`
`QS`	СМО з m ідентичних пристроїв без черги (m може бути необмеженим)	пристрої, час обслуговування і затримка звільнення, маршрут з СМО, кількість пристроїв у кожному стані; `accept`, `end_of_service`, `vacate`, `update_stats`
`Channel`	пристрій СМО	стан K, момент завершення обслуговування, запит у пристрої

Клас	Призначення	Вміст класу
`Route`	дуга мережі з блокуванням	СМО-джерело і СМО-приймач, предикат $B(S,t)$, черга заблокованих пристроїв; `transfer`, `unblock`
`Exit`	вихід з мережі	`accept` фіксує час проходження деталі
`Part`, `Robot`	запити мережі: деталь і робот	моменти подій деталі; положення робота і точка призначення холостого ходу

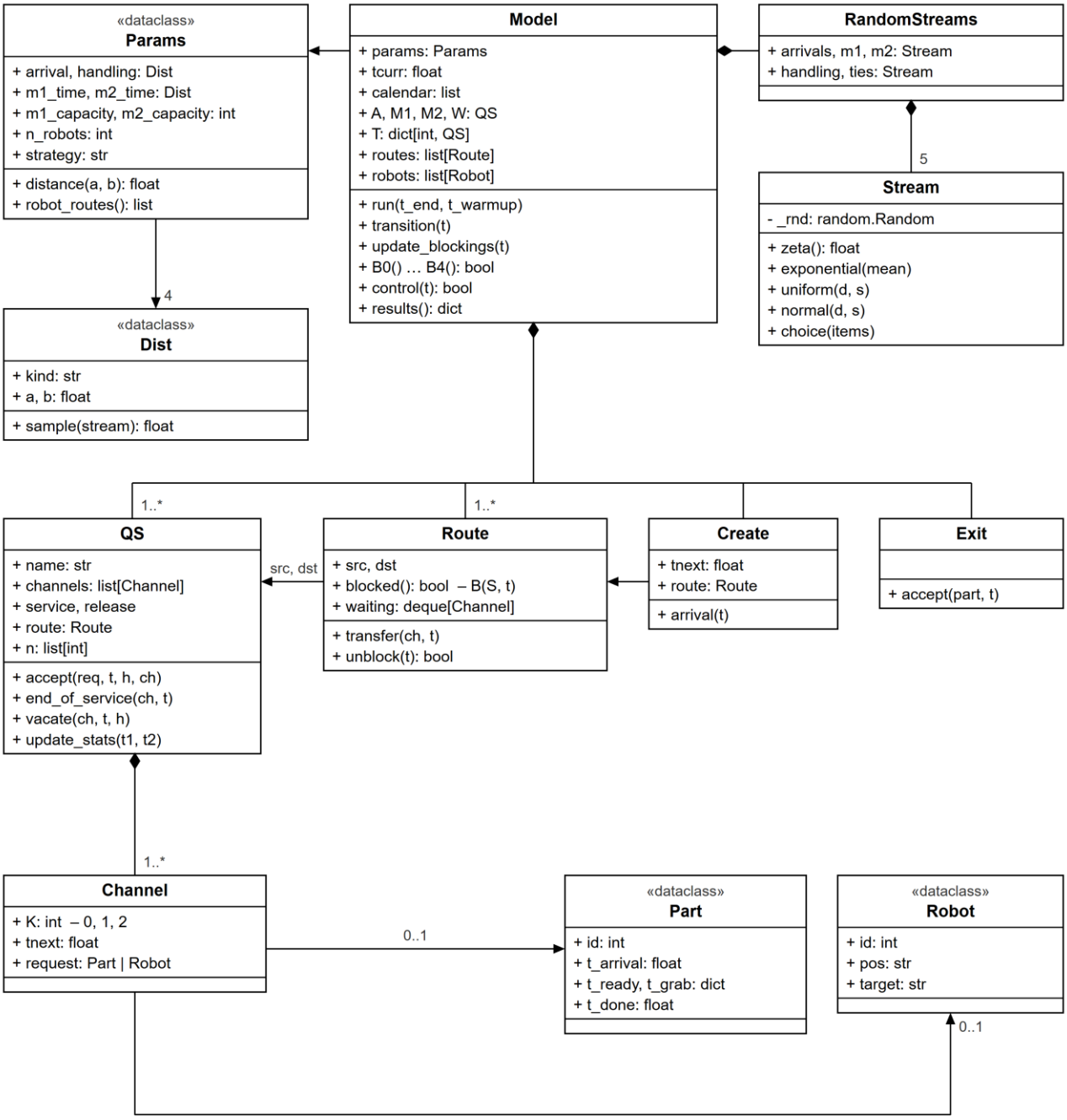


Рисунок 3.1 – UML-діаграма класів імітаційної моделі

Модель складається з об’єктів класів `Create`, `QS`, `Route` і `Exit`, тобто з елементів мережі МО. Площадка A, верстати, склад, СМО роботів і СМО

фрагмента руху роботів є об’єктами одного класу `QS` і відрізняються лише кількістю пристроїв, часом обслуговування і затримкою звільнення. Тому головний цикл однаково звертається до будь-якого пристрою: знаходить момент найближчої події, виконує подію і оновлює статистику.

3.9 Верифікація моделі

Коректність алгоритму імітації доведено верифікацією, яку виконують варіюванням вхідних змінних моделі. Розглянуто 9 сценаріїв, наведених у таблиці 3.4. Кожен сценарій змінює одну вхідну змінну або групу змінних відносно базових значень (таблиця 1.2), і для кожного сформульовано очікувану реакцію моделі. У таблиці k_d – множник часу переміщення роботів (базове значення 1), m_2 – кількість місць на верстаті М2.

Таблиця 3.4 – Сценарії верифікації: змінені вхідні змінні та очікувана реакція моделі

№	Змінена вхідна змінна	Очікувана реакція моделі
1	базові значення (таблиця 1.2)	базовий прогін для порівняння
2	$\tau_A = \exp(80)$	система недовантажена: $\Lambda \approx 3600/80 = 45$ дет./год, черга на площадці мала, коефіцієнти використання менші, ніж у базовому прогоні
3	$\tau_A = \exp(30)$	перевантаження сильніше: Λ не зростає, бо її обмежує М2; черга і час проходження більші, ніж у базовому прогоні
4	$t_{\text{обр2}} = \exp(50)$	М2 перестає бути вузьким місцем: $\Lambda \approx 90$ дет./год, черга не накопичується, час проходження різко менший
5	$t_{\text{обр1}} = \text{norm}(120{,}10)$	вузьким місцем стає М1: $\rho_{\text{М1}} \approx 1$, Λ менша, ніж у базовому прогоні
6	$m_2 = 6$	як у сценарії 4: $\Lambda \approx 90$ дет./год, черга не накопичується
7	$t_{\text{зх}},\ t_{\text{вив}} = \text{unif}(2;\ 0{,}5)$	роботи швидші: їх коефіцієнт використання менший, Λ більша
8	$k_d = 3$	роботи повільніші: їх коефіцієнт використання більший, Λ менша, час проходження більший
9	усі часи детерміновані: $\tau_A = 300$, $t_{\text{обр1}} = 60$, $t_{\text{обр2}} = 100$, $t_{\text{зх}} = t_{\text{вив}} = 8$ с	$\Lambda = 3600/300 = 12$ дет./год; у варіанті 1 кожна деталь проходить систему за 226 с, у варіанті 2 – перші три за 226, 232 і 239 с, наступні за 244 с (розрахунок у підрозділі 3.9)

Для кожного сценарію і кожного варіанта виконано 5 прогонів тривалістю $T_{\text{mod}} = 100\,000$ с модельного часу. Статистика збиралася після перехідного

періоду $T_{\text{п}} = 10\,000$ с. Змінні, не зазначені в таблиці 3.4, мали базові значення, а для сценарію 9 базовими були час переміщення і кількість місць. Середні за 5 прогонами значення вихідних змінних наведено в таблицях 3.5 і 3.6, де $\bar{\rho}_{\text{R}}$ – середній коефіцієнт використання роботів, $N_{\text{кін}}$ – кількість деталей у системі в кінці прогону.

Таблиця 3.5 – Результати верифікації, варіант 1

№	Λ, дет./год	$\bar{\tau}$, с	L_{max}, шт.	$\bar{L}_A$, од.	ρ_{M1}	ρ_{M2}	$\bar{\rho}_{\text{R}}$	$N_{\text{кін}}$, шт.
1	83,1	4800	225,4	114,9	0,998	0,957	0,865	225,6
2	44,6	257	5,8	0,2	0,360	0,495	0,370	2,6
3	83,2	17 137	1044,4	569,9	1,000	0,958	0,866	1050,8
4	90,8	275	16,2	2,0	0,761	0,648	0,738	7,2
5	71,7	11 450	537,0	285,0	1,000	0,816	0,759	541,4
6	90,8	308	14,4	1,5	0,731	0,557	0,720	7,6
7	91,0	588	36,2	8,2	0,873	0,894	0,675	12,2
8	59,0	19 075	887,4	479,0	1,000	0,752	0,918	892,2
9	12,0	226	1,0	0,0	0,084	0,129	0,093	1,0

Таблиця 3.6 – Результати верифікації, варіант 2

№	Λ, дет./год	$\bar{\tau}$, с	L_{max}, шт.	$\bar{L}_A$, од.	ρ_{M1}	ρ_{M2}	$\bar{\rho}_{\text{R}}$	$N_{\text{кін}}$, шт.
1	76,6	8668	404,2	212,9	0,922	0,906	0,911	404,0
2	44,5	271	6,6	0,4	0,363	0,493	0,389	2,8
3	76,6	20 215	1228,4	672,7	0,922	0,906	0,911	1234,0
4	91,0	407	26,4	5,0	0,776	0,644	0,833	9,8
5	67,6	13 838	650,2	346,3	0,971	0,777	0,784	653,8
6	91,0	374	20,0	3,2	0,734	0,526	0,788	8,8
7	89,7	1426	71,0	29,0	0,941	0,918	0,781	51,8
8	54,2	22 000	1020,2	552,8	0,773	0,767	0,980	1024,8
9	12,0	244	1,0	0,1	0,084	0,129	0,093	1,0

Навантеження не близьке до 1, але спостерігається накопичення

Реакція моделі на зміну вхідних змінних в обох варіантах відповідає очікуваній:

1) у базовому прогоні черга на площадці накопичується: L_{max} і $N_{\text{кін}}$ досягають сотень деталей, а пропускна здатність (83,1 і 76,6 дет./год) менша за інтенсивність надходження 90 дет./год, тобто система перевантажена;

2) при exp(80) пропускна здатність дорівнює інтенсивності надходження 45 дет./год, черга мала, коефіцієнти використання зменшились приблизно вдвічі;

3) при $\exp(30)$ пропускна здатність не змінилась, а черга і час проходження зросли, тобто пропускну здатність обмежує обладнання, а не потік деталей;

4) прискорення обробки на М2 удвічі або 6 місць на М2 (сценарії 4 і 6) роблять систему стійкою: пропускна здатність близька до 90 дет./год, черга не накопичується; отже, вузьким місцем базової системи є верстат М2; незначне перевищення 90 дет./год пояснюється тим, що за час спостереження на склад надходять і деталі, які були в системі на момент $T_{\text{п}}$;

5) при подвоєному часі обробки на М1 коефіцієнт використання М1 наближається до 1 (1,000 і 0,971), а пропускна здатність зменшується до 71,7 і 67,6 дет./год;

6) прискорення захоплення і вивільнення зменшує завантаження роботів і збільшує пропускну здатність, а потроєння часу переміщення діє навпаки; у варіанті 2 роботи при цьому завантажені майже повністю ($\bar{\rho}_{\text{R}} = 0{,}980$);

7) при детермінованих часах (сценарій 9) пропускна здатність дорівнює $3600/300 = 12$ дет./год, а час проходження – точно 226 с у варіанті 1 і 244 с у варіанті 2.

Результат сценарію 9 збігається з розрахунком. Перша деталь проходить систему за $8 + 6 + 8 + 60 + 8 + 7 + 8 + 100 + 8 + 5 + 8 = 226$ с, бо роботи стоять у точках завантаження. У варіанті 1 кожен робот після вивільнення повертається до точки завантаження за 6, 7 або 5 с, тобто задовго до надходження наступної деталі через 300 с. Тому кожна наступна деталь застає свого робота на місці і теж проходить систему за 226 с. У варіанті 2 друга і третя деталі проходять систему за 232 і 239 с, бо найближчі вільні роботи стоять у М1 і М2. Далі всі роботи збираються на складі, і кожну деталь перевозить один робот: холостий хід зі складу до А триває 18 с, а перевезення М1 → М2 і М2 → W він виконує без холостого ходу. Тому час проходження становить $226 + 18 = 244$ с. Ці значення перевіряються автоматичним тестом.

Додатково правильність алгоритму перевіряють 23 автоматичні тести, зокрема:

– закон збереження: кількість деталей, що надійшли, дорівнює сумі доставлених на склад і тих, що перебувають у пристроях мережі;

– стан кожного пристрою набуває лише значень 0, 1, 2, а лічильники пристроїв у кожному стані збігаються з фактичними станами;

– на верстаті ніколи не буває більше трьох зайнятих місць;

– система не зависає при сильному перевантаженні ($\exp(15)$);

– у варіанті 2 біля кожного верстата чекає не більше одного робота з деталлю, а $u_S \le f_S + 1$;

– у варіанті 1 кожен робот виконує перевезення лише свого шляху і звільняється після повернення до точки завантаження, у варіанті 2 – виконує перевезення щонайменше двох шляхів, а на деталь призначається найближчий вільний робот;

– однакове зерно генераторів дає однаковий результат.

Правильність послідовності подій підтверджує протокол подій, у якому для кожної події вказано момент часу, пристрій мережі, зміст події і стан мережі після неї. Фрагмент протоколу для варіанта 2 на початку моделювання:

```
14.788  T1:R1  деталь #1: перехід T1 -> M1, вивільнення 8.810
14.788  T1:R1  затримка звільнення до 23.597
14.788  M1.1   деталь #1: обслуговування до 78.547
16.321  Create надходження деталі #2
16.321  A.1    деталь #2: маршрут A -> T1 заблоковано (B0)
16.321  П:R2   холостий хід M1 -> A (6 с) до деталі шляху 1
        стан   A=1 M1[100] M2[000] T1[100] T2[000] T3[000] X[010]
               R1@A R2@M1 R3@M2
22.321  X:R2   кінець холостого ходу, робот у A
22.321  A.1    деталь #2: маршрут A -> T1 відкрито (B0 = 0)
22.321  A.1    деталь #2: перехід A -> T1, захоплення 8.279
```

Деталь №1 перейшла на верстат M1, робот R1 вивільняє її (пристрій T1:R1 у затримці звільнення). На момент надходження деталі №2 у точці A немає вільного робота, тому маршрут A → T1 заблоковано ($B_0 = 1$), і деталь чекає на місці площадки. Вільні роботи R2 (у точці M1, холостий хід 6 с) і R3 (у точці M2, 13 с). Керування роботами відправляє найближчого, R2, що відповідає правилу

(2.5). Коли R2 прибуває в A, блокування знімається, і деталь переходить до нього. Протоколи подій для обох варіантів наведено в додатку А.

3.10 Висновки до розділу

У розділі розроблено власний алгоритм імітації мережі МО ГВС з просуванням часу до найближчої події і реалізовано його мовою Python. Подіями алгоритму є лише надходження деталі і завершення обслуговування в пристрої, а стан мережі описується станом її елементів, як визначено у формалізмі мережі МО. Програма складається з класів, що відповідають елементам мережі: надходженню, СМО з пристроями, дугам з блокуванням і виходу. Предикати блокування і керування роботами реалізовано безпосередньо за формулами розділу 2. Обидва варіанти реалізовано тим самим алгоритмом, вони відрізняються лише складом мережі і предикатами блокування. Наведено опис класів і UML-діаграму класів. Верифікація варіюванням вхідних змінних у дев’яти сценаріях, детермінований сценарій з точно розрахованим результатом і автоматичні тести підтверджують коректність алгоритму.

4 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ МОДЕЛІ

У розділі визначено мету і план експериментального дослідження та умови проведення одного експерименту: тривалість перехідного періоду і кількість прогонів. Способи закріплення роботів порівняно методом дисперсійного аналізу. Досліджено межу стійкості системи, коефіцієнти використання обладнання, закон розподілу часу проходження деталей і місткість площадки прибуття. Фактором названо вхідну змінну або модифікацію моделі, вплив якої досліджується, а відгуком – вихідну змінну, що досліджується.

4.1 Мета і план експериментального дослідження

Мету використання моделі визначає завдання: знайти спосіб закріплення роботів до операцій переміщення, за якого пропускна здатність ГВС найбільша, і для кожного способу визначити розподіл часу проходження деталей, коефіцієнти використання роботів і верстатів та максимальну місткість місця зберігання на ділянці прибуття. Головним відгуком моделі є пропускна здатність Λ. Додаткові відгуки – час проходження τ, коефіцієнти використання ρ і максимальна кількість деталей на площадці L_{max}.

Фактором є спосіб закріплення роботів. Це якісний фактор із двома рівнями: варіант 1 і варіант 2. Вплив такого фактора оцінюють дисперсійним аналізом, який показує, чи спричинена різниця відгуків зміною фактора, чи випадковістю [3, с. 82-85]. Результат одного експерименту є середнім значенням відгуку за серією з p незалежних прогонів при фіксованих вхідних змінних. Для часу проходження, крім середнього і середнього квадратичного відхилення, виконано ідентифікацію закону розподілу.

Умови експериментів обрано за результатами верифікації (підрозділ 3.9). За умови завдання, тобто при надходженні exp(40) або 90 дет./год, система перевантажена в обох варіантах: пропускна здатність менша за інтенсивність надходження, і черга на площадці А зростає необмежено. У такому режимі середня довжина черги і середній час проходження залежать від часу моделювання, тобто сталих значень не мають. Пропускна здатність, навпаки,

досягає найбільшого можливого значення і стабілізується. Тому способи закріплення порівнюються за пропускною здатністю і коефіцієнтами використання за умови завдання $\exp(40)$. Розподіл часу проходження і місткість площадки мають сенс лише в стійкому режимі. Для їх визначення інтервал надходження змінено на $\exp(60)$, тобто 60 дет./год, що менше за межу стійкості обох варіантів (підрозділ 4.4). Таку зміну числових характеристик моделі методичні рекомендації передбачають для випадку, коли сталий режим не досягається [3, с. 80]. Додатково як кількісний фактор досліджено інтервал надходження деталей. Залежність пропускної здатності від нього показує, за якого навантаження кожен варіант ще справляється з потоком деталей.

План експериментів наведено в таблиці 4.1. В усіх серіях прогін з номером i для обох варіантів має однакові початкові значення генераторів, тобто варіанти отримують спільні випадкові числа (підрозділ 3.6).

Таблиця 4.1 – План експериментального дослідження

№	Експеримент	Інтервал надходження	T_{mod}, с	$T_{\text{п}}$, с	Кількість прогонів	Відгук
1	тривалість перехідного періоду	$\exp(40)$; $\exp(60)$	300 000; 1 500 000	0	5	Λ, $\bar{\tau}$, $\bar{L}_A$, ρ_{M2}, $\bar{\rho}_{\text{R}}$
2	кількість прогонів (пілотна серія)	$\exp(40)$; $\exp(60)$	300 000; 1 500 000	60 000; 50 000	20	Λ, $\bar{\tau}$, L_{max}, ρ_{M2}, $\bar{\rho}_{\text{R}}$
3	порівняння способів закріплення роботів	$\exp(40)$; $\exp(60)$	300 000; 1 500 000	60 000; 50 000	30	усі вихідні змінні
4	межа стійкості	від $\exp(34)$ до $\exp(80)$, 19 значень	300 000	60 000	10	Λ, $N_{\text{кін}}$
5	розподіл часу проходження	$\exp(60)$	1 500 000	50 000	20	τ
6	пробні запуски пропозицій (підрозділ 5.2)	$\exp(40)$	300 000	60 000	20	Λ, $\bar{\tau}$, L_{max}, ρ

Час моделювання, перехідний період і кількість прогонів у серіях 2-6 обрано за результатами серій 1 і 2 (підрозділ 4.2).

4.2 Тривалість перехідного періоду та кількість прогонів

Перехідним є період, протягом якого модель виходить на сталі значення відгуків. На початку моделювання система порожня, тому значення, накопичені

в цей період, зміщують оцінки відгуків. Тривалість перехідного періоду визначено за п’ятьма пробними прогонами з різними початковими значеннями генераторів, тоді як методичні рекомендації вимагають щонайменше чотири. Для кожного прогону i з кроком 1000 с при $\exp(40)$ і 5000 с при $\exp(60)$ обчислювалось середнє значення відгуку $\bar{y}_i(t)$ на інтервалі $[0;\ t]$. Тривалістю перехідного періоду вважався найменший момент T, після якого середні значення всіх прогонів відрізняються менше ніж на задану точність ε [3, с. 80]:

$$T\colon\quad \forall i,j,\ \ \forall t \geq T\colon\ \ |\bar{y}_i(t) - \bar{y}_j(t)| < \varepsilon. \tag{4.1}$$

Умова перевіряється для всіх $t \geq T$, а не лише в момент T, щоб випадковий збіг кривих до виходу на сталий рівень не прийняти за кінець перехідного періоду. Для характеристик, які усереднюються за часом (ρ, $\bar{L}_A$), середнє обчислюється з урахуванням нерівних інтервалів просування часу:

$$\bar{y}_i(t) = \frac{\sum_{s=1}^{n} y_{is}\ \Delta t_s}{\sum_{s=1}^{n} \Delta\, t_s}, \qquad t = \sum_{s=1}^{n} \Delta\, t_s, \tag{4.2}$$

де Δt_s – інтервали, на які просувається час в алгоритмі імітації; y_{is} – значення величини на інтервалі s. Пропускна здатність на момент t дорівнює $3600\, N_W(t)/t$, середній час проходження – середньому за деталями, що надійшли на склад до моменту t. Точність ε задано як 5% від рівня відгуку, тобто від середнього за п’ятьма прогонами значення в кінці моделювання. Результати наведено в таблиці 4.2, динаміку пропускної здатності при $\exp(40)$ і середнього часу проходження при $\exp(60)$ – на рисунках 4.1 і 4.2.

Таблиця 4.2 – Тривалість перехідного періоду за умовою (4.1), 5 прогонів, ε = 5% від рівня відгуку

Відгук	Інтервал надходження	Варіант 1: рівень	Варіант 1: T, тис. с	Варіант 2: рівень	Варіант 2: T, тис. с
Λ, дет./год	$\exp(40)$	83,1	58	76,4	20
Λ, дет./год	$\exp(60)$	59,8	140	59,8	140
$\bar{\tau}$, с	$\exp(40)$	12 005	не досягається	22 936	не досягається
$\bar{\tau}$, с	$\exp(60)$	299	605	336	1480
$\bar{L}_A$, од.	$\exp(40)$	292,8	не досягається	565,7	не досягається
$\bar{L}_A$, од.	$\exp(60)$	0,67	не досягається	1,11	не досягається

Відгук	Інтервал надходження	Варіант 1: рівень	Варіант 1: T, тис. с	Варіант 2: рівень	Варіант 2: T, тис. с
ρ_{M2}	exp(40)	0,956	16	0,906	33
ρ_{M2}	exp(60)	0,676	225	0,678	230
$\bar{\rho}_R$	exp(40)	0,864	35	0,911	13
$\bar{\rho}_R$	exp(60)	0,530	230	0,586	275

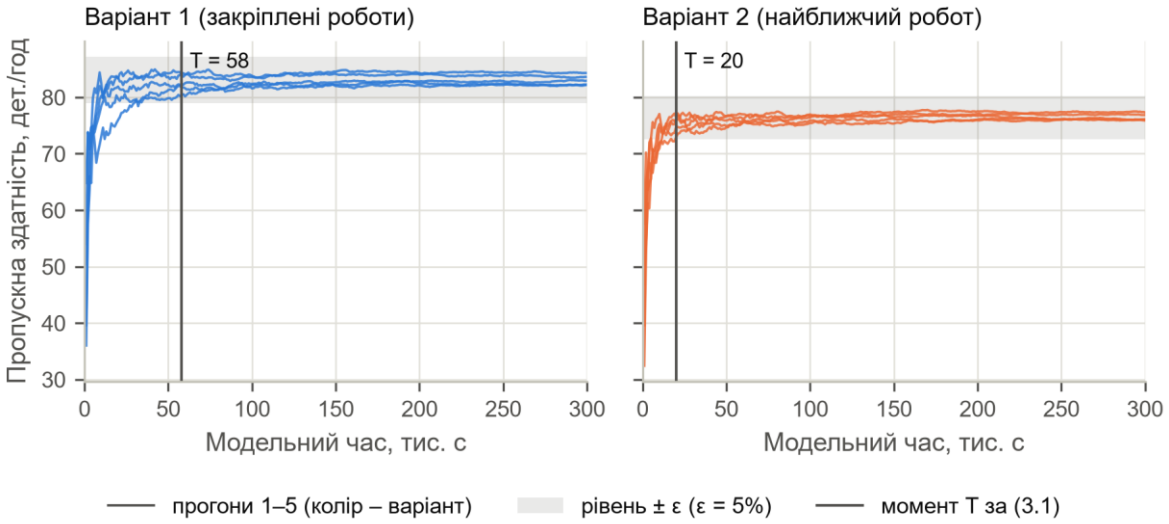


Рисунок 4.1 – Пропускна здатність ГВС, обчислена від початку моделювання, при надходженні exp(40): п’ять прогонів кожного варіанта

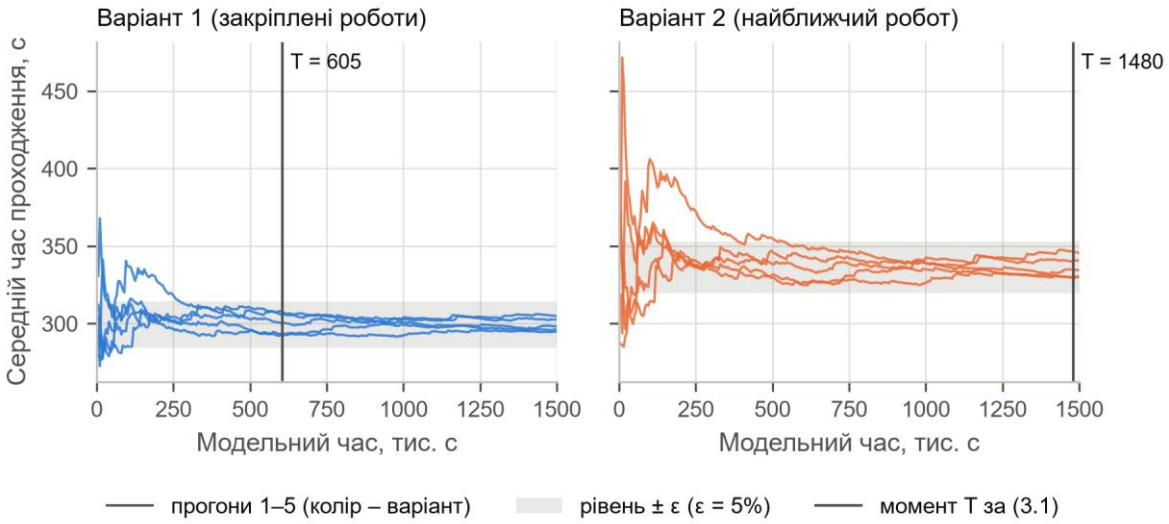


Рисунок 4.2 – Середній час проходження деталей, обчислений від початку моделювання, при надходженні exp(60): п’ять прогонів кожного варіанта

При exp(40) пропускна здатність і коефіцієнти використання стабілізуються, і умова (4.1) для них виконується не пізніше ніж через 58 000 с.

Середній час проходження і середня кількість деталей на площадці умову (4.1) не задовольняють. У другій половині прогону, від 150 000 до 300 000 с, їх середні значення зросли на 87-94%. Це ознака відсутності сталого режиму. Черга на площадці А зростає лінійно з часом моделювання (рисунок 4.3), тому середні значення на інтервалі $[0;\ t]$ зростають майже пропорційно t.

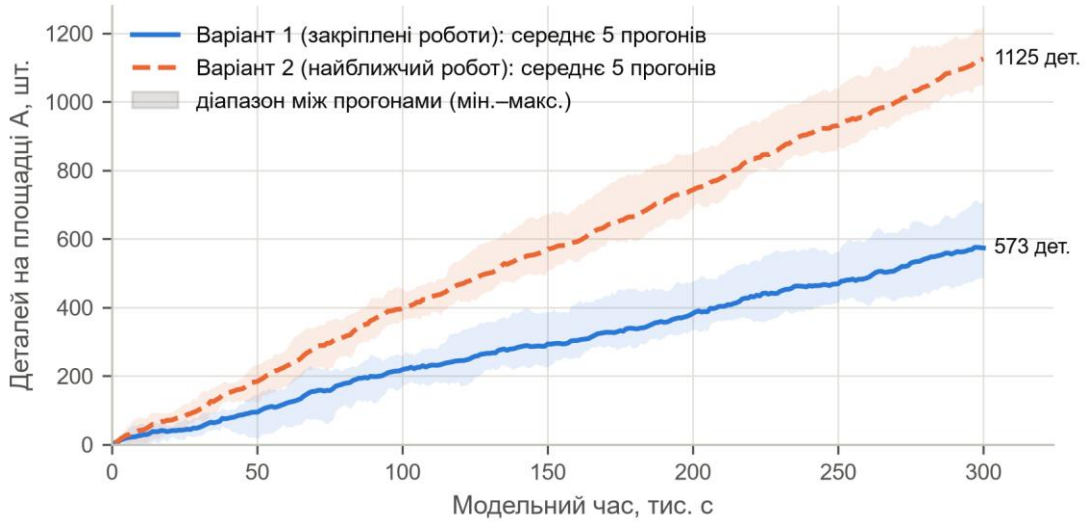


Рисунок 4.3 – Кількість деталей на площадці А при надходженні exp(40): середнє і діапазон значень за п’ятьма прогонами

При exp(60) система працює в стійкому режимі. Умова (4.1) виконується для всіх відгуків, крім $\bar{L}_A$: для пропускної здатності і коефіцієнтів використання T становить 140 000-275 000 с, для середнього часу проходження – 605 000 с у варіанті 1 і 1 480 000 с у варіанті 2. Такі великі значення пояснюються переважно розкидом відгуку в окремому прогоні. Час обробки на М2 має експоненціальний закон, окремі деталі проходять систему за 1500-1900 с, і середнє за одним прогоном наближується до сталого рівня повільно (рисунок 4.2). Отже, при exp(60) умова (4.1) визначає час моделювання, потрібний для точності 5% уже в одному прогоні. Для $\bar{L}_A$ точність 5% від рівня становить лише 0,03-0,06 деталі і за 1 500 000 с не досягається. Проте в другій половині прогону середнє $\bar{L}_A$ не зростає, а зменшується на 1,5-2,6%, тобто сталий режим існує. Оскільки $\bar{L}_A$ є допоміжним відгуком, для нього меншу точність прийнято допустимою.

Вплив перехідного періоду на результат зменшено двома способами, які методичні рекомендації називають найбільш дієвими [3, с. 81]: статистика

збирається лише після перехідного періоду, а час моделювання в кілька разів перевищує перехідний період. Прийнято такі умови одного прогону:

– при $\exp(40)$ $T_{\text{п}} = 60\,000$ с, що більше за найбільше значення $T = 58\,000$ с для відгуків, які стабілізуються, а $T_{\text{mod}} = 300\,000$ с, тобто статистика збирається протягом 240 000 с, що в 4 рази довше за перехідний період;

– при $\exp(60)$ $T_{\text{mod}} = 1\,500\,000$ с, щоб умова (4.1) для середнього часу проходження виконувалась уже в одному прогоні, а $T_{\text{п}} = 50\,000$ с, щоб відкинути деталі, які пройшли систему, поки вона ще не заповнилась.

Кількість прогонів визначено за формулою (2.11), що випливає з нерівності Чебишова. Середнє квадратичне відхилення σ оцінено за пілотною серією з 20 прогонів з наведеними вище умовами, точність задано як $\varepsilon = 0{,}02\,\bar{x}$. Результати наведено в таблиці 4.3.

Таблиця 4.3 – Кількість прогонів за формулою (2.11), пілотна серія з 20 прогонів, ε = 2% від середнього

Відгук	Інтервал надходження	Варіант 1: $\bar{x}$	Варіант 1: σ	Варіант 1: p	Варіант 2: $\bar{x}$	Варіант 2: σ	Варіант 2: p
Λ, дет./год	$\exp(40)$	83,06	0,69	4	76,36	0,66	4
Λ, дет./год	$\exp(60)$	59,91	0,32	2	59,91	0,32	2
$\bar{\tau}$, с	$\exp(40)$	14 369,6	2357,3	—	27 439,7	2411,0	—
$\bar{\tau}$, с	$\exp(60)$	300,0	3,8	9	337,4	7,8	27
L_{max}, шт.	$\exp(40)$	590,9	88,4	—	1133,5	87,6	—
L_{max}, шт.	$\exp(60)$	16,5	2,7	1363	20,6	3,9	1779
ρ_{M2}	$\exp(40)$	0,959	0,001	1	0,907	0,002	1
ρ_{M2}	$\exp(60)$	0,678	0,005	4	0,680	0,006	4
$\bar{\rho}_{\text{R}}$	$\exp(40)$	0,866	0,002	1	0,912	0,002	1
$\bar{\rho}_{\text{R}}$	$\exp(60)$	0,532	0,004	4	0,588	0,006	5

Для пропускної здатності і коефіцієнтів використання достатньо 1-5 прогонів, для середнього часу проходження при $\exp(60)$ – 9 у варіанті 1 і 27 у варіанті 2. Для $\bar{\tau}$ і L_{max} при $\exp(40)$ кількість прогонів не визначалась, бо ці відгуки сталих значень не мають. Для L_{max} при $\exp(60)$ відносна точність 2%, тобто 0,3-0,4 деталі, потребувала б 1363-1779 прогонів. Ціле значення L_{max} не потребує такої точності, тому для нього точність задано абсолютною – 1-2 деталі (підрозділ 2.7).

Для порівняння варіантів прийнято $p = 30$ прогонів, що більше за найбільшу потрібну кількість 27. Точність, яку забезпечує серія з p прогонів, випливає з формули (2.11):

$$\varepsilon = \sigma\sqrt{20/p}. \tag{4.3}$$

При $p = 30$ пропускна здатність при $\exp(40)$ оцінюється з точністю $\pm 0{,}56$ дет./год (0,7%), середній час проходження при $\exp(60)$ – $\pm 3{,}1$ с у варіанті 1 і $\pm 6{,}4$ с у варіанті 2 (1,0 і 1,9%), L_{max} – $\pm 2{,}2$ і $\pm 3{,}2$ деталі. Нерівність Чебишова справедлива за будь-якого закону розподілу відгуку, тому дає гарантовану, але завищену оцінку. Оцінка кількості прогонів за центральною граничною теоремою, $p = (1{,}96\,\sigma/\varepsilon)^2$, приблизно в п’ять разів менша [2, с. 226-228]. Середнє за 30 прогонами розподілене близько до нормального закону, тому 95%-й довірчий інтервал за розподілом Стьюдента для L_{max} становить $\pm 1{,}2$ і $\pm 1{,}7$ деталі (таблиця 4.4), що відповідає заданій точності 1-2 деталі. У серіях 4 і 6 головним відгуком є пропускна здатність, для якої достатньо 4 прогонів, тому 10 і 20 прогонів у цих серіях забезпечують задану точність.

4.3 Порівняння способів закріплення роботів

Порівняння виконано за планом однофакторного експерименту з якісним фактором [2, с. 238]. Фактор має $q = 2$ рівні, на кожному рівні виконано $p = 30$ прогонів. Позначимо y_{ij} значення відгуку в i-му прогоні на j-му рівні фактора, $\bar{y}_j$ – середнє значення на j-му рівні, $\bar{y}$ – загальне середнє. Факторна сума квадратів відхилень характеризує вплив фактора, залишкова – вплив випадкових причин:

$$S_{\text{факт}} = p\sum_{j=1}^{q}(\bar{y}_j - \bar{y})^2, \qquad S_{\text{залиш}} = \sum_{j=1}^{q}\sum_{i=1}^{p}(y_{ij} - \bar{y}_j)^2. \tag{4.4}$$

Факторну і залишкову дисперсії отримують діленням цих сум на кількість ступенів вільності і порівнюють за критерієм Фішера:

$$F = \frac{d_{\text{факт}}}{d_{\text{залиш}}} = \frac{S_{\text{факт}}/(q-1)}{S_{\text{залиш}}/(q(p-1))}. \tag{4.5}$$

Якщо $F > F_{\text{кр}}$, вплив фактора значущий. Критичне значення при рівні значущості $\alpha = 0{,}05$ і ступенях вільності $q - 1 = 1$ і $q(p - 1) = 58$ дорівнює $F_{\text{кр}} = 4{,}01$. Знаменники у формулі (4.5) взято за посібником [2, с. 239]. У формулі (3.10) методичних рекомендацій факторну суму поділено на $qp - 1$, тобто на кількість ступенів вільності загальної дисперсії [3, с. 85]. Тоді значення F зменшилося б у $(qp - 1)/(q - 1) = 59$ разів, і його вже не можна було б порівнювати з квантилем розподілу Фішера з $(1;\ 58)$ ступенями вільності.

Варіанти отримують спільні випадкові числа, тому прогони з однаковим номером у двох варіантах залежні: у них однакові моменти надходження деталей і часи обробки. Дисперсійний аналіз цю залежність не враховує, і за спільних випадкових чисел його висновок про значущість стає обережнішим. Тому додатково обчислено середню різницю відгуків у парах прогонів $\bar{y}_2 - \bar{y}_1$ з 95%-м довірчим інтервалом за розподілом Стьюдента. Вихідні характеристики обох варіантів наведено в таблиці 4.4, результати дисперсійного аналізу – в таблиці 4.5.

Таблиця 4.4 – Вихідні характеристики ГВС для двох способів закріплення роботів (середнє ± півширина 95%-го довірчого інтервалу за 30 прогонами)

Характеристика	exp(40): варіант 1	exp(40): варіант 2	exp(60): варіант 1	exp(60): варіант 2
Λ, дет./год	83,13 ± 0,29	76,43 ± 0,28	59,91 ± 0,13	59,91 ± 0,13
$\bar{\tau}$, с	зростає	зростає	299,9 ± 1,3	337,1 ± 2,6
90%-квантиль τ, с	зростає	зростає	491,1 ± 3,3	566,8 ± 6,0
L_{max}, шт.	зростає	зростає	16,6 ± 1,2	20,8 ± 1,7
$\bar{L}_A$, од.	зростає	зростає	0,68 ± 0,02	1,12 ± 0,04
99%-квантиль L_A, шт.	зростає	зростає	7,0 ± 0,2	10,1 ± 0,6
ρ_{M1}	1,000 ± 0,000	0,922 ± 0,000	0,550 ± 0,002	0,567 ± 0,003
ρ_{M2}	0,958 ± 0,001	0,907 ± 0,001	0,678 ± 0,002	0,680 ± 0,002
ρ_{R1}	1,000 ± 0,000	0,912 ± 0,001	0,534 ± 0,002	0,588 ± 0,002
ρ_{R2}	0,998 ± 0,000	0,913 ± 0,002	0,628 ± 0,002	0,588 ± 0,002
ρ_{R3}	0,600 ± 0,002	0,912 ± 0,002	0,433 ± 0,001	0,588 ± 0,002
$\bar{\rho}_{\text{R}}$	0,866 ± 0,001	0,912 ± 0,001	0,532 ± 0,002	0,588 ± 0,002

Таблиця 4.5 – Дисперсійний аналіз впливу способу закріплення роботів ($q = 2, p = 30, F_{\text{кр}} = 4{,}01$)

Відгук	Інтервал надходження	$\bar{y}_1$	$\bar{y}_2$	$\bar{y}_2 - \bar{y}_1$	F	Вплив фактора
Λ, дет./год	exp(40)	83,13	76,43	−6,70 ± 0,09	1139	значущий
ρ_{M1}	exp(40)	1,000	0,922	−0,0783 ± 0,0005	105 063	значущий
ρ_{M2}	exp(40)	0,958	0,907	−0,052 ± 0,001	7466	значущий
$\bar{\rho}_{\text{R}}$	exp(40)	0,866	0,912	0,0460 ± 0,0004	8713	значущий
Λ, дет./год	exp(60)	59,91	59,91	0,000 ± 0,001	$7{,}6 \cdot 10^{-6}$	незначущий
$\bar{\tau}$, с	exp(60)	299,9	337,1	37,3 ± 1,4	702,9	значущий
L_{max}, шт.	exp(60)	16,6	20,8	4,2 ± 1,0	16,8	значущий
$\bar{L}_A$, од.	exp(60)	0,68	1,12	0,44 ± 0,02	555,5	значущий
ρ_{M1}	exp(60)	0,550	0,567	0,018 ± 0,001	106,9	значущий
ρ_{M2}	exp(60)	0,678	0,680	0,0022 ± 0,0003	2,35	незначущий
$\bar{\rho}_{\text{R}}$	exp(60)	0,532	0,588	0,057 ± 0,001	1712	значущий

При exp(40) пропускна здатність варіанта 1 становить $83{,}13 \pm 0{,}29$ дет./год, варіанта 2 – $76{,}43 \pm 0{,}28$ дет./год. Значення $F = 1139$ майже в триста разів перевищує критичне, тобто спосіб закріплення роботів істотно впливає на пропускну здатність. У парах прогонів пропускна здатність варіанта 2 менша на $6{,}70 \pm 0{,}09$ дет./год, тобто на 8,1%. Вплив фактора значущий і для коефіцієнтів використання. У варіанті 2 верстати завантажені менше, а роботи в середньому більше: $\bar{\rho}_{\text{R}} = 0{,}912$ проти 0,866.

При exp(60) обидва варіанти обробляють усі деталі, що надходять. Пропускна здатність дорівнює інтенсивності надходження і від способу закріплення не залежить ($F \approx 0$). Отже, у стійкому режимі пропускна здатність не розрізняє варіанти, і її порівнюють за перевантаження або за межею стійкості (підрозділ 4.4). Для решти відгуків, крім завантаження M2, вплив фактора значущий. Середній час проходження у варіанті 2 більший на $37{,}3 \pm 1{,}4$ с (12,4%), 90%-квантиль – на 76 с, L_{max} – на $4{,}2 \pm 1{,}0$ деталі. Роботи у варіанті 2 завантажені більше ($\bar{\rho}_{\text{R}} = 0{,}588$ проти 0,532), M1 – трохи більше, бо оброблені деталі довше чекають на робота, а завантаження M2 однакове. Отже, за всіма відгуками, які стосуються мети моделювання, варіант 1 кращий.

4.4 Пропускна здатність залежно від інтенсивності надходження деталей

Щоб визначити, з яким потоком деталей справляється кожен варіант, як фактор розглянуто середній інтервал надходження τ_A. Досліджено 19 значень від 34 до 80 с, причому поблизу межі стійкості, від 40 до 50 с, інтервал змінювався з кроком 1 с. Для кожного значення виконано 10 прогонів (серія 4 таблиці 4.1). Режим вважався стійким, якщо пропускна здатність становить не менше 98% інтенсивності надходження $\lambda = 3600/\tau_A$, а в системі в кінці прогону менше 30 деталей. Результати наведено в таблиці 4.6 і на рисунку 4.4.

Таблиця 4.6 – Пропускна здатність ГВС залежно від інтервалу надходження деталей (10 прогонів)

τ_A, с	λ, дет./год	Варіант 1: Λ, дет./год	Варіант 1: $N_{кін}$, шт.	Варіант 1: режим	Варіант 2: Λ, дет./год	Варіант 2: $N_{кін}$, шт.	Варіант 2: режим
34	105,9	83,0	1918	нестійкий	76,3	2472	нестійкий
36	100,0	83,0	1420	нестійкий	76,3	1974	нестійкий
38	94,7	83,0	981	нестійкий	76,3	1534	нестійкий
40	90,0	83,0	590	нестійкий	76,3	1140	нестійкий
41	87,8	83,0	412	нестійкий	76,3	957	нестійкий
42	85,7	83,0	247	нестійкий	76,3	788	нестійкий
43	83,7	82,8	104	нестійкий	76,3	626	нестійкий
44	81,8	81,8	31	нестійкий	76,3	471	нестійкий
45	80,0	80,1	15	стійкий	76,3	322	нестійкий
46	78,3	78,4	18	стійкий	76,3	189	нестійкий
47	76,6	76,8	16	стійкий	76,2	78	нестійкий
48	75,0	75,2	8	стійкий	75,3	16	стійкий
49	73,5	73,7	11	стійкий	73,7	15	стійкий
50	72,0	72,3	6	стійкий	72,3	10	стійкий
52	69,2	69,6	6	стійкий	69,6	8	стійкий
55	65,5	65,8	8	стійкий	65,8	10	стійкий
60	60,0	60,3	6	стійкий	60,3	7	стійкий
70	51,4	51,7	4	стійкий	51,7	4	стійкий
80	45,0	45,4	4	стійкий	45,4	4	стійкий

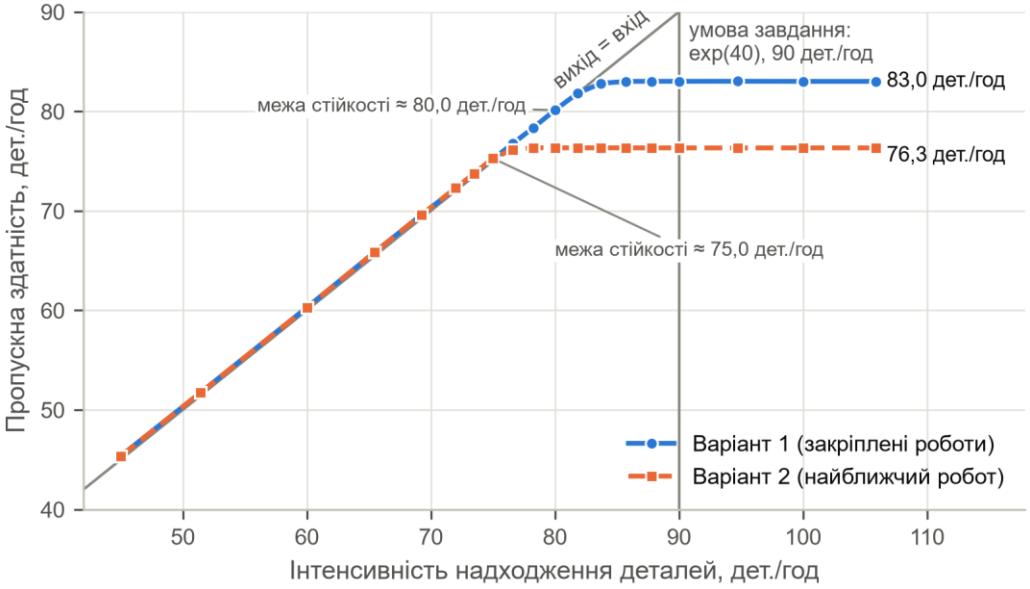


Рисунок 4.4 – Залежність пропускної здатності ГВС від інтенсивності надходження деталей

Поки інтенсивність надходження не перевищує 75 дет./год, обидва варіанти обробляють усі деталі, і $\Lambda \approx \lambda$. Подальше збільшення потоку пропускну здатність не збільшує: вона досягає граничного значення 83,0 дет./год у варіанті 1 і 76,3 дет./год у варіанті 2 і далі не змінюється. Тобто граничну пропускну здатність визначає обладнання, а не потік деталей. Межа стійкості варіанта 1 лежить між 80,0 дет./год (режим стійкий) і 81,8 дет./год (режим нестійкий), варіанта 2 – між 75,0 і 76,6 дет./год. При 81,8 дет./год варіант 1 ще обробляє майже всі деталі, але в системі в кінці прогону в середньому 31 деталь, тобто черга вже не встигає розсмоктуватись. За інтенсивності від 76,6 до 80 дет./год стійкий лише варіант 1. За умови завдання, 90 дет./год, обидва варіанти перевантажені. Щоб система справлялася з таким потоком, її граничну пропускну здатність треба збільшити щонайменше на 8% для варіанта 1 і на 18% для варіанта 2 (підрозділ 5.2).

4.5 Коефіцієнти використання роботів і верстатів

Коефіцієнти використання роботів і верстатів при exp(40) наведено на рисунку 4.5, розподіл часу кожного робота за станами його пристроїв – на рисунку 4.6. Числові значення з довірчими інтервалами для обох інтервалів надходження наведено в таблиці 4.4.

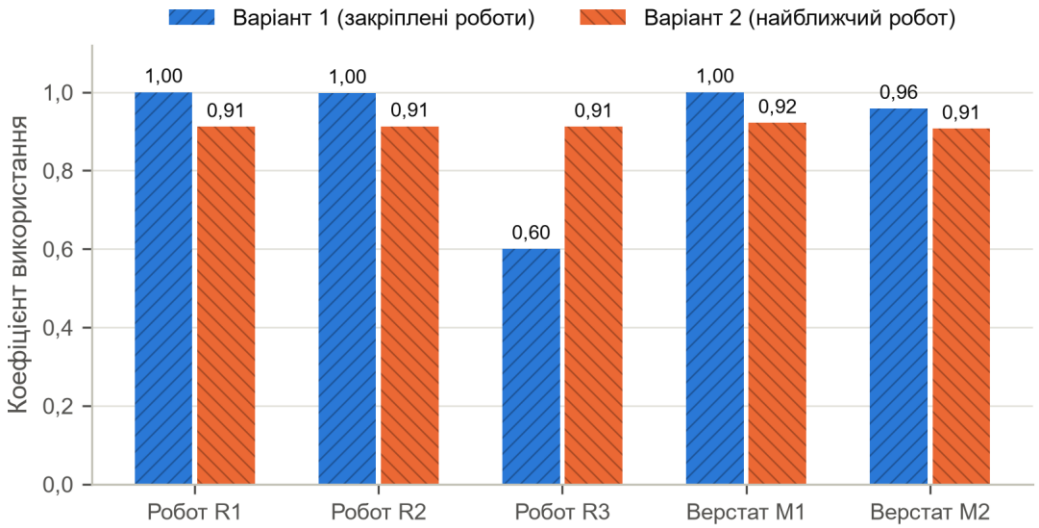


Рисунок 4.5 – Коефіцієнти використання роботів і верстатів при надходженні exp(40)

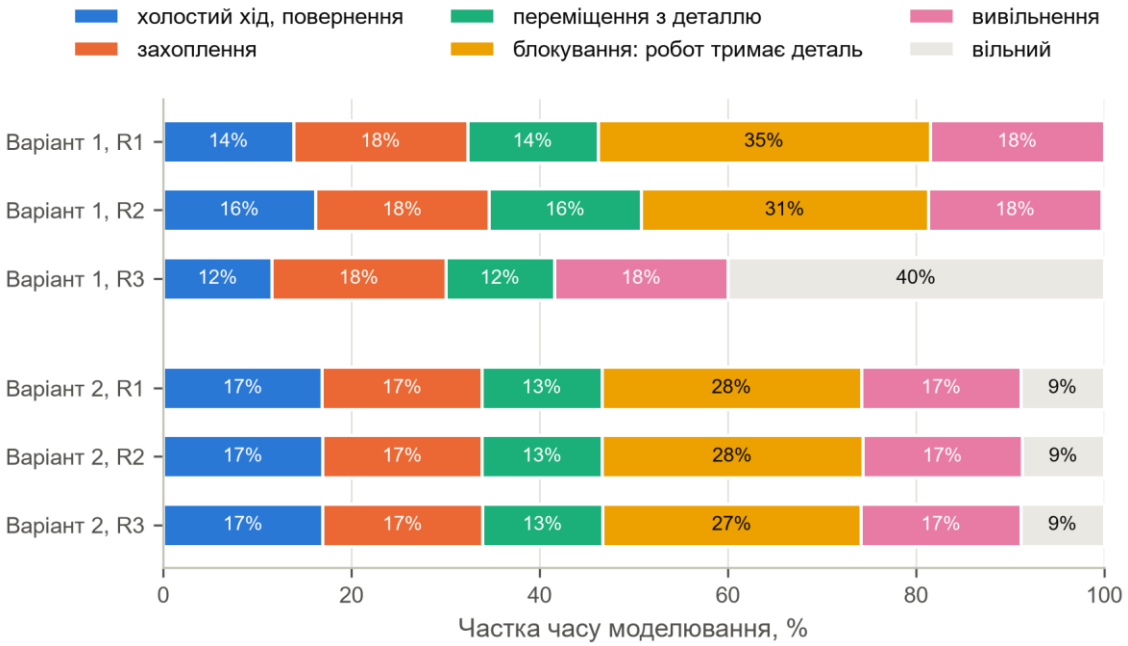


Рисунок 4.6 – Розподіл часу роботів за станами пристроїв при надходженні exp(40)

У варіанті 1 роботи завантажені нерівномірно: $\rho_{R1} = 1{,}000$, $\rho_{R2} = 0{,}998$, $\rho_{R3} = 0{,}600$. Велику частку часу R1 і R2 не перевозять деталі, а тримають їх біля заповненого верстата, тобто їх пристрої заблоковані: R1 – 35% часу біля M1, R2 – 31% біля M2. Робот R3 доставляє деталі на склад, який приймає їх завжди, тому R3 ніколи не блокується, а 40% часу вільний. Повернення до точки завантаження

займає 12-16% часу кожного робота. Верстат M1 зайнятий весь час ($\rho_{\text{M1}} = 1{,}000$), але власне обробка займає лише 46% часу кожного місця, а 42% часу місце блоковане: оброблена деталь чекає на робота R2. Отже, M1 заповнений не через власну тривалість обробки, а через те, що R2 стоїть із деталлю біля заповненого M2. Верстат M2 зайнятий 96% часу: обробка займає 77%, блокування – 6,5%, вивільнення і захоплення – 12%.

У варіанті 2 всі три роботи завантажені однаково, на 91%. Холостий хід займає 17% часу кожного робота, очікування з деталлю біля заповненого верстата – 28%, ще 9% часу робот вільний. Коефіцієнти використання верстатів менші, ніж у варіанті 1: $\rho_{\text{M1}} = 0{,}922$, $\rho_{\text{M2}} = 0{,}907$. Причини цього пояснено в підрозділі 5.1.

При $\exp(60)$ завантаження всіх елементів менше, і найбільш завантаженим є верстат M2: $\rho_{\text{M2}} = 0{,}678$ у варіанті 1 і 0,680 у варіанті 2. Коефіцієнти використання роботів у варіанті 1 становлять 0,534, 0,628 і 0,433, у варіанті 2 – по 0,588.

4.6 Розподіл часу проходження деталей

Закон розподілу часу проходження ідентифіковано в такій послідовності: формування масиву спостережуваних значень, побудова гістограми частот, формування гіпотези про вид закону, визначення параметрів закону і перевірка відповідності за критерієм згоди χ^2.

Масив значень сформовано за 20 прогонами при $\exp(60)$ з $T_{\text{mod}} = 1\,500\,000$ с і $T_{\text{п}} = 50\,000$ с (серія 5 таблиці 4.1). За один прогін на склад надходить близько 24 100 деталей. Деталі, які проходять систему одна за одною, затримуються в тих самих чергах, тому їх часи проходження залежні. Щоб зменшити цю залежність, з кожного прогону взято кожну п’яту деталь, разом 96 500 значень. На такій вибірці критерій χ^2 відхиляє всі розглянуті закони: найменші значення χ^2 становлять 1004 і 1178 при критичному 38,9. При дуже великому обсязі вибірки критерій виявляє навіть незначні відхилення від теоретичного закону, які не мають практичного значення. Тому для перевірки з

масиву рівномірно відібрано $n = 500$ значень. Сусідні значення в цій вибірці розділені приблизно 965 деталями, тому їх можна вважати незалежними. Статистичні характеристики вибірки наведено в таблиці 4.7.

Таблиця 4.7 – Статистичні характеристики вибірки часу проходження деталей, $\exp(60)$

Характеристика	Варіант 1	Варіант 2
обсяг вибірки n	500	500
середнє $\bar{\tau}$, с	297,4	333,7
середнє квадратичне відхилення s_τ, с	134,3	166,9
коефіцієнт варіації $s_\tau/\bar{\tau}$	0,452	0,500
найменше значення, с	114,5	119,4
медіана, с	263,5	291,8
90%-квантиль, с	501,3	553,2
95%-квантиль, с	565,7	709,0
найбільше значення, с	864,7	1191,8
коефіцієнт асиметрії	1,25	1,52

Гістограми відносних частот побудовано на 12 інтервалах однакової ширини між найменшим і найбільшим значеннями вибірки (рисунки 4.7 і 4.8). Гіпотези про вид закону сформульовано за виглядом гістограм і характеристиками вибірки:

– розподіл несиметричний, з довгим правим «хвостом»: коефіцієнт асиметрії 1,25 і 1,52, тому нормальний закон малоймовірний; його перевірено для порівняння;

– час проходження не може бути меншим за суму найменших тривалостей операцій, і найменше значення у вибірці становить 114-119 с, тому розглянуто закони зі зсувом c, тобто з нижньою межею;

– час проходження складається з кількох послідовних затримок, а найбільший розкид дає експоненціальна обробка на М2. Тому розглянуто експоненціальний закон зі зсувом, гамма-закон без зсуву і зі зсувом, а також логнормальний закон зі зсувом, який часто описує тривалості з правою асиметрією.

Щільності законів, що описують вибірку найкраще:

$$f(\tau) = \frac{(\tau - c)^{k-1}}{\Gamma(k)\,\theta^k}\, e^{-(\tau-c)/\theta}, \qquad \tau > c, \tag{4.6}$$

$$f(\tau) = \frac{1}{(\tau - c)\, s\sqrt{2\pi}} \exp\left(-\frac{(\ln(\tau - c) - \mu)^2}{2s^2}\right), \qquad \tau > c, \tag{4.7}$$

де (4.6) – гамма-закон зі зсувом c, параметром форми k і параметром масштабу θ; (4.7) – логнормальний закон зі зсувом c і параметрами μ, s логарифма величини $\tau - c$. Середнє значення гамма-закону дорівнює $c + k\theta$. Параметри законів оцінено методом найбільшої правдоподібності функцією `fit` бібліотеки SciPy [7]. Для гамма-закону зі зсувом пошук починається з оцінок методу моментів $k = (2/\gamma)^2$, $\theta = s_\tau/\sqrt{k}$, $c = \bar{\tau} - k\theta$, де γ – коефіцієнт асиметрії. Для експоненціального закону зі зсувом прийнято $c = \tau_{min}$, $m = \bar{\tau} - \tau_{min}$.

Відповідність закону перевірено за критерієм χ^2 [2, с. 27-28]:

$$\chi^2 = \sum_{l=1}^{h} \frac{(n_l - n\,p_l)^2}{n\,p_l}, \qquad \nu = h - 1 - r, \tag{4.8}$$

де n_l – кількість значень в l-му інтервалі; p_l – імовірність потрапляння в інтервал за гіпотетичним законом; h – кількість інтервалів; ν – кількість ступенів вільності; r – кількість оцінених параметрів закону (2 або 3). Крайні інтервали продовжено до $-\infty$ і $+\infty$, щоб сума p_l дорівнювала 1. Інтервали, для яких $n\,p_l < 5$, об'єднано із сусідніми [2, с. 28]. Гіпотеза приймається, якщо $\chi^2 < \chi^2_{\text{кр}}$ при рівні значущості 0,05. Результати перевірки наведено в таблиці 4.8, гістограми з кривими трьох найкращих законів – на рисунках 4.7 і 4.8.

Таблиця 4.8 – Перевірка гіпотез про закон розподілу часу проходження за критерієм χ^2 ($\alpha = 0{,}05$)

Варіант	Закон розподілу	Оцінка параметрів	χ^2	ν	$\chi^2_{\text{кр}}$	Гіпотеза
1	гамма зі зсувом	$k = 1{,}895;\ c = 112{,}6;\ \theta = 97{,}5$	5,82	6	12,59	приймається
1	логнормальний зі зсувом	$s = 0{,}626;\ c = 83{,}6;\ \mu = 5{,}176$	12,64	7	14,07	приймається
1	гамма без зсуву	$k = 5{,}708;\ \theta = 52{,}1$	28,85	6	12,59	відхиляється
1	експоненціальний зі зсувом	$c = 114{,}5;\ m = 182{,}9$	60,18	8	15,51	відхиляється

Варіант	Закон розподілу	Оцінка параметрів	χ^2	ν	$\chi^2_{\text{кр}}$	Гіпотеза
1	нормальний	$\mu = 297{,}4;\ \sigma = 134{,}2$	80,01	5	11,07	відхиляється
2	гамма зі зсувом	$k = 1{,}792;\ c = 117{,}4;\ \theta = 120{,}7$	9,12	5	11,07	приймається
2	логнормальний зі зсувом	$s = 0{,}688;\ c = 95{,}1;\ \mu = 5{,}248$	11,60	6	12,59	приймається
2	гамма без зсуву	$k = 4{,}883;\ \theta = 68{,}3$	35,89	5	11,07	відхиляється
2	експоненціальний зі зсувом	$c = 119{,}4;\ m = 214{,}3$	47,55	8	15,51	відхиляється
2	нормальний	$\mu = 333{,}7;\ \sigma = 166{,}8$	69,96	4	9,49	відхиляється

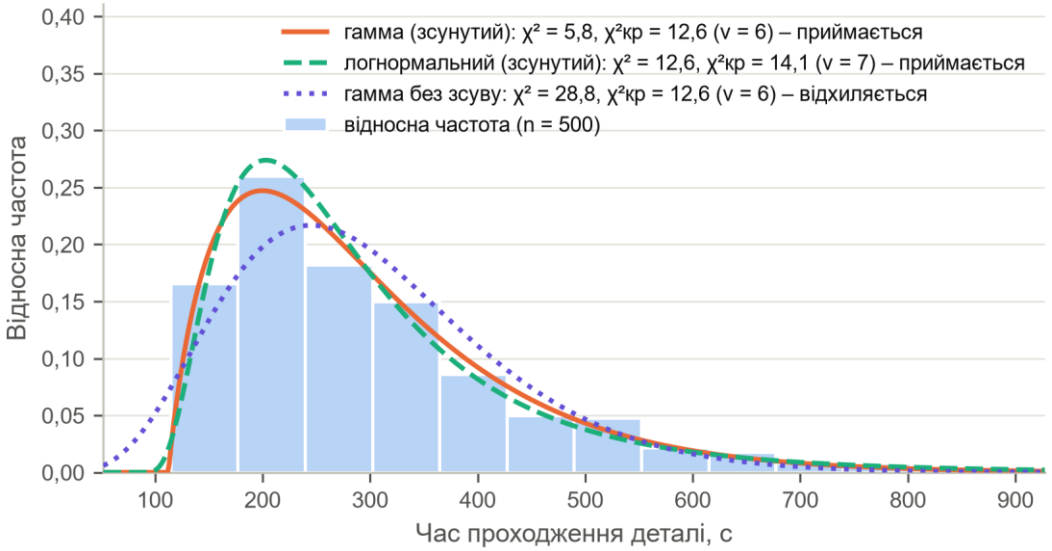


Рисунок 4.7 – Гістограма відносних частот часу проходження деталей і щільності законів розподілу, варіант 1

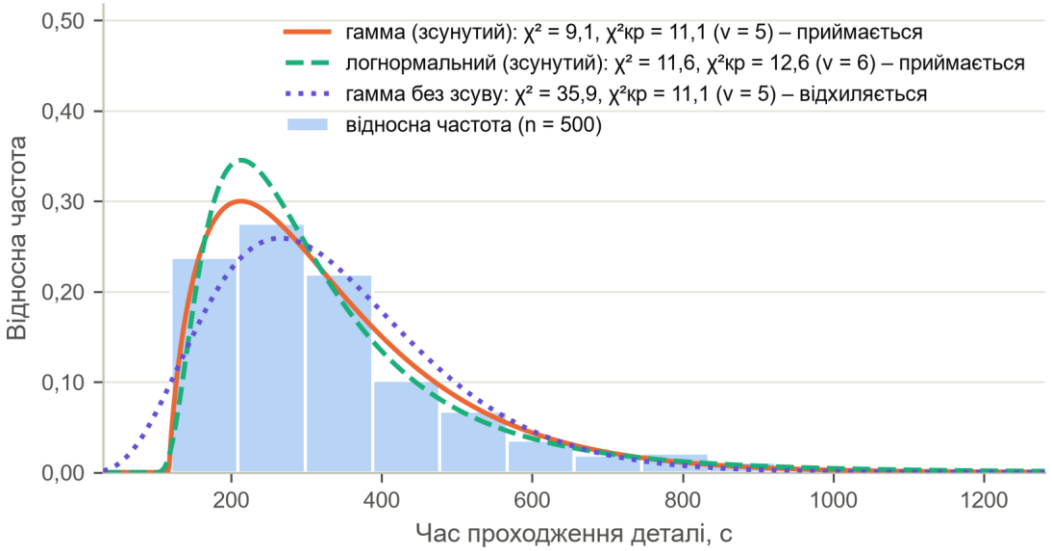


Рисунок 4.8 – Гістограма відносних частот часу проходження деталей і щільності законів розподілу, варіант 2

В обох варіантах приймаються гіпотези про гамма-закон зі зсувом і логнормальний закон зі зсувом, а нормальний, експоненціальний зі зсувом і гамма-закон без зсуву відхиляються. Відхилення гамма-закону без зсуву показує, що нижня межа часу проходження суттєва. Гамма-закон зі зсувом в обох варіантах має найменше значення χ^2, тому його прийнято як закон розподілу часу проходження деталей:

– варіант 1: $k = 1{,}895$, $c = 112{,}6$ с, $\theta = 97{,}5$ с, середнє $c + k\theta = 297{,}4$ с;

– варіант 2: $k = 1{,}792$, $c = 117{,}4$ с, $\theta = 120{,}7$ с, середнє $c + k\theta = 333{,}7$ с.

Параметри форми у варіантах близькі. У варіанті 1 зсув менший, тобто найшвидші деталі проходять систему швидше: закріплений робот повертається до точки завантаження одразу після вивільнення і часто вже чекає там на деталь. Параметр масштабу у варіанті 2 більший на 24%, тому більші середнє, розкид і частка довгих часів проходження: 90% деталей проходять систему не довше ніж за 491 с у варіанті 1 і 567 с у варіанті 2 (таблиця 4.4).

4.7 Максимальна місткість місця зберігання на ділянці прибуття

При $\exp(40)$ черга на площадці А зростає необмежено (рисунок 4.3): у середньому на $90 - 83{,}1 = 6{,}9$ деталі за годину у варіанті 1 і на $90 - 76{,}4 = 13{,}6$ деталі за годину у варіанті 2. Наприкінці прогону тривалістю 300 000 с на площадці в середньому 547 і 1099 деталей. Отже, за умови завдання жодна скінченна місткість площадки не достатня, і L_{max} визначається лише тривалістю роботи системи.

При $\exp(60)$ найбільша кількість деталей на площадці за час спостереження 1 450 000 с (близько 400 год) становить $16{,}6 \pm 1{,}2$ у варіанті 1 і $20{,}8 \pm 1{,}7$ у варіанті 2. В окремих прогонах вона змінювалась від 13 до 26 деталей у варіанті 1 і від 17 до 38 у варіанті 2. Середня кількість деталей на площадці значно менша: 0,68 і 1,12, а 99% часу на ній не більше 7 і 10 деталей (таблиця 4.4). L_{max} є найбільшим значенням випадкового процесу, тому воно зростає зі збільшенням часу спостереження. Наприклад, у серії 4 з часом спостереження 240 000 с при $\exp(60)$ отримано L_{max} = 13,5 і 16,9 деталі.

Місткість площадки також різко зростає з наближенням до межі стійкості: при exp(50) за 240 000 с L_{max} = 27,0 і 41,6 деталі.

4.8 Програмна реалізація експериментів

Експерименти реалізовано в пакеті `fms.experiments`. Кожна серія таблиці 4.1 відповідає окремому модулю і команді командного інтерфейсу. Результати зберігаються у файлах CSV у каталозі `results`, таблиці записки будуються з цих файлів окремим скриптом, а рисунки – модулем `report_plots.py`. Серії незалежних прогонів виконує функція `replicate`. Прогони з номером i мають зерно генераторів i, тому однакові для обох варіантів. Прогони виконуються паралельно в окремих процесах. Команди для відтворення всіх результатів розділу:

```
uv run fms warmup --arrival 40
uv run fms warmup --arrival 60 --t-end 1500000 --step 5000
uv run fms reps --arrival 40 --t-end 300000 --warmup 60000
uv run fms reps --arrival 60 --t-end 1500000 --warmup 50000
uv run fms compare --arrival 40 --reps 30 --t-end 300000 --warmup 60000
uv run fms compare --arrival 60 --reps 30 --t-end 1500000 --warmup 50000
uv run fms capacity --reps 10 --t-end 300000 --warmup 60000
uv run fms dist --arrival 60 --reps 20 --t-end 1500000 --warmup 50000 \
    --max-n 500 --bins 12
uv run fms improve --reps 20 --t-end 300000 --warmup 60000
uv run fms plots
```

Тривалість перехідного періоду за умовою (4.1) визначає функція `transient_time`. Вона перебирає точки спостереження від кінця прогону до початку і зупиняється на першій точці, у якій розкид середніх значень прогонів не менший за ε:

```
def transient_time(series, metric, eps):
    runs = list(series.values())
    n_points = min(len(r) for r in runs)
    T = math.nan
    for k in range(n_points - 1, -1, -1):   # від кінця прогону до початку
        vals = [r[k][metric] for r in runs]
        if any(math.isnan(v) for v in vals) or max(vals) - min(vals) >= eps:
            break
        T = runs[0][k]["t"]
    return T
```

Дисперсійний аналіз за формулами (4.4) і (4.5) виконує функція `anova` (спрощено: прибрано обчислення допоміжних величин):

```
def anova(groups, alpha=0.05):
    q, p = len(groups), len(groups[0])
    means = np.array([g.mean() for g in groups])
```

```
    grand = np.concatenate(groups).mean()
    s_f = p * ((means - grand) ** 2).sum()                    # (4.4)
    s_r = sum(((g - g.mean()) ** 2).sum() for g in groups)
    df1, df2 = q - 1, q * (p - 1)
    f = (s_f / df1) / (s_r / df2)                             # (4.5)
    f_crit = stats.f.ppf(1 - alpha, df1, df2)
    return f, f_crit, f > f_crit
```

Критерій χ^2 за формулою (4.8) з об'єднанням інтервалів обчислює функція `chi_square`:

```
def chi_square(data, dist, params, n_params, bins):
    counts, edges = np.histogram(data, bins=bins)
    cdf = dist.cdf(edges, *params)
    cdf[0], cdf[-1] = 0.0, 1.0       # крайні інтервали – до нескінченності
    probs = np.diff(cdf)
    n = len(data)
    obs, exp_, acc_o, acc_e = [], [], 0.0, 0.0
    for o, p in zip(counts, probs):  # об'єднання інтервалів з n*p < 5
        acc_o += o
        acc_e += n * p
        if acc_e >= 5:
            obs.append(acc_o)
            exp_.append(acc_e)
            acc_o = acc_e = 0.0
    if acc_e > 0 and exp_:           # залишок – до останнього інтервалу
        obs[-1] += acc_o
        exp_[-1] += acc_e
    obs, exp_ = np.array(obs), np.array(exp_)
    chi2 = float(((obs - exp_) ** 2 / exp_).sum())
    df = len(obs) - 1 - n_params
    return chi2, df, stats.chi2.ppf(0.95, df)
```

Повний код модулів експериментального дослідження наведено в додатку В.

4.9 Висновки до розділу

Експериментальне дослідження показало таке:

1) за умови завдання $\exp(40)$ система перевантажена в обох варіантах, тому способи закріплення роботів порівнюються за пропускною здатністю і коефіцієнтами використання, а розподіл часу проходження і місткість площадки визначено в стійкому режимі при $\exp(60)$;

2) перехідний період для відгуків, що стабілізуються, при $\exp(40)$ не перевищує 58 000 с. Прийнято $T_{\text{п}} = 60\,000$ с і $T_{\text{mod}} = 300\,000$ с при $\exp(40)$, $T_{\text{п}} = 50\,000$ с і $T_{\text{mod}} = 1\,500\,000$ с при $\exp(60)$. Серія з 30 прогонів забезпечує точність не гірше 2% для пропускної здатності, часу проходження і коефіцієнтів використання та $\pm 1{,}2$-$1{,}7$ деталі для L_{max} за довірчим інтервалом;

3) спосіб закріплення роботів істотно впливає на пропускну здатність ($F = 1139$ при $F_{\text{кр}} = 4{,}01$): у варіанті 1 вона становить 83,1 дет./год, у варіанті 2 – 76,4 дет./год. Межа стійкості варіанта 1 лежить у межах 80,0-81,8 дет./год, варіанта 2 – 75,0-76,6 дет./год;

4) у стійкому режимі варіант 1 також кращий: середній час проходження менший на 37 с, L_{max} – на 4,2 деталі;

5) час проходження деталей в обох варіантах розподілений за гамма-законом зі зсувом, середнє значення при $\exp(60)$ становить 300 с у варіанті 1 і 337 с у варіанті 2;

6) вузьким місцем системи є верстат М2. У варіанті 1 роботи R1 і R2 завантажені майже повністю, а R3 – на 60%. У варіанті 2 роботи завантажені рівномірно, на 91%, але більше часу витрачають на холостий хід.

Отже, найкращим з погляду пропускної здатності є варіант 1, закріплення по одному роботу за кожним шляхом переміщення деталей.

5 ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ МОДЕЛЮВАННЯ СИСТЕМИ, ФОРМУЛЮВАННЯ ВИСНОВКІВ ТА ПРОПОЗИЦІЙ

У розділі сформульовано висновки щодо роботи ГВС і пропозиції щодо її поліпшення, перевірені пробними запусками моделі.

5.1 Інтерпретація результатів моделювання

Деяких умов роботи системи текст завдання не визначає, тому під час розробки моделі прийнято такі рішення:

7) що робить робот після вивільнення деталі, у завданні не сказано. У варіанті 1 робот закріплений за шляхом, тому після вивільнення він одразу повертається порожнім до стартової точки свого шляху і чекає на наступну деталь там. У варіанті 2 робот залишається в пункті вивантаження і рухається порожнім, коли його призначають на наступне перевезення;

8) розташування пунктів у завданні не задано. Прийнято, що вони стоять на одній лінії A – M1 – M2 – W, а час переміщення між несусідніми пунктами дорівнює сумі часів між сусідніми;

9) у варіанті 2 робот не бере деталь для верстата, до якого вже везуть деталі на одного робота більше, ніж там вільних місць. Без цієї умови всі три роботи можуть узяти деталі для одного заповненого верстата, і робота системи зупиняється: у пробних прогонах це ставалось уже за перші 320-1172 с;

10) розподіл часу проходження і місткість площадки визначено при надходженні деталі в середньому раз на 60 с, а не раз на 40 с, як у завданні. Якщо деталі надходять раз на 40 с, система не встигає їх обробляти, черга зростає безмежно, і ці характеристики сталих значень не мають.

Перевірка моделі (підрозділ 3.9) показала, що вона правильно реагує на зміну вхідних змінних, а в сценарії з детермінованими часами результат моделювання точно збігається з ручним розрахунком. Отже, модель придатна для дослідження системи. Результати моделювання можна пояснити так.

Пропускна здатність. За умови завдання деталі надходять у середньому 90 разів на годину. Жоден варіант із таким потоком не справляється: у варіанті 1 на склад надходить 83,1 деталі за годину, у варіанті 2 – 76,4. Решта деталей, 6,9 і

13,6 за годину, накопичується на площадці прибуття. Через 83 години роботи від порожнього стану там перебуває близько 550 деталей у варіанті 1 і 1100 у варіанті 2, і черга продовжує зростати.

Найкращий спосіб закріплення роботів – варіант 1, по одному роботу на кожному шляху. Його пропускна здатність на 6,7 деталі за годину, тобто на 8,8%, більша, ніж у варіанті 2, і ця різниця не випадкова (підрозділ 4.3). Варіант 1 також справляється з більшим потоком: до 80 деталей за годину проти 75 у варіанті 2. Коли потік менший і обидва варіанти встигають обробити всі деталі, у варіанті 1 деталь проходить систему швидше, а на площадці прибуття накопичується менше деталей. Навіть при потоці 45-51 деталь за годину середній час проходження у варіанті 1 на 13-20 с менший (258-273 с проти 272-293 с).

Чому варіант 2 гірший. Здавалося б, роботи, які можуть виконувати будь-яке перевезення, мають працювати краще. Модель показує протилежне. Вузьким місцем системи є другий верстат M2. Обробка на ньому триває в середньому 100 с, а разом із вивільненням і захопленням деталі місце зайняте 116 с. Отже, навіть без жодних простоїв три місця M2 обробили б не більше $3 \cdot 3600/116 \approx 93$ деталей за годину, тобто майже стільки, скільки надходить. Тому пропускна здатність усієї системи залежить від того, як швидко звільнене місце M2 отримує нову деталь і як швидко оброблену деталь забирають на склад. Складові часу, за який одне місце M2 обробляє одну деталь, і показники роботи роботів наведено в таблиці 5.1. Тут T – час спостереження, N_W – кількість деталей, що надійшли на склад.

Таблиця 5.1 – Складові циклу місця верстата M2 і робота роботів при $\exp(40)$ (середнє за 30 прогонами)

Величина	Варіант 1	Варіант 2
Цикл місця M2 на одну деталь, с		
обробка	100,0	100,1
вивільнення і захоплення роботом	16,0	16,0
блокування: оброблена деталь чекає на робота	8,5	12,1
простій: місце вільне, нову деталь ще не доставлено	5,4	13,2
разом, $3T/N_W$	129,9	141,3
Холостий хід робота на одне перевезення (у варіанті 1 – повернення), с		

Величина	Варіант 1	Варіант 2
шлях 1: до пункту прибуття A	6,0	12,6
шлях 2: до верстата M1	7,0	7,1
шлях 3: до верстата M2	5,0	4,2
разом на одну деталь	18,0	23,9
Частка часу, коли біля верстата чекає робот з деталлю		
біля M1	0,35	0,45
біля M2	0,31	0,38

У варіанті 1 робот R2 перевозить деталі лише з M1 на M2 і майже третину часу стоїть біля M2 з наступною деталлю, тому звільнене місце отримує деталь у середньому через 5,4 с. Робот R3 після кожної доставки на склад одразу повертається до M2 і 40% часу чекає там вільним, тому оброблена деталь чекає на нього лише 8,5 с. У варіанті 2 правило найближчого робота не враховує, яке перевезення важливіше для системи. По нову деталь до пункту прибуття часто їде робот, що стоїть біля M2 або на складі, тому холостий хід до A зростає вдвічі, з 6,0 до 12,6 с. Роботи також частіше стоять із деталями біля заповненого M1 (45% часу проти 35%), хоча M1 не є вузьким місцем. Унаслідок цього місце M2 на кожну деталь простоює 13,2 с замість 5,4 с, а оброблена деталь чекає на робота 12,1 с замість 8,5 с. Час обробки однієї деталі одним місцем M2 зростає зі 129,9 до 141,3 с, і пропускна здатність зменшується з 83,1 до 76,4 деталі за годину. Вищий коефіцієнт використання роботів у варіанті 2 (0,91 проти 0,87) не є перевагою: роботи більше часу витрачають на холості переїзди.

Завантаження обладнання. У варіанті 1 роботи R1 і R2 зайняті майже весь час, але третину часу вони не перевозять деталі, а тримають їх біля заповнених верстатів. Перший верстат заповнений постійно, хоча обробка займає лише 46% часу, бо оброблені деталі чекають на робота R2, який стоїть біля заповненого M2. Отже, нестача місць на M2 зупиняє і перший верстат, і роботи R1, R2, тому додатковий робот без нового місця на M2 майже не допомагає (підрозділ 5.2).

Час проходження при потоці 60 деталей за годину в середньому становить 5,0 хв у варіанті 1 і 5,6 хв у варіанті 2. Половина деталей проходить систему швидше ніж за 4,4-4,8 хв, 90% – швидше ніж за 8,2 і 9,4 хв, а окремі деталі

затримуються до 23-27 хв, коли обробка на М2 виявляється довгою. Розподіл описується гамма-законом зі зсувом близько 2 хв, що відповідає найменшому часу, за який деталь проходить усі операції без очікування.

Місткість площадки прибуття. При потоці 60 деталей за годину за 400 годин роботи на площадці одночасно буває в середньому до 17 деталей у варіанті 1 і до 21 у варіанті 2, а 99% часу – не більше 7 і 10 деталей. Найбільша кількість зростає з тривалістю роботи, тому площадку доцільно розраховувати із запасом: у жодному з 30 прогонів вона не перевищила 26 і 38 деталей відповідно. За умови завдання, 90 деталей за годину, жодна місткість площадки не достатня, бо черга зростає безмежно.

5.2 Пропозиції щодо поліпшення функціонування системи

Оскільки вузьким місцем є верстат М2, розглянуто три пропозиції:

1) додати четверте місце обробки на верстаті М2. Тоді без простоїв М2 міг би обробити до $4 \cdot 3600/116 \approx 124$ деталей за годину;

2) додати четвертий робот. У варіанті 1 він закріплюється за одним із шляхів, і перевірено всі три шляхи. У варіанті 2 він входить у загальну групу роботів;

3) поєднати обидві зміни. У варіанті 1 четвертий робот закріплено за шляхом М1 → М2, бо окремо саме він дає найбільший приріст.

Кожну пропозицію перевірено пробними запусками моделі за умови завдання $\exp(40)$, 20 прогонів по 300 000 с зі збором статистики після 60 000 с (серія 6 таблиці 4.1). Зерна генераторів такі самі, як для базової системи, тому відмінності пояснюються саме зміною системи. Результати наведено в таблиці 5.2 і на рисунку 5.1.

Таблиця 5.2 – Результати пробних запусків пропозицій, $\exp(40)$, 20 прогонів

Пропозиція	Варіант	Λ, дет./год	Режим	$\bar{\tau}$, с	L_{max}, шт.	ρ_{M2}	$\bar{\rho}_R$
базова система	1	83,1 ± 0,3	нестійкий	14 370*	590,9*	0,959	0,866
базова система	2	76,4 ± 0,3	нестійкий	27 440*	1133,5*	0,907	0,912
4 місця на М2	1	90,0 ± 0,4	стійкий	411	27,9	0,803	0,777

Пропозиція	Варіант	Λ, дет./год	Режим	$\bar{\tau}$, с	L_{max}, шт.	ρ_{M2}	$\bar{\rho}_R$
4 місця на M2	2	90,0 ± 0,4	стійкий	1208	72,4	0,802	0,903
4-й робот на маршруті 1 (A → M1)	1	83,1 ± 0,3	нестійкий	14 367*	589,9*	0,959	0,899
4-й робот на маршруті 2 (M1 → M2)	1	86,0 ± 0,4	нестійкий	8984*	363,1*	1,000	0,896
4-й робот на маршруті 3 (M2 → W)	1	83,1 ± 0,3	нестійкий	14 359*	590,8*	0,897	0,649
4-й робот	2	83,7 ± 0,3	нестійкий	13 220*	543,9*	0,941	0,745
4 місця на M2 і 4-й робот на маршруті 2	1	89,9 ± 0,4	стійкий	349	21,9	0,810	0,656
4 місця на M2 і 4-й робот	2	90,0 ± 0,4	стійкий	380	26,2	0,752	0,634

Значення, позначені «*», отримано в нестійкому режимі. Вони зростають зі збільшенням часу моделювання і наведені лише для порівняння пропозицій між собою.

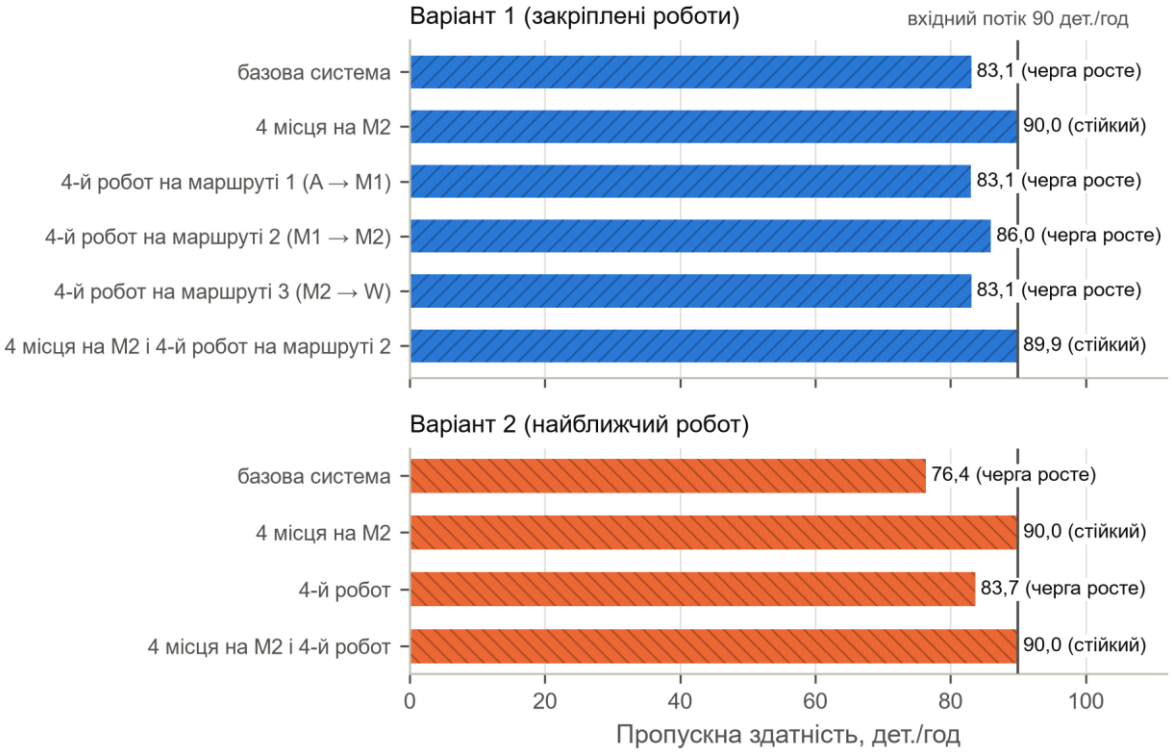


Рисунок 5.1 – Пропускна здатність ГВС для базової системи і пропозицій щодо її поліпшення при надходженні exp(40)

Четверте місце на M2 – єдина окрема зміна, після якої система справляється з потоком 90 деталей за годину в обох варіантах. Пропускна здатність зростає до 90 деталей за годину, тобто на 6,9 (8,3%) у варіанті 1 і на 13,6 (17,8%) у варіанті 2. Черга на площадці перестає накопичуватись: у варіанті

1 середній час проходження становить 411 с, а найбільша кількість деталей на площадці за 67 годин – 28. Варіант 1 і тут кращий за варіант 2, у якому час проходження 1208 с, а на площадці буває до 72 деталей. Коефіцієнт використання M2 зменшується з 0,96 до 0,80, тобто з'являється запас продуктивності. Погіршується лише одна характеристика: у варіанті 1 частка часу, коли оброблена деталь на M2 чекає на робота, зростає з 0,065 до 0,078, бо через M2 проходить більше деталей, а вивозить їх той самий робот R3. Витрати на цю пропозицію – доопрацювання верстата M2 або заміна його верстатом із чотирма місцями обробки.

Четвертий робот без зміни M2 проблему не розв'язує. У варіанті 1 робот на шляху A → M1 або M2 → W пропускної здатності не змінює, бо ці шляхи не обмежують роботу системи. Робот на шляху M1 → M2 швидше підвозить деталі до M2 і збільшує пропускну здатність лише до 86,0 деталі за годину. У варіанті 2 четвертий робот збільшує її до 83,7 деталі за годину. В усіх випадках черга продовжує зростати.

Поєднання обох змін дає найменший час проходження: 349 с у варіанті 1 і 380 с у варіанті 2, а найбільша кількість деталей на площадці – 22-26. Порівняно з самим четвертим місцем на M2 час проходження зменшується на 15% у варіанті 1 і на 69% у варіанті 2. Водночас коефіцієнт використання роботів зменшується до 0,63-0,66, тобто четвертий робот використовується неповністю.

Отже, доцільно додати четверте місце обробки на верстаті M2 і зберегти закріплення роботів за шляхами (варіант 1). Це найменша зміна, після якої система обробляє всі деталі за умови завдання. Купувати четвертий робот доцільно, лише якщо потрібно додатково скоротити час проходження деталей. Тоді його варто закріпити за шляхом M1 → M2, що зменшує середній час проходження з 411 до 349 с.

5.3 Пропозиції щодо подальшого дослідження

Модель можна використати для таких подальших досліджень:

– правила призначення роботів, які враховують вузьке місце системи, наприклад перевагу перевезенням на M2 або змішане закріплення, коли один

робот закріплений за шляхом М1 → М2, а решта виконують будь-які перевезення. Для цього в моделі достатньо змінити склад СМО перевезень і предикати блокування;

– межа стійкості і запас продуктивності системи з чотирма місцями на М2, а також найвигідніше поєднання кількості місць на верстатах і кількості роботів з урахуванням їх вартості;

– вплив реального розташування обладнання, відмов верстатів і роботів, а також законів розподілу часу обробки на М2, відмінних від експоненціального.

5.4 Висновки до розділу

Найкращим з погляду пропускної здатності є закріплення по одному роботу за кожним шляхом переміщення деталей (варіант 1): його пропускна здатність на 8,8% більша, а межа стійкості вища, ніж у варіанті 2. Правило призначення найближчого робота програє через довші холості переїзди і несвоєчасну доставку деталей на верстат М2, який є вузьким місцем системи. За умови завдання обидва варіанти перевантажені. Пробні запуски показали, що четверте місце обробки на М2 робить систему стійкою при потоці 90 деталей за годину, а четвертий робот без цієї зміни недостатній.

ВИСНОВКИ

У курсовій роботі розроблено імітаційну модель експериментальної роботизованої гнучкої виробничої системи на основі формального опису мережею масового обслуговування і досліджено два способи закріплення роботів до операцій переміщення деталей.

Формалізовані моделі обох варіантів побудовано як розімкнені мережі масового обслуговування. Місця площадки прибуття, верстатів і складу та роботи представлено пристроями СМО зі станами «вільний», «зайнятий» і «блокований»: верстати – триканальними СМО без черги, роботів – трьома одноканальними СМО у варіанті 1 і триканальною СМО на кожному шляху у варіанті 2. Положення роботів і їх холості ходи у варіанті 2 відстежує окремий фрагмент мережі, запитами якого є роботи. Умови «немає вільного робота» і «на верстаті немає місця» описано предикатами блокування маршрутів через стан елементів мережі. Захоплення і вивільнення деталі та повернення закріпленого робота враховано затримкою звільнення пристроїв. Алгоритм імітації власний: просування часу до найближчої події, подіями є надходження деталі і завершення обслуговування в пристрої, а програма складається з класів, що відповідають елементам мережі МО. Статистичну обробку результатів експериментів виконано з використанням бібліотек NumPy і SciPy, рисунки побудовано бібліотекою Matplotlib. Повний код моделі та експериментального дослідження розміщено в репозиторії GitHub: https://github.com/tannnnnny/course_work_csdes.git [10]. Обидва варіанти реалізовано тим самим алгоритмом, вони відрізняються лише складом мережі і предикатами блокування. Коректність алгоритму підтверджено верифікацією в дев’яти сценаріях, точним збігом із ручним розрахунком у детермінованому сценарії, автоматичними тестами і протоколом подій.

Під час розробки моделі прийнято такі припущення: пункти розташовані на одній лінії, закріплений робот після вивільнення деталі повертається до стартової точки свого шляху, а у варіанті 2 залишається в пункті вивантаження, час обробки на першому верстаті має усічений нормальний закон, одночасні

події виконуються у випадковому порядку. Для варіанта 2 введено умову, за якою робот не бере деталь для верстата, до якого вже везуть деталі на одного робота більше, ніж там вільних місць. Без неї система переходила у стан взаємного блокування роботів. Розподіл часу проходження і місткість площадки визначено при надходженні exp(60), бо за умови завдання exp(40) система перевантажена, і ці характеристики сталих значень не мають. Усі інші умови завдання відтворено в моделі без спрощень.

Результати моделювання такі:

1) найкращим з погляду пропускної здатності є варіант 1, по одному роботу на кожному шляху: 83,1 дет./год проти 76,4 дет./год у варіанті 2 при надходженні exp(40); вплив способу закріплення значущий за критерієм Фішера;

2) варіант 1 залишається стійким до 80 дет./год, варіант 2 – до 75 дет./год; за умови завдання, 90 дет./год, перевантажені обидва варіанти, і черга на площадці прибуття зростає безмежно;

3) вузьким місцем системи є верстат М2. Варіант 2 програє через довші холості переїзди роботів і несвоєчасну доставку деталей на М2 та вивезення деталей з нього;

4) коефіцієнти використання при exp(40): у варіанті 1 роботів – 1,00, 1,00 і 0,60, верстатів М1 і М2 – 1,00 і 0,96; у варіанті 2 роботів – по 0,91, верстатів – 0,92 і 0,91;

5) час проходження деталей в обох варіантах розподілений за гамма-законом зі зсувом; при exp(60) середнє значення становить 300 с у варіанті 1 і 337 с у варіанті 2;

6) максимальна кількість деталей на площадці прибуття при exp(60) за 400 годин роботи становить у середньому 17 у варіанті 1 і 21 у варіанті 2; за умови завдання жодна скінченна місткість не достатня.

Запропоновано додати четверте місце обробки на верстаті М2 і зберегти закріплення роботів за шляхами. Пробні запуски показали, що після цієї зміни

система обробляє всі 90 деталей за годину, середній час проходження становить 411 с. Четвертий робот без зміни М2 проблему не розв’язує. Отже, імітаційне моделювання системи виконано успішно: мету роботи досягнуто, на всі питання завдання отримано відповіді із заданою точністю.

Формалізм мережі масового обслуговування добре відповідає системі, у якій деталі послідовно проходять обробку і транспортування. Найскладнішими в розробці виявилися дві особливості. Перша – одночасна зайнятість робота і місця під час захоплення і вивільнення деталі та блокування верстатів, коли робот тримає деталь біля заповненого верстата. Їх описано станами пристроїв, затримкою звільнення і блокуванням маршрутів. Друга – правило вибору найближчого робота у варіанті 2, для якого довелося окремо відстежувати маршрути роботів. Перевагою імітаційного методу є те, що він враховує ці особливості без спрощень, недоступних аналітичним методам, і дає змогу перевірити пропозиції без змін у реальній системі. Перевагою розробленого алгоритму є те, що обидва варіанти і всі пропозиції реалізовано однією програмою зі зміною лише параметрів, а спільні випадкові числа зменшують вплив випадковості на порівняння варіантів. Подальші дослідження доцільно спрямувати на правила призначення роботів, що враховують вузьке місце системи, і на пошук найвигіднішого поєднання кількості місць на верстатах і кількості роботів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Томашевський В. М. Моделювання систем. Київ : Видавнича група BHV, 2005. 352 с.
   URL: https://pdf.lib.vntu.edu.ua/books/2016/Tomashev_2005_352.pdf (дата звернення: 28.10.2026).
2. Стеценко І. В. Моделювання систем : навч. посіб. Черкаси : ЧДТУ, 2010. 399 с.
3. Стеценко І. В., Дифучина О. Ю., Дифучин А. Ю. Моделювання систем. Курсова робота : рекомендації до виконання курсової роботи : навч. посіб. Київ : КПІ ім. Ігоря Сікорського, 2024. 109 с .
4. Python 3 Documentation. Python Software Foundation. URL: https://docs.python.org/3/ (дата звернення: 28.10.2026).
5. Business Process Model and Notation (BPMN). Version 2.0.2. Object Management Group, 2013. URL: https://www.omg.org/spec/BPMN/2.0.2/(дата звернення: 28.10.2026).
6. NumPy Documentation. URL: https://numpy.org/doc/stable/ (дата звернення: 28.10.2026).
7. SciPy Documentation. URL: https://docs.scipy.org/doc/scipy/(дата звернення: 28.10.2026).
8. Matplotlib: Visualization with Python. URL: https://matplotlib.org/stable/ (дата звернення: 28.10.2026).
9. Unified Modeling Language (UML). Version 2.5.1. Object Management Group, 2017. URL: https://www.omg.org/spec/UML/2.5.1/(дата звернення: 28.10.2026).
10. Tanya Ponzel. course_work_csdes. GitHub.
    URL: https://github.com/tannnnnny/course_work_csdes.git (дата звернення: 02.11.2026).

ДОДАТОК А. ПРОТОКОЛИ ПОДІЙ

Протоколи отримано командою `uv run fms trace --until 600` при значеннях параметрів і вхідних змінних з таблиць 1.1 і 1.2 та початковому значенні генераторів 1. Наведено події до моменту 300 с. Кожен рядок містить момент події в секундах, пристрій мережі, у якому сталася подія, і зміст події. Пристрої позначено так: A.k, M1.k, M2.k, W.k – k-й пристрій СМО A, M1, M2, W; R1, R2, R3 – СМО роботів у варіанті 1; T1:R1, T2:R3 тощо – пристрій СМО перевезень T1–T3 у варіанті 2 і робот, який ним є; П:R1, Х:R1 тощо – пристрої фрагмента руху роботів; Create – надходження деталей, вихід – вихід з мережі. Запис «затримка звільнення до t» означає, що пристрій передав деталь далі, але залишається зайнятим до моменту t. Після кожної групи одночасних подій виведено стан мережі: кількість зайнятих пристроїв СМО A і стан K кожного пристрою інших СМО (0 – вільний, 1 – зайнятий, 2 – блокований), а у варіанті 2 також точку кожного робота, у якій він звільнився або куди прибув холостим ходом (R1@A – робот R1 у точці A).

A.1 Протокол подій для варіанта 1

```
        стан      A=0 M1[000] M2[000] R1[0] R2[0] R3[0]
  1.248 Create    надходження деталі #1
  1.248 A.1       деталь #1: перехід A -> R1, захоплення 7.540
  1.248 A.1       затримка звільнення до 8.788
  1.248 R1        деталь #1: обслуговування до 14.788
        стан      A=1 M1[000] M2[000] R1[1] R2[0] R3[0]
  8.788 A.1       пристрій вільний
        стан      A=0 M1[000] M2[000] R1[1] R2[0] R3[0]
 14.788 R1        деталь #1: перехід R1 -> M1, вивільнення 8.810
 14.788 R1        затримка звільнення до 29.597
 14.788 M1.1      деталь #1: обслуговування до 78.547
        стан      A=0 M1[100] M2[000] R1[1] R2[0] R3[0]
 16.321 Create    надходження деталі #2
 16.321 A.1       деталь #2: маршрут A -> R1 заблоковано (B0)
        стан      A=1 M1[100] M2[000] R1[1] R2[0] R3[0]
 29.597 R1        пристрій вільний
 29.597 A.1       деталь #2: маршрут A -> R1 відкрито (B0 = 0)
 29.597 A.1       деталь #2: перехід A -> R1, захоплення 8.279
 29.597 A.1       затримка звільнення до 37.877
 29.597 R1        деталь #2: обслуговування до 43.877
        стан      A=1 M1[100] M2[000] R1[1] R2[0] R3[0]
 37.877 A.1       пристрій вільний
        стан      A=0 M1[100] M2[000] R1[1] R2[0] R3[0]
 43.877 R1        деталь #2: перехід R1 -> M1, вивільнення 8.180
 43.877 R1        затримка звільнення до 58.057
 43.877 M1.2      деталь #2: обслуговування до 98.115
        стан      A=0 M1[110] M2[000] R1[1] R2[0] R3[0]
 58.057 R1        пристрій вільний
        стан      A=0 M1[110] M2[000] R1[0] R2[0] R3[0]
 78.547 M1.1      деталь #1: перехід M1 -> R2, захоплення 8.947
 78.547 M1.1      затримка звільнення до 87.494
 78.547 R2        деталь #1: обслуговування до 94.494
        стан      A=0 M1[110] M2[000] R1[0] R2[1] R3[0]
 87.494 M1.1      пристрій вільний
        стан      A=0 M1[010] M2[000] R1[0] R2[1] R3[0]
 94.494 R2        деталь #1: перехід R2 -> M2, вивільнення 8.003
 94.494 R2        затримка звільнення до 109.497
 94.494 M2.1      деталь #1: обслуговування до 112.780
        стан      A=0 M1[010] M2[100] R1[0] R2[1] R3[0]
 97.633 Create    надходження деталі #3
 97.633 A.1       деталь #3: перехід A -> R1, захоплення 8.517
 97.633 A.1       затримка звільнення до 106.150
 97.633 R1        деталь #3: обслуговування до 112.150
        стан      A=1 M1[010] M2[100] R1[1] R2[1] R3[0]
 98.115 M1.2      деталь #2: маршрут M1 -> R2 заблоковано (B2)
        стан      A=1 M1[020] M2[100] R1[1] R2[1] R3[0]
106.150 A.1       пристрій вільний
        стан      A=0 M1[020] M2[100] R1[1] R2[1] R3[0]
109.497 R2        пристрій вільний
109.497 M1.2      деталь #2: маршрут M1 -> R2 відкрито (B2 = 0)
109.497 M1.2      деталь #2: перехід M1 -> R2, захоплення 7.675
109.497 M1.2      затримка звільнення до 117.172
109.497 R2        деталь #2: обслуговування до 124.172
        стан      A=0 M1[010] M2[100] R1[1] R2[1] R3[0]
112.150 R1        деталь #3: перехід R1 -> M1, вивільнення 7.319
112.150 R1        затримка звільнення до 125.469
112.150 M1.1      деталь #3: обслуговування до 184.644
        стан      A=0 M1[110] M2[100] R1[1] R2[1] R3[0]
112.780 M2.1      деталь #1: перехід M2 -> R3, захоплення 7.464
112.780 M2.1      затримка звільнення до 120.244
112.780 R3        деталь #1: обслуговування до 125.244
        стан      A=0 M1[110] M2[100] R1[1] R2[1] R3[1]
117.172 M1.2      пристрій вільний
        стан      A=0 M1[100] M2[100] R1[1] R2[1] R3[1]
120.244 M2.1      пристрій вільний
        стан      A=0 M1[100] M2[000] R1[1] R2[1] R3[1]
124.172 R2        деталь #2: перехід R2 -> M2, вивільнення 7.073
124.172 R2        затримка звільнення до 138.245
```

```
124.172 M2.1      деталь #2: обслуговування до 161.581
        стан      A=0 M1[100] M2[100] R1[1] R2[1] R3[1]
125.244 R3        деталь #1: перехід R3 -> W, вивільнення 8.642
125.244 R3        затримка звільнення до 138.886
125.244 W.1       деталь #1: обслуговування до 133.886
        стан      A=0 M1[100] M2[100] R1[1] R2[1] R3[1]
125.469 R1        пристрій вільний
        стан      A=0 M1[100] M2[100] R1[0] R2[1] R3[1]
133.886 вихід     деталь #1 на складі, час проходження 132.638
        стан      A=0 M1[100] M2[100] R1[0] R2[1] R3[1]
138.245 R2        пристрій вільний
        стан      A=0 M1[100] M2[100] R1[0] R2[0] R3[1]
138.886 R3        пристрій вільний
        стан      A=0 M1[100] M2[100] R1[0] R2[0] R3[0]
161.581 M2.1      деталь #2: перехід M2 -> R3, захоплення 7.713
161.581 M2.1      затримка звільнення до 169.294
161.581 R3        деталь #2: обслуговування до 174.294
        стан      A=0 M1[100] M2[100] R1[0] R2[0] R3[1]
169.294 M2.1      пристрій вільний
        стан      A=0 M1[100] M2[000] R1[0] R2[0] R3[1]
169.390 Create    надходження деталі #4
169.390 A.1       деталь #4: перехід A -> R1, захоплення 7.483
169.390 A.1       затримка звільнення до 176.873
169.390 R1        деталь #4: обслуговування до 182.873
        стан      A=1 M1[100] M2[000] R1[1] R2[0] R3[1]
174.294 R3        деталь #2: перехід R3 -> W, вивільнення 8.911
174.294 R3        затримка звільнення до 188.205
174.294 W.1       деталь #2: обслуговування до 183.205
        стан      A=1 M1[100] M2[000] R1[1] R2[0] R3[1]
176.873 A.1       пристрій вільний
        стан      A=0 M1[100] M2[000] R1[1] R2[0] R3[1]
182.873 R1        деталь #4: перехід R1 -> M1, вивільнення 7.151
182.873 R1        затримка звільнення до 196.024
182.873 M1.2      деталь #4: обслуговування до 237.916
        стан      A=0 M1[110] M2[000] R1[1] R2[0] R3[1]
183.205 вихід     деталь #2 на складі, час проходження 166.884
        стан      A=0 M1[110] M2[000] R1[1] R2[0] R3[1]
184.644 M1.1      деталь #3: перехід M1 -> R2, захоплення 7.339
184.644 M1.1      затримка звільнення до 191.983
184.644 R2        деталь #3: обслуговування до 198.983
        стан      A=0 M1[110] M2[000] R1[1] R2[1] R3[1]
188.205 R3        пристрій вільний
        стан      A=0 M1[110] M2[000] R1[1] R2[1] R3[0]
191.983 M1.1      пристрій вільний
        стан      A=0 M1[010] M2[000] R1[1] R2[1] R3[0]
195.939 Create    надходження деталі #5
195.939 A.1       деталь #5: маршрут A -> R1 заблоковано (B0)
        стан      A=1 M1[010] M2[000] R1[1] R2[1] R3[0]
196.024 R1        пристрій вільний
196.024 A.1       деталь #5: маршрут A -> R1 відкрито (B0 = 0)
196.024 A.1       деталь #5: перехід A -> R1, захоплення 8.651
196.024 A.1       затримка звільнення до 204.675
196.024 R1        деталь #5: обслуговування до 210.675
        стан      A=1 M1[010] M2[000] R1[1] R2[1] R3[0]
198.983 R2        деталь #3: перехід R2 -> M2, вивільнення 7.305
198.983 R2        затримка звільнення до 213.288
198.983 M2.1      деталь #3: обслуговування до 219.283
        стан      A=1 M1[010] M2[100] R1[1] R2[1] R3[0]
204.675 A.1       пристрій вільний
        стан      A=0 M1[010] M2[100] R1[1] R2[1] R3[0]
210.675 R1        деталь #5: перехід R1 -> M1, вивільнення 8.663
210.675 R1        затримка звільнення до 225.339
210.675 M1.1      деталь #5: обслуговування до 285.635
        стан      A=0 M1[110] M2[100] R1[1] R2[1] R3[0]
213.288 R2        пристрій вільний
        стан      A=0 M1[110] M2[100] R1[1] R2[0] R3[0]
219.283 M2.1      деталь #3: перехід M2 -> R3, захоплення 8.619
219.283 M2.1      затримка звільнення до 227.902
219.283 R3        деталь #3: обслуговування до 232.902
        стан      A=0 M1[110] M2[100] R1[1] R2[0] R3[1]
```

```
225.339 R1        пристрій вільний
        стан      A=0 M1[110] M2[100] R1[0] R2[0] R3[1]
227.902 M2.1      пристрій вільний
        стан      A=0 M1[110] M2[000] R1[0] R2[0] R3[1]
230.992 Create    надходження деталі #6
230.992 A.1       деталь #6: перехід A -> R1, захоплення 7.573
230.992 A.1       затримка звільнення до 238.565
230.992 R1        деталь #6: обслуговування до 244.565
        стан      A=1 M1[110] M2[000] R1[1] R2[0] R3[1]
232.902 R3        деталь #3: перехід R3 -> W, вивільнення 8.025
232.902 R3        затримка звільнення до 245.927
232.902 W.1       деталь #3: обслуговування до 240.927
        стан      A=1 M1[110] M2[000] R1[1] R2[0] R3[1]
237.916 M1.2      деталь #4: перехід M1 -> R2, захоплення 8.268
237.916 M1.2      затримка звільнення до 246.184
237.916 R2        деталь #4: обслуговування до 253.184
        стан      A=1 M1[110] M2[000] R1[1] R2[1] R3[1]
238.565 A.1       пристрій вільний
        стан      A=0 M1[110] M2[000] R1[1] R2[1] R3[1]
240.927 вихід     деталь #3 на складі, час проходження 143.294
        стан      A=0 M1[110] M2[000] R1[1] R2[1] R3[1]
244.565 R1        деталь #6: перехід R1 -> M1, вивільнення 8.409
244.565 R1        затримка звільнення до 258.974
244.565 M1.3      деталь #6: обслуговування до 319.736
        стан      A=0 M1[111] M2[000] R1[1] R2[1] R3[1]
245.178 Create    надходження деталі #7
245.178 A.1       деталь #7: маршрут A -> R1 заблоковано (B0)
        стан      A=1 M1[111] M2[000] R1[1] R2[1] R3[1]
245.927 R3        пристрій вільний
        стан      A=1 M1[111] M2[000] R1[1] R2[1] R3[0]
246.184 M1.2      пристрій вільний
        стан      A=1 M1[101] M2[000] R1[1] R2[1] R3[0]
253.184 R2        деталь #4: перехід R2 -> M2, вивільнення 7.417
253.184 R2        затримка звільнення до 267.601
253.184 M2.1      деталь #4: обслуговування до 294.329
        стан      A=1 M1[101] M2[100] R1[1] R2[1] R3[0]
258.974 R1        пристрій вільний
258.974 A.1       деталь #7: маршрут A -> R1 відкрито (B0 = 0)
258.974 A.1       деталь #7: перехід A -> R1, захоплення 7.736
258.974 A.1       затримка звільнення до 266.710
258.974 R1        деталь #7: обслуговування до 272.710
        стан      A=1 M1[101] M2[100] R1[1] R2[1] R3[0]
266.710 A.1       пристрій вільний
        стан      A=0 M1[101] M2[100] R1[1] R2[1] R3[0]
267.601 R2        пристрій вільний
        стан      A=0 M1[101] M2[100] R1[1] R2[0] R3[0]
272.710 R1        деталь #7: перехід R1 -> M1, вивільнення 8.395
272.710 R1        затримка звільнення до 287.104
272.710 M1.2      деталь #7: обслуговування до 334.205
        стан      A=0 M1[111] M2[100] R1[1] R2[0] R3[0]
272.731 Create    надходження деталі #8
272.731 A.1       деталь #8: маршрут A -> R1 заблоковано (B0)
        стан      A=1 M1[111] M2[100] R1[1] R2[0] R3[0]
285.635 M1.1      деталь #5: перехід M1 -> R2, захоплення 7.975
285.635 M1.1      затримка звільнення до 293.610
285.635 R2        деталь #5: обслуговування до 300.610
        стан      A=1 M1[111] M2[100] R1[1] R2[1] R3[0]
287.104 R1        пристрій вільний
287.104 A.1       деталь #8: маршрут A -> R1 відкрито (B0 = 0)
287.104 A.1       деталь #8: перехід A -> R1, захоплення 7.877
287.104 A.1       затримка звільнення до 294.982
287.104 R1        деталь #8: обслуговування до 300.982
        стан      A=1 M1[111] M2[100] R1[1] R2[1] R3[0]
293.610 M1.1      пристрій вільний
        стан      A=1 M1[011] M2[100] R1[1] R2[1] R3[0]
```

A.2 Протокол подій для варіанта 2

```
       стан      A=0 M1[000] M2[000] T1[000] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
  1.248 Create    надходження деталі #1
  1.248 A.1       деталь #1: перехід A -> T1, захоплення 7.540
  1.248 A.1       затримка звільнення до 8.788
  1.248 T1:R1     деталь #1: обслуговування до 14.788
       стан      A=1 M1[000] M2[000] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
  8.788 A.1       пристрій вільний
       стан      A=0 M1[000] M2[000] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
 14.788 T1:R1     деталь #1: перехід T1 -> M1, вивільнення 8.810
 14.788 T1:R1     затримка звільнення до 23.597
 14.788 M1.1      деталь #1: обслуговування до 78.547
       стан      A=0 M1[100] M2[000] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
 16.321 Create    надходження деталі #2
 16.321 A.1       деталь #2: маршрут A -> T1 заблоковано (B0)
 16.321 П:R2      холостий хід M1 -> A (6 с) до деталі шляху 1
       стан      A=1 M1[100] M2[000] T1[100] T2[000] T3[000] X[010] R1@A R2@M1 R3@M2
 22.321 X:R2      кінець холостого ходу, робот у A
 22.321 A.1       деталь #2: маршрут A -> T1 відкрито (B0 = 0)
 22.321 A.1       деталь #2: перехід A -> T1, захоплення 8.279
 22.321 A.1       затримка звільнення до 30.601
 22.321 T1:R2     деталь #2: обслуговування до 36.601
       стан      A=1 M1[100] M2[000] T1[110] T2[000] T3[000] X[000] R1@A R2@A R3@M2
 23.597 T1:R1     робот вільний у M1
       стан      A=1 M1[100] M2[000] T1[010] T2[000] T3[000] X[000] R1@M1 R2@A R3@M2
 30.601 A.1       пристрій вільний
       стан      A=0 M1[100] M2[000] T1[010] T2[000] T3[000] X[000] R1@M1 R2@A R3@M2
 36.601 T1:R2     деталь #2: перехід T1 -> M1, вивільнення 8.180
 36.601 T1:R2     затримка звільнення до 44.781
 36.601 M1.2      деталь #2: обслуговування до 90.839
       стан      A=0 M1[110] M2[000] T1[010] T2[000] T3[000] X[000] R1@M1 R2@A R3@M2
 44.781 T1:R2     робот вільний у M1
       стан      A=0 M1[110] M2[000] T1[000] T2[000] T3[000] X[000] R1@M1 R2@M1 R3@M2
 78.547 M1.1      деталь #1: перехід M1 -> T2, захоплення 8.947
 78.547 M1.1      затримка звільнення до 87.494
 78.547 T2:R2     деталь #1: обслуговування до 94.494
       стан      A=0 M1[110] M2[000] T1[000] T2[010] T3[000] X[000] R1@M1 R2@M1 R3@M2
 87.494 M1.1      пристрій вільний
       стан      A=0 M1[010] M2[000] T1[000] T2[010] T3[000] X[000] R1@M1 R2@M1 R3@M2
 90.839 M1.2      деталь #2: перехід M1 -> T2, захоплення 8.003
 90.839 M1.2      затримка звільнення до 98.842
 90.839 T2:R1     деталь #2: обслуговування до 105.842
       стан      A=0 M1[010] M2[000] T1[000] T2[110] T3[000] X[000] R1@M1 R2@M1 R3@M2
 94.494 T2:R2     деталь #1: перехід T2 -> M2, вивільнення 8.517
 94.494 T2:R2     затримка звільнення до 103.011
 94.494 M2.1      деталь #1: обслуговування до 113.294
       стан      A=0 M1[010] M2[100] T1[000] T2[110] T3[000] X[000] R1@M1 R2@M1 R3@M2
 97.633 Create    надходження деталі #3
 97.633 A.1       деталь #3: маршрут A -> T1 заблоковано (B0)
 97.633 П:R3      холостий хід M2 -> A (13 с) до деталі шляху 1
       стан      A=1 M1[010] M2[100] T1[000] T2[110] T3[000] X[001] R1@M1 R2@M1 R3@M2
 98.842 M1.2      пристрій вільний
       стан      A=1 M1[000] M2[100] T1[000] T2[110] T3[000] X[001] R1@M1 R2@M1 R3@M2
103.011 T2:R2     робот вільний у M2
       стан      A=1 M1[000] M2[100] T1[000] T2[100] T3[000] X[001] R1@M1 R2@M2 R3@M2
105.842 T2:R1     деталь #2: перехід T2 -> M2, вивільнення 7.675
105.842 T2:R1     затримка звільнення до 113.517
105.842 M2.2      деталь #2: обслуговування до 143.854
       стан      A=1 M1[000] M2[110] T1[000] T2[100] T3[000] X[001] R1@M1 R2@M2 R3@M2
110.633 X:R3      кінець холостого ходу, робот у A
110.633 A.1       деталь #3: маршрут A -> T1 відкрито (B0 = 0)
110.633 A.1       деталь #3: перехід A -> T1, захоплення 7.319
110.633 A.1       затримка звільнення до 117.952
110.633 T1:R3     деталь #3: обслуговування до 123.952
       стан      A=1 M1[000] M2[110] T1[001] T2[100] T3[000] X[000] R1@M1 R2@M2 R3@A
113.294 M2.1      деталь #1: перехід M2 -> T3, захоплення 7.464
113.294 M2.1      затримка звільнення до 120.758
113.294 T3:R2     деталь #1: обслуговування до 125.758
```

```
        стан      A=1 M1[000] M2[110] T1[001] T2[100] T3[010] X[000] R1@M1 R2@M2 R3@A
113.517 T2:R1     робот вільний у M2
        стан      A=1 M1[000] M2[110] T1[001] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@A
117.952 A.1       пристрій вільний
        стан      A=0 M1[000] M2[110] T1[001] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@A
120.758 M2.1      пристрій вільний
        стан      A=0 M1[000] M2[010] T1[001] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@A
123.952 T1:R3     деталь #3: перехід T1 -> M1, вивільнення 7.073
123.952 T1:R3     затримка звільнення до 131.025
123.952 M1.1      деталь #3: обслуговування до 196.200
        стан      A=0 M1[100] M2[010] T1[001] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@A
125.758 T3:R2     деталь #1: перехід T3 -> W, вивільнення 8.642
125.758 T3:R2     затримка звільнення до 134.400
125.758 W.1       деталь #1: обслуговування до 134.400
        стан      A=0 M1[100] M2[010] T1[001] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@A
131.025 T1:R3     робот вільний у M1
        стан      A=0 M1[100] M2[010] T1[000] T2[000] T3[010] X[000] R1@M2 R2@M2 R3@M1
134.400 T3:R2     робот вільний у W
134.400 вихід     деталь #1 на складі, час проходження 133.152
        стан      A=0 M1[100] M2[010] T1[000] T2[000] T3[000] X[000] R1@M2 R2@W R3@M1
143.854 M2.2      деталь #2: перехід M2 -> T3, захоплення 7.713
143.854 M2.2      затримка звільнення до 151.567
143.854 T3:R1     деталь #2: обслуговування до 156.567
        стан      A=0 M1[100] M2[010] T1[000] T2[000] T3[100] X[000] R1@M2 R2@W R3@M1
151.567 M2.2      пристрій вільний
        стан      A=0 M1[100] M2[000] T1[000] T2[000] T3[100] X[000] R1@M2 R2@W R3@M1
156.567 T3:R1     деталь #2: перехід T3 -> W, вивільнення 7.483
156.567 T3:R1     затримка звільнення до 164.050
156.567 W.1       деталь #2: обслуговування до 164.050
        стан      A=0 M1[100] M2[000] T1[000] T2[000] T3[100] X[000] R1@M2 R2@W R3@M1
164.050 вихід     деталь #2 на складі, час проходження 147.728
164.050 T3:R1     робот вільний у W
        стан      A=0 M1[100] M2[000] T1[000] T2[000] T3[000] X[000] R1@W R2@W R3@M1
169.390 Create    надходження деталі #4
169.390 A.1       деталь #4: маршрут A -> T1 заблоковано (B0)
169.390 П:R3      холостий хід M1 -> A (6 с) до деталі шляху 1
        стан      A=1 M1[100] M2[000] T1[000] T2[000] T3[000] X[001] R1@W R2@W R3@M1
175.390 X:R3      кінець холостого ходу, робот у A
175.390 A.1       деталь #4: маршрут A -> T1 відкрито (B0 = 0)
175.390 A.1       деталь #4: перехід A -> T1, захоплення 8.911
175.390 A.1       затримка звільнення до 184.301
175.390 T1:R3     деталь #4: обслуговування до 190.301
        стан      A=1 M1[100] M2[000] T1[001] T2[000] T3[000] X[000] R1@W R2@W R3@A
184.301 A.1       пристрій вільний
        стан      A=0 M1[100] M2[000] T1[001] T2[000] T3[000] X[000] R1@W R2@W R3@A
190.301 T1:R3     деталь #4: перехід T1 -> M1, вивільнення 7.151
190.301 T1:R3     затримка звільнення до 197.452
190.301 M1.2      деталь #4: обслуговування до 245.344
        стан      A=0 M1[110] M2[000] T1[001] T2[000] T3[000] X[000] R1@W R2@W R3@A
195.939 Create    надходження деталі #5
195.939 A.1       деталь #5: маршрут A -> T1 заблоковано (B0)
195.939 П:R1      холостий хід W -> A (18 с) до деталі шляху 1
        стан      A=1 M1[110] M2[000] T1[001] T2[000] T3[000] X[100] R1@W R2@W R3@A
196.200 M1.1      деталь #3: маршрут M1 -> T2 заблоковано (B2)
196.200 П:R2      холостий хід W -> M1 (12 с) до деталі шляху 2
        стан      A=1 M1[210] M2[000] T1[001] T2[000] T3[000] X[110] R1@W R2@W R3@A
197.452 T1:R3     робот вільний у M1
197.452 M1.1      деталь #3: маршрут M1 -> T2 відкрито (B2 = 0)
197.452 M1.1      деталь #3: перехід M1 -> T2, захоплення 7.339
197.452 M1.1      затримка звільнення до 204.791
197.452 T2:R3     деталь #3: обслуговування до 211.791
        стан      A=1 M1[110] M2[000] T1[000] T2[001] T3[000] X[110] R1@W R2@W R3@M1
204.791 M1.1      пристрій вільний
        стан      A=1 M1[010] M2[000] T1[000] T2[001] T3[000] X[110] R1@W R2@W R3@M1
208.200 X:R2      кінець холостого ходу, робот у M1
        стан      A=1 M1[010] M2[000] T1[000] T2[001] T3[000] X[100] R1@W R2@M1 R3@M1
211.791 T2:R3     деталь #3: перехід T2 -> M2, вивільнення 8.651
211.791 T2:R3     затримка звільнення до 220.442
211.791 M2.1      деталь #3: обслуговування до 233.438
        стан      A=1 M1[010] M2[100] T1[000] T2[001] T3[000] X[100] R1@W R2@M1 R3@M1
```

```
213.939 X:R1      кінець холостого ходу, робот у A
213.939 A.1       деталь #5: маршрут A -> T1 відкрито (B0 = 0)
213.939 A.1       деталь #5: перехід A -> T1, захоплення 7.305
213.939 A.1       затримка звільнення до 221.243
213.939 T1:R1     деталь #5: обслуговування до 227.243
        стан      A=1 M1[010] M2[100] T1[100] T2[001] T3[000] X[000] R1@A R2@M1 R3@M1
220.442 T2:R3     робот вільний у M2
        стан      A=1 M1[010] M2[100] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
221.243 A.1       пристрій вільний
        стан      A=0 M1[010] M2[100] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
227.243 T1:R1     деталь #5: перехід T1 -> M1, вивільнення 8.663
227.243 T1:R1     затримка звільнення до 235.907
227.243 M1.1      деталь #5: обслуговування до 302.203
        стан      A=0 M1[110] M2[100] T1[100] T2[000] T3[000] X[000] R1@A R2@M1 R3@M2
230.992 Create    надходження деталі #6
230.992 A.1       деталь #6: маршрут A -> T1 заблоковано (B0)
230.992 П:R2      холостий хід M1 -> A (6 с) до деталі шляху 1
        стан      A=1 M1[110] M2[100] T1[100] T2[000] T3[000] X[010] R1@A R2@M1 R3@M2
233.438 M2.1      деталь #3: перехід M2 -> T3, захоплення 8.619
233.438 M2.1      затримка звільнення до 242.056
233.438 T3:R3     деталь #3: обслуговування до 247.056
        стан      A=1 M1[110] M2[100] T1[100] T2[000] T3[001] X[010] R1@A R2@M1 R3@M2
235.907 T1:R1     робот вільний у M1
        стан      A=1 M1[110] M2[100] T1[000] T2[000] T3[001] X[010] R1@M1 R2@M1 R3@M2
236.992 X:R2      кінець холостого ходу, робот у A
236.992 A.1       деталь #6: маршрут A -> T1 відкрито (B0 = 0)
236.992 A.1       деталь #6: перехід A -> T1, захоплення 7.573
236.992 A.1       затримка звільнення до 244.565
236.992 T1:R2     деталь #6: обслуговування до 250.565
        стан      A=1 M1[110] M2[100] T1[010] T2[000] T3[001] X[000] R1@M1 R2@A R3@M2
242.056 M2.1      пристрій вільний
        стан      A=1 M1[110] M2[000] T1[010] T2[000] T3[001] X[000] R1@M1 R2@A R3@M2
244.565 A.1       пристрій вільний
        стан      A=0 M1[110] M2[000] T1[010] T2[000] T3[001] X[000] R1@M1 R2@A R3@M2
245.178 Create    надходження деталі #7
245.178 A.1       деталь #7: маршрут A -> T1 заблоковано (B0)
245.178 П:R1      холостий хід M1 -> A (6 с) до деталі шляху 1
        стан      A=1 M1[110] M2[000] T1[010] T2[000] T3[001] X[100] R1@M1 R2@A R3@M2
245.344 M1.2      деталь #4: маршрут M1 -> T2 заблоковано (B2)
        стан      A=1 M1[120] M2[000] T1[010] T2[000] T3[001] X[100] R1@M1 R2@A R3@M2
247.056 T3:R3     деталь #3: перехід T3 -> W, вивільнення 8.025
247.056 T3:R3     затримка звільнення до 255.081
247.056 W.1       деталь #3: обслуговування до 255.081
        стан      A=1 M1[120] M2[000] T1[010] T2[000] T3[001] X[100] R1@M1 R2@A R3@M2
250.565 T1:R2     деталь #6: перехід T1 -> M1, вивільнення 8.268
250.565 T1:R2     затримка звільнення до 258.833
250.565 M1.3      деталь #6: обслуговування до 325.595
        стан      A=1 M1[121] M2[000] T1[010] T2[000] T3[001] X[100] R1@M1 R2@A R3@M2
251.178 X:R1      кінець холостого ходу, робот у A
251.178 A.1       деталь #7: маршрут A -> T1 відкрито (B0 = 0)
251.178 A.1       деталь #7: перехід A -> T1, захоплення 8.409
251.178 A.1       затримка звільнення до 259.587
251.178 T1:R1     деталь #7: обслуговування до 265.587
        стан      A=1 M1[121] M2[000] T1[110] T2[000] T3[001] X[000] R1@A R2@A R3@M2
255.081 T3:R3     робот вільний у W
255.081 вихід     деталь #3 на складі, час проходження 157.448
255.081 П:R3      холостий хід W -> M1 (12 с) до деталі шляху 2
        стан      A=1 M1[121] M2[000] T1[110] T2[000] T3[000] X[001] R1@A R2@A R3@W
258.833 T1:R2     робот вільний у M1
258.833 M1.2      деталь #4: маршрут M1 -> T2 відкрито (B2 = 0)
258.833 M1.2      деталь #4: перехід M1 -> T2, захоплення 7.417
258.833 M1.2      затримка звільнення до 266.250
258.833 T2:R2     деталь #4: обслуговування до 273.250
        стан      A=1 M1[111] M2[000] T1[100] T2[010] T3[000] X[001] R1@A R2@M1 R3@W
259.587 A.1       пристрій вільний
        стан      A=0 M1[111] M2[000] T1[100] T2[010] T3[000] X[001] R1@A R2@M1 R3@W
265.587 T1:R1     деталь #7: маршрут T1 -> M1 заблоковано (B1)
        стан      A=0 M1[111] M2[000] T1[200] T2[010] T3[000] X[001] R1@A R2@M1 R3@W
266.250 M1.2      пристрій вільний
266.250 T1:R1     деталь #7: маршрут T1 -> M1 відкрито (B1 = 0)
```

```
266.250 T1:R1     деталь #7: перехід T1 -> M1, вивільнення 7.736
266.250 T1:R1     затримка звільнення до 273.986
266.250 M1.2      деталь #7: обслуговування до 327.086
        стан      A=0 M1[111] M2[000] T1[100] T2[010] T3[000] X[001] R1@A R2@M1 R3@W
267.081 X:R3      кінець холостого ходу, робот у M1
        стан      A=0 M1[111] M2[000] T1[100] T2[010] T3[000] X[000] R1@A R2@M1 R3@M1
272.731 Create    надходження деталі #8
272.731 A.1       деталь #8: маршрут A -> T1 заблоковано (B0)
272.731 П:R3      холостий хід M1 -> A (6 с) до деталі шляху 1
        стан      A=1 M1[111] M2[000] T1[100] T2[010] T3[000] X[001] R1@A R2@M1 R3@M1
273.250 T2:R2     деталь #4: перехід T2 -> M2, вивільнення 8.395
273.250 T2:R2     затримка звільнення до 281.645
273.250 M2.1      деталь #4: обслуговування до 315.373
        стан      A=1 M1[111] M2[100] T1[100] T2[010] T3[000] X[001] R1@A R2@M1 R3@M1
273.986 T1:R1     робот вільний у M1
        стан      A=1 M1[111] M2[100] T1[000] T2[010] T3[000] X[001] R1@M1 R2@M1 R3@M1
278.731 X:R3      кінець холостого ходу, робот у A
278.731 A.1       деталь #8: маршрут A -> T1 відкрито (B0 = 0)
278.731 A.1       деталь #8: перехід A -> T1, захоплення 7.975
278.731 A.1       затримка звільнення до 286.705
278.731 T1:R3     деталь #8: обслуговування до 292.705
        стан      A=1 M1[111] M2[100] T1[001] T2[010] T3[000] X[000] R1@M1 R2@M1 R3@A
281.645 T2:R2     робот вільний у M2
        стан      A=1 M1[111] M2[100] T1[001] T2[000] T3[000] X[000] R1@M1 R2@M2 R3@A
286.705 A.1       пристрій вільний
        стан      A=0 M1[111] M2[100] T1[001] T2[000] T3[000] X[000] R1@M1 R2@M2 R3@A
292.705 T1:R3     деталь #8: маршрут T1 -> M1 заблоковано (B1)
        стан      A=0 M1[111] M2[100] T1[002] T2[000] T3[000] X[000] R1@M1 R2@M2 R3@A
```

ДОДАТОК Б. ЛІСТИНГ КОДУ АЛГОРИТМУ ІМІТАЦІЇ

Наведено модулі пакета `fms`, що реалізують алгоритм імітації (таблиця 3.1). Повний код програми, зокрема командний інтерфейс, автоматичні тести і побудову рисунків, розміщено в репозиторії проєкту.

Б.1 Модуль rng.py – генератори випадкових величин

```
from __future__ import annotations

import math
import random
from dataclasses import dataclass


class Stream:

    def __init__(self, seed: int | str):
        self._rnd = random.Random(seed)
        self._spare_normal: float | None = None

    def zeta(self) -> float:
        return 1.0 - self._rnd.random()

    def exponential(self, mean: float) -> float:
        return -mean * math.log(self.zeta())

    def uniform(self, center: float, half_width: float) -> float:
        return center - half_width + 2.0 * half_width * self._rnd.random()

    def standard_normal(self) -> float:
        if self._spare_normal is not None:
            value, self._spare_normal = self._spare_normal, None
            return value
        while True:
            a1 = 2.0 * self._rnd.random() - 1.0
            a2 = 2.0 * self._rnd.random() - 1.0
            s = a1 * a1 + a2 * a2
            if 0.0 < s <= 1.0:
                break
        beta = math.sqrt(-2.0 * math.log(s) / s)
        self._spare_normal = a2 * beta
        return a1 * beta

    def normal(self, mean: float, std: float) -> float:
        return mean + std * self.standard_normal()

    def shuffle(self, items: list) -> None:
        self._rnd.shuffle(items)

    def random(self) -> float:
        return self._rnd.random()

    def choice(self, items: list):
        return items[0] if len(items) == 1 else self._rnd.choice(items)


@dataclass(frozen=True)
class Dist:

    kind: str
    a: float
    b: float = 0.0

    def sample(self, stream: Stream) -> float:
        if self.kind == "const":
            return self.a
        if self.kind == "exp":
            return stream.exponential(self.a)
        if self.kind == "unif":
            return stream.uniform(self.a, self.b)
        if self.kind == "norm":
            while True:
                value = stream.normal(self.a, self.b)
                if value > 0.0:
```

```
                    return value
        raise ValueError(f"Невідомий закон розподілу: {self.kind}")

    @property
    def mean(self) -> float:
        return self.a

    def __str__(self) -> str:
        if self.kind == "const":
            return f"{self.a:g}"
        if self.kind == "exp":
            return f"exp({self.a:g})"
        return f"{self.kind}({self.a:g},{self.b:g})"


def const(d: float) -> Dist:
    return Dist("const", d)


def exp(mean: float) -> Dist:
    return Dist("exp", mean)


def unif(center: float, half_width: float) -> Dist:
    return Dist("unif", center, half_width)


def norm(mean: float, std: float) -> Dist:
    return Dist("norm", mean, std)


STREAM_NAMES = ("arrivals", "m1", "m2", "handling", "ties")


class RandomStreams:

    def __init__(self, seed: int):
        self.seed = seed
        for name in STREAM_NAMES:
            setattr(self, name, Stream(f"{seed}:{name}"))

    arrivals: Stream
    m1: Stream
    m2: Stream
    handling: Stream
    ties: Stream
```

Б.2 Модуль params.py – вхідні змінні та параметри моделі

```
from __future__ import annotations

from dataclasses import dataclass, field, replace

from .rng import Dist, exp, norm, unif

DEDICATED = "dedicated"  # 1) по одному роботу на кожний шлях переміщення
NEAREST = "nearest"      # 2) будь-який робот на будь-якому шляху, займається найближчий


DEFAULT_POSITIONS = {"A": 0.0, "M1": 6.0, "M2": 13.0, "W": 18.0}


@dataclass(frozen=True)
class Params:
    arrival: Dist = exp(40.0)        # інтервал надходження деталей
    m1_time: Dist = norm(60.0, 10.0)  # час обробки на першому верстаті
    m2_time: Dist = exp(100.0)        # час обробки на другому верстаті
    handling: Dist = unif(8.0, 1.0)   # час захоплення або вивільнення деталі роботом

    m1_capacity: int = 3
    m2_capacity: int = 3
    n_robots: int = 3
    positions: dict[str, float] = field(default_factory=lambda: dict(DEFAULT_POSITIONS))

    strategy: str = DEDICATED
    dedicated_routes: tuple[int, ...] | None = None

    def distance(self, a: str, b: str) -> float:
        return abs(self.positions[a] - self.positions[b])

    def with_(self, **changes) -> "Params":
        return replace(self, **changes)

    def scaled_travel(self, factor: float) -> "Params":
        return replace(self, positions={k: v * factor for k, v in
self.positions.items()})

    def robot_routes(self) -> list[tuple[int, ...]]:
        if self.strategy == NEAREST:
            return [(1, 2, 3)] * self.n_robots
        if self.strategy == DEDICATED:
            routes = self.dedicated_routes or tuple(k % 3 + 1 for k in
range(self.n_robots))
            if len(routes) != self.n_robots or set(routes) != {1, 2, 3}:
                raise ValueError(f"dedicated_routes має покривати маршрути 1–3 для
кожного з "
                                 f"{self.n_robots} роботів: {routes}")
            return [(r,) for r in routes]
        raise ValueError(f"Невідома стратегія: {self.strategy}")

    def describe(self) -> str:
        pos = ", ".join(f"{k}={v:g}" for k, v in self.positions.items())
        return (
            f"strategy={self.strategy}, "
            f"arrival={self.arrival}, M1={self.m1_time}, M2={self.m2_time}, "
            f"handling={self.handling}, cap=({self.m1_capacity},{self.m2_capacity}), "
            f"robots={self.n_robots}"
            + (f", routes={self.dedicated_routes}" if self.dedicated_routes else "")
            + f", positions[{pos}]"
        )
```

Б.3 Модуль elements.py – елементи мережі масового обслуговування

```
from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Callable

if TYPE_CHECKING:
    from .model import Model

INF = math.inf
FREE, BUSY, BLOCKED = 0, 1, 2


@dataclass(eq=False)
class Part:

    id: int
    t_arrival: float
    t_ready: dict[int, float] = field(default_factory=dict)  # шлях -> момент готовності
до перевезення
    t_grab: dict[int, float] = field(default_factory=dict)   # шлях -> початок захоплення
    t_done: float = INF

    def __str__(self) -> str:
        return f"деталь #{self.id}"


@dataclass(eq=False)
class Robot:

    id: int
    pos: str
    target: str = ""

    def __str__(self) -> str:
        return f"R{self.id}"


class Channel:

    __slots__ = ("qs", "index", "K", "tnext", "request", "t_from", "t_split", "seq")

    def __init__(self, qs: "QS", index: int):
        self.qs = qs
        self.index = index
        self.K = FREE
        self.tnext = INF
        self.request = None
        self.t_from = 0.0     # початок поточного стану
        self.t_split = 0.0    # кінець захоплення або вивільнення в поточному стані
        self.seq = 0          # номер запису в календарі подій

    @property
    def name(self) -> str:
        qs = self.qs
        if qs.names:
            return qs.names[self.index]
        return qs.name if len(qs.channels) == 1 and not qs.unlimited else
f"{qs.name}.{self.index + 1}"


CATS = ("free", "busy_h", "busy", "post_h", "post", "blocked")


class QS:
```

```
    def __init__(self, model: "Model", name: str, m: int | None,
                 service: Callable, release: Callable | None = None,
                 on_done: Callable | None = None, on_free: Callable | None = None,
                 names: list[str] | None = None):
        self.model = model
        self.name = name
        self.names = names
        self.unlimited = m is None
        self.channels: list[Channel] = [] if m is None else [Channel(self, j) for j in
range(m)]
        self._spare: list[Channel] = []
        self.service = service
        self.release = release or (lambda req, h: 0.0)
        self.on_done = on_done
        self.on_free = on_free
        self.route: Route | None = None
        self.n = [len(self.channels), 0, 0]   # кількість каналів у станах 0, 1, 2
        self.requests = 0                     # кількість запитів у СМО
        self.reset_stats()

    def set_state(self, ch: Channel, K: int, t: float) -> None:
        self.n[ch.K] -= 1
        self.n[K] += 1
        ch.K = K
        ch.t_from = t

    def free_channel(self) -> Channel | None:
        if self.unlimited:
            if self._spare:
                return self._spare.pop()
            ch = Channel(self, len(self.channels))
            self.channels.append(ch)
            self.n[FREE] += 1
            return ch
        return next((ch for ch in self.channels if ch.K == FREE), None)

    def occupied(self) -> int:
        return self.n[BUSY] + self.n[BLOCKED]

    def accept(self, req, t: float, h: float = 0.0, ch: Channel | None = None) ->
Channel:
        ch = ch or self.free_channel()
        ch.request = req
        self.requests += 1
        self.set_state(ch, BUSY, t)
        d = self.service(req, h)
        ch.t_split = t + h
        if d > 0:
            self.model.schedule(ch, t + d)
            if isinstance(req, Part):
                self.model.log(t, ch.name, f"{req}: обслуговування до {t + d:.3f}")
        else:                                 # нульова затримка: запит одразу намагається
вийти
            ch.tnext = t
            self.end_of_service(ch, t)
        return ch

    def end_of_service(self, ch: Channel, t: float) -> None:
        ch.tnext = INF
        if ch.request is None:                # кінець затримки звільнення: пристрій вільний
            self._vacated(ch, t, delayed=True)
            return
        if self.on_done:
            self.on_done(ch.request, t)
        route = self.route
        if route.blocked():                   # B(S, t) = 1: запит лишається в пристрої
            self.set_state(ch, BLOCKED, t)
            route.waiting.append(ch)
            if isinstance(ch.request, Part):
                self.model.log(t, ch.name, f"{ch.request}: маршрут {route.name}
```

```
заблоковано ({route.label})")
        else:
            route.transfer(ch, t)

    def vacate(self, ch: Channel, t: float, h: float) -> None:
        req = ch.request
        ch.request = None
        self.requests -= 1
        delay = self.release(req, h)
        if delay > 0:
            self.set_state(ch, BUSY, t)
            ch.t_split = t + h
            self.model.schedule(ch, t + delay)
            self.model.log(t, ch.name, f"затримка звільнення до {t + delay:.3f}")
        else:
            self._vacated(ch, t)

    def _vacated(self, ch: Channel, t: float, delayed: bool = False) -> None:
        if delayed and not self.on_free:
            self.model.log(t, ch.name, "пристрій вільний")
        self.set_state(ch, FREE, t)
        if self.unlimited:
            self._spare.append(ch)
        if self.on_free:
            self.on_free(ch, t)

    def reset_stats(self) -> None:
        self.time = {ch.index: dict.fromkeys(CATS, 0.0) for ch in self.channels} if not
self.unlimited else {}

    def update_stats(self, t1: float, t2: float) -> None:
        if self.unlimited:
            return
        for ch in self.channels:
            acc = self.time[ch.index]
            dt = t2 - t1
            if ch.K == FREE:
                acc["free"] += dt
            elif ch.K == BLOCKED:
                acc["blocked"] += dt
            else:
                h = max(0.0, min(t2, ch.t_split) - t1)     # частина інтервалу до кінця
захоплення/вивільнення
                if ch.request is not None:
                    acc["busy_h"] += h
                    acc["busy"] += dt - h
                else:
                    acc["post_h"] += h
                    acc["post"] += dt - h

    def share(self, period: float, cats, channels=None) -> float:
        chs = channels if channels is not None else [ch.index for ch in self.channels]
        if period <= 0 or not chs:
            return math.nan
        return sum(self.time[j][c] for j in chs for c in cats) / (period * len(chs))

    def state_str(self) -> str:
        return f"{self.name}[{''.join(str(ch.K) for ch in self.channels)}]"


class Route:

    def __init__(self, model: "Model", src, dst, blocked: Callable[[], bool] | None =
None,
                 label: str = "", handling: str = "", select: Callable | None = None,
                 on_transfer: Callable | None = None):
        self.model = model
        self.src, self.dst = src, dst
        self.blocked = blocked or (lambda: False)
        self.label = label
```

```
        self.handling = handling      # «захоплення» або «вивільнення», якщо перехід їх
супроводжує
        self.select = select
        self.on_transfer = on_transfer
        self.waiting: deque[Channel] = deque()   # заблоковані канали src у порядку
блокування
        self.name = f"{src.name} -> {dst.name}"
        src.route = self

    def transfer(self, ch: Channel, t: float) -> None:
        req = ch.request
        h = self.model.handling_time() if self.handling else 0.0
        dst_ch = self.select(req) if self.select else None
        if isinstance(req, Part) and self.dst is not self.model.exit:
            self.model.log(t, ch.name, f"{req}: перехід {self.name}" + (f",
{self.handling} {h:.3f}" if h else ""))
        if self.on_transfer:
            self.on_transfer(req, ch, dst_ch, t)
        self.src.vacate(ch, t, h)
        self.dst.accept(req, t, h, dst_ch)

    def unblock(self, t: float) -> bool:
        moved = False
        while self.waiting and not self.blocked():
            ch = self.waiting.popleft()
            self.model.log(t, ch.name, f"{ch.request}: маршрут {self.name} відкрито
({self.label} = 0)")
            self.transfer(ch, t)
            moved = True
        return moved


class Create:

    def __init__(self, model: "Model", name: str, dist, stream):
        self.model = model
        self.name = name
        self.dist = dist
        self.stream = stream
        self.tnext = dist.sample(stream)
        self.created = 0
        self.route: Route | None = None

    def arrival(self, t: float) -> None:
        self.created += 1
        part = Part(id=self.created, t_arrival=t)
        self.model.log(t, self.name, f"надходження деталі #{part.id}")
        self.route.dst.accept(part, t)
        self.tnext = t + self.dist.sample(self.stream)


class Exit:

    name = "вихід"

    def __init__(self, model: "Model"):
        self.model = model

    def accept(self, part: Part, t: float, h: float = 0.0, ch=None) -> None:
        self.model.record_completion(part, t)
```

Б.4 Модуль model.py – алгоритм імітації

```
from __future__ import annotations

import heapq
import math
from collections import defaultdict
from typing import Callable

from .elements import BLOCKED, FREE, INF, Channel, Create, Exit, Part, QS, Robot, Route
from .params import DEDICATED, Params
from .rng import RandomStreams

SRC = {1: "A", 2: "M1", 3: "M2"}     # точка завантаження шляху S
DST = {1: "M1", 2: "M2", 3: "W"}     # точка вивантаження шляху S


class Model:
    def __init__(self, params: Params, seed: int = 1, trace: bool = False, trace_until:
float = INF):
        self.params = params
        self.streams = RandomStreams(seed)
        self.seed = seed
        self.tcurr = 0.0
        self.trace = False
        self.trace_until = trace_until
        self.trace_lines: list[str] = []
        self.calendar: list = []          # календар подій: (момент, номер запису, канал)
        self._seq = 0
        self.completed: list[Part] = []
        self.completed_total = 0
        self.t_stats_start = 0.0
        self._created_at_reset = 0
        self.dedicated = params.strategy == DEDICATED
        p = params
        self.travel = {S: p.distance(SRC[S], DST[S]) for S in (1, 2, 3)}

        self.create = Create(self, "Create", p.arrival, self.streams.arrivals)
        self.A = QS(self, "A", None, service=lambda part, h: 0.0, release=lambda part, h:
h,
                    on_done=lambda part, t: self._ready(part, 1, t))
        self.M1 = QS(self, "M1", p.m1_capacity, service=lambda part, h: h +
p.m1_time.sample(self.streams.m1),
                     release=lambda part, h: h, on_done=lambda part, t: self._ready(part,
2, t))
        self.M2 = QS(self, "M2", p.m2_capacity, service=lambda part, h: h +
p.m2_time.sample(self.streams.m2),
                     release=lambda part, h: h, on_done=lambda part, t: self._ready(part,
3, t))
        self.W = QS(self, "W", None, service=lambda part, h: h)
        self.exit = Exit(self)
        self.M = {1: self.M1, 2: self.M2}

        self.T: dict[int, QS] = {}
        self.robot_channels: dict[int, list[Channel]] = defaultdict(list)   # робот ->
його пристрої
        self.robots: list[Robot] = []
        routes_of = p.robot_routes()
        if self.dedicated:
            for S in (1, 2, 3):
                ids = [k + 1 for k, r in enumerate(routes_of) if r[0] == S]
                qs = QS(self, f"R{S}", len(ids), service=lambda part, h, S=S: h +
self.travel[S],
                        release=lambda part, h, S=S: h + self.travel[S])
                for ch, k in zip(qs.channels, ids):
                    self.robot_channels[k].append(ch)
                self.T[S] = qs
        else:
            n = p.n_robots
```

```
            self.robots = [Robot(id=k + 1, pos=("A", "M1", "M2")[k % 3]) for k in
range(n)]
            for S in (1, 2, 3):
                qs = QS(self, f"T{S}", n, service=lambda part, h, S=S: h +
self.travel[S],
                        release=lambda part, h: h, on_free=lambda ch, t, S=S:
self._robot_at(ch, DST[S], t),
                        names=[f"T{S}:R{k + 1}" for k in range(n)])
                for ch in qs.channels:
                    self.robot_channels[ch.index + 1].append(ch)
                self.T[S] = qs
            self.P = QS(self, "П", n, service=lambda robot, h: 0.0, names=[f"П:R{k + 1}"
for k in range(n)])
            self.X = QS(self, "X", n, service=lambda robot, h: p.distance(robot.pos,
robot.target),
                        names=[f"X:R{k + 1}" for k in range(n)])
            for ch in self.X.channels:
                self.robot_channels[ch.index + 1].append(ch)

        Route(self, self.create, self.A)
        B = self._blockings()
        self.routes: list[Route] = [
            Route(self, self.T[1], self.M1, B[1], "B1", handling="вивільнення"),
            Route(self, self.T[2], self.M2, B[3], "B3", handling="вивільнення"),
            Route(self, self.M1, self.T[2], B[2], "B2", handling="захоплення",
select=self._robot_for(2),
                  on_transfer=self._picked(2)),
            Route(self, self.M2, self.T[3], B[4], "B4", handling="захоплення",
select=self._robot_for(3),
                  on_transfer=self._picked(3)),
            Route(self, self.A, self.T[1], B[0], "B0", handling="захоплення",
select=self._robot_for(1),
                  on_transfer=self._picked(1)),
        ]
        Route(self, self.T[3], self.W, handling="вивільнення")
        Route(self, self.W, self.exit)
        self.stat_qs = [self.M1, self.M2, *self.T.values()]
        self.robot_control: Callable[[float], bool] | None = None
        if not self.dedicated:
            self.route_PX = Route(self, self.P, self.X, lambda: True, "BX",
                                  select=lambda robot: self.X.channels[robot.id - 1])
            Route(self, self.X, self.P, select=lambda robot: self.P.channels[robot.id -
1],
                  on_transfer=lambda robot, ch, dst, t: self._arrived(robot, t))
            for r in self.robots:
                self.P.accept(r, 0.0, 0.0, self.P.channels[r.id - 1])
            self.stat_qs.append(self.X)
            self.robot_control = self.control
        self.trace = trace
        self.reset_stats()

    def _blockings(self) -> dict[int, Callable[[], bool]]:
        if self.dedicated:
            return {0: self.B0, 1: self.B1, 2: self.B2, 3: self.B3, 4: self.B4}
        return {0: self.B0_n, 1: self.B1, 2: self.B2_n, 3: self.B3, 4: self.B4_n}

    def B0(self) -> bool:
        return self.T[1].n[FREE] == 0

    def B2(self) -> bool:
        return self.T[2].n[FREE] == 0

    def B4(self) -> bool:
        return self.T[3].n[FREE] == 0

    def B1(self) -> bool:
        return self.M1.n[FREE] == 0

    def B3(self) -> bool:
        return self.M2.n[FREE] == 0
```

```
    def B0_n(self) -> bool:
        return not self.free_at("A") or self.D(1)

    def B2_n(self) -> bool:
        return not self.free_at("M1") or self.D(2)

    def B4_n(self) -> bool:
        return not self.free_at("M2")

    def is_free(self, robot: Robot) -> bool:
        return all(ch.K == FREE for ch in self.robot_channels[robot.id])

    def free_at(self, point: str) -> list[Robot]:
        return [r for r in self.robots if r.pos == point and self.is_free(r)]

    def heading(self, point: str) -> int:
        return sum(1 for ch in self.X.channels if ch.K != FREE and ch.request.target ==
point)

    def u(self, S: int) -> int:
        return self.T[S].requests + self.heading(SRC[S])

    def D(self, S: int) -> bool:
        return S != 3 and self.u(S) > self.M[S].n[FREE]

    def waiting(self, S: int) -> int:
        return len(self.route_from(SRC[S]).waiting)

    def route_from(self, name: str) -> Route:
        return {"A": self.A, "M1": self.M1, "M2": self.M2}[name].route

    def control(self, t: float) -> bool:
        moved = False
        while True:
            best = None
            for r in self.robots:
                if not self.is_free(r):
                    continue
                for S in (1, 2, 3):
                    src = SRC[S]
                    q = self.waiting(S) - self.heading(src) - len(self.free_at(src))
                    if r.pos == src or q <= 0 or self.D(S):
                        continue
                    key = (self.params.distance(r.pos, src), self.streams.ties.random())
                    if best is None or key < best[0]:
                        best = (key, r, S)
            if best is None:
                return moved
            (d, _), r, S = best
            r.target = SRC[S]
            ch = self.P.channels[r.id - 1]
            self.route_PX.waiting.remove(ch)
            self.empty_travel[S] += d
            self.log(t, ch.name, f"холостий хід {r.pos} -> {r.target} ({d:g} с) до деталі
шляху {S}")
            self.route_PX.transfer(ch, t)
            moved = True

    def _robot_for(self, S: int) -> Callable:
        if self.dedicated:
            return lambda part: self.T[S].free_channel()

        def select(part):
            r = self.streams.ties.choice(self.free_at(SRC[S]))
            return self.T[S].channels[r.id - 1]
        return select

    def _robot_at(self, ch: Channel, point: str, t: float) -> None:
        robot = self.robots[ch.index]
```

```
        robot.pos = point
        self.log(t, ch.name, f"робот вільний у {point}")

    def _arrived(self, robot: Robot, t: float) -> None:
        robot.pos = robot.target
        self.log(t, f"X:{robot}", f"кінець холостого ходу, робот у {robot.pos}")

    def _ready(self, part: Part, S: int, t: float) -> None:
        part.t_ready[S] = t

    def _picked(self, S: int) -> Callable:
        def hook(part, ch, dst_ch, t):
            part.t_grab[S] = t
            robot = next(k for k, chs in self.robot_channels.items() if dst_ch in chs)
            self.jobs[(robot, S)] += 1
        return hook

    def handling_time(self) -> float:
        return self.params.handling.sample(self.streams.handling)

    def record_completion(self, part: Part, t: float) -> None:
        part.t_done = t
        self.completed_total += 1
        self.log(t, "вихід", f"{part} на складі, час проходження {t -
part.t_arrival:.3f}")
        if t >= self.t_stats_start:
            self.completed.append(part)

    def wip(self) -> int:
        return self.create.created - self.completed_total

    def schedule(self, ch: Channel, t: float) -> None:
        ch.tnext = t
        self._seq += 1
        ch.seq = self._seq
        heapq.heappush(self.calendar, (t, self._seq, ch))

    def _next_event_time(self) -> float:
        cal = self.calendar
        while cal and (cal[0][2].seq != cal[0][1] or cal[0][2].tnext != cal[0][0]):
            heapq.heappop(cal)                      # застарілий запис календаря
        return min(self.create.tnext, cal[0][0] if cal else INF)

    def transition(self, t: float) -> None:
        due = []
        while self.calendar and self.calendar[0][0] == t:
            _, seq, ch = heapq.heappop(self.calendar)
            if ch.seq == seq and ch.tnext == t:
                due.append(ch)
        self.streams.ties.shuffle(due)              # одночасні події – у випадковому
порядку
        for ch in due:
            ch.qs.end_of_service(ch, t)             # завершення обслуговування в каналі
        if self.create.tnext == t:
            self.create.arrival(t)                  # надходження – після завершень
обслуговування
        self.update_blockings(t)

    def update_blockings(self, t: float) -> None:
        changed = True
        while changed:
            changed = False
            for route in self.routes:
                changed |= route.unblock(t)
            if self.robot_control:
                changed |= self.robot_control(t)

    def run(self, t_end: float, t_warmup: float = 0.0,
            observe_every: float | None = None,
            observer: Callable[["Model"], None] | None = None) -> "Model":
```

```
        self.t_end = t_end
        pending_reset = t_warmup > 0
        next_obs = observe_every if observe_every else INF
        self.update_blockings(0.0)
        self.log_state(0.0)
        while True:
            t_next = self._next_event_time()
            while True:
                t_service = min(t_warmup if pending_reset else INF, next_obs)
                if t_service > min(t_next, t_end):
                    break
                self._advance_stats(t_service)
                if pending_reset and t_service == t_warmup:
                    self.reset_stats()
                    pending_reset = False
                if t_service == next_obs:
                    observer(self)
                    next_obs += observe_every
            if t_next > t_end:
                self._advance_stats(t_end)
                break
            self._advance_stats(t_next)
            self.transition(self.tcurr)
            self.log_state(self.tcurr)
        return self

    def reset_stats(self) -> None:
        self.t_stats_start = self.tcurr
        self.completed = []
        for qs in self.stat_qs:
            qs.reset_stats()
        self.sum_LA = 0.0
        self.hist_LA: dict[int, float] = {}
        self.max_LA = self.A.occupied()
        self.robot_wait = {1: 0.0, 2: 0.0}
        self.jobs: dict[tuple[int, int], int] = defaultdict(int)
        self.empty_travel: dict[int, float] = defaultdict(float)
        self._created_at_reset = self.create.created

    def _advance_stats(self, t_to: float) -> None:
        dt = t_to - self.tcurr
        if dt > 0:
            for qs in self.stat_qs:
                qs.update_stats(self.tcurr, t_to)
            LA = self.A.occupied()
            self.sum_LA += LA * dt
            self.hist_LA[LA] = self.hist_LA.get(LA, 0.0) + dt
            self.max_LA = max(self.max_LA, LA)
            for i in (1, 2):
                if self.T[i].n[BLOCKED]:
                    self.robot_wait[i] += dt
        self.tcurr = t_to

    @property
    def period(self) -> float:
        return self.tcurr - self.t_stats_start

    def sojourn_times(self) -> list[float]:
        return [p.t_done - p.t_arrival for p in self.completed]

    def results(self) -> dict:
        return collect_results(self)

    def robot_time(self, k: int) -> dict[str, float]:
        out = dict.fromkeys(("grab", "loaded_move", "wait_slot", "release",
"empty_move"), 0.0)
        for ch in self.robot_channels[k]:
            acc = ch.qs.time[ch.index]
            if ch.qs is getattr(self, "X", None):
                out["empty_move"] += acc["busy_h"] + acc["busy"]
```

```
                continue
            out["grab"] += acc["busy_h"]
            out["loaded_move"] += acc["busy"]
            out["wait_slot"] += acc["blocked"]
            out["release"] += acc["post_h"]
            out["empty_move"] += acc["post"]          # варіант 1: повернення до точки
завантаження
        return out

    def log(self, t: float, who: str, what: str) -> None:
        if self.trace and t <= self.trace_until:
            self.trace_lines.append(f"{t:10.3f}  {who:<7} {what}")

    def log_state(self, t: float) -> None:
        if self.trace and t <= self.trace_until:
            parts = [f"A={self.A.occupied()}", self.M1.state_str(), self.M2.state_str(),
                     " ".join(self.T[S].state_str() for S in (1, 2, 3))]
            if not self.dedicated:
                parts.append(self.X.state_str())
                parts.append(" ".join(f"{r}@{r.pos}" for r in self.robots))
            self.trace_lines.append(f"{'':10}  {'стан':<7} " + " ".join(parts))


def _mean(xs: list[float]) -> float:
    return sum(xs) / len(xs) if xs else math.nan


def _std(xs: list[float]) -> float:
    if len(xs) < 2:
        return math.nan
    m = _mean(xs)
    return math.sqrt(sum((x - m) ** 2 for x in xs) / (len(xs) - 1))


def _quantile(sorted_xs: list[float], q: float) -> float:
    if not sorted_xs:
        return math.nan
    pos = q * (len(sorted_xs) - 1)
    lo, hi = math.floor(pos), math.ceil(pos)
    return sorted_xs[lo] + (sorted_xs[hi] - sorted_xs[lo]) * (pos - lo)


def _time_quantile(dist: dict[int, float], q: float) -> float:
    acc = 0.0
    for value, share in dist.items():
        acc += share
        if acc >= q - 1e-12:
            return value
    return max(dist) if dist else math.nan


def collect_results(m: Model) -> dict:
    T = m.period
    soj = sorted(m.sojourn_times())
    parts = m.completed
    hist = {k: v / T for k, v in sorted(m.hist_LA.items())} if T > 0 else {}
    res: dict = {
        "strategy": m.params.strategy,
        "seed": m.seed,
        "t_end": m.tcurr,
        "t_warmup": m.t_stats_start,
        "arrived": m.create.created - m._created_at_reset,
        "completed": len(parts),
        "wip_end": m.wip(),
        "throughput_per_hour": len(parts) / T * 3600 if T > 0 else math.nan,
        "sojourn_mean": _mean(soj),
        "sojourn_std": _std(soj),
        "sojourn_min": soj[0] if soj else math.nan,
        "sojourn_p50": _quantile(soj, 0.5),
        "sojourn_p90": _quantile(soj, 0.9),
```

```
        "sojourn_p95": _quantile(soj, 0.95),
        "sojourn_max": soj[-1] if soj else math.nan,
        "wait_route1_mean": _mean([p.t_grab[1] - p.t_ready[1] for p in parts]),
        "wait_route2_mean": _mean([p.t_grab[2] - p.t_ready[2] for p in parts]),
        "wait_route3_mean": _mean([p.t_grab[3] - p.t_ready[3] for p in parts]),
        "buffer_max": m.max_LA,
        "buffer_mean": m.sum_LA / T if T > 0 else math.nan,
        "buffer_end": m.A.occupied(),
        "buffer_q95": _time_quantile(hist, 0.95),
        "buffer_q99": _time_quantile(hist, 0.99),
    }
    for i, mach in ((1, m.M1), (2, m.M2)):
        res[f"M{i}_occupied"] = 1.0 - mach.share(T, ("free",))
        res[f"M{i}_processing"] = mach.share(T, ("busy",))
        res[f"M{i}_blocked"] = mach.share(T, ("blocked",))
        res[f"M{i}_handling"] = mach.share(T, ("busy_h", "post_h", "post"))
        res[f"M{i}_robot_waiting"] = m.robot_wait[i] / T if T > 0 else math.nan
    for S in (1, 2, 3):
        n = sum(v for (k, s), v in m.jobs.items() if s == S)
        travel = m.travel[S] * n if m.dedicated else m.empty_travel[S]
        res[f"empty_route{S}_mean"] = travel / n if n else math.nan
    utils = []
    for k in sorted(m.robot_channels):
        phases = m.robot_time(k)
        u = sum(phases.values()) / T if T > 0 else math.nan
        utils.append(u)
        res[f"robot{k}_util"] = u
        for name, v in phases.items():
            res[f"robot{k}_{name}"] = v / T if T > 0 else math.nan
        for S in (1, 2, 3):
            res[f"robot{k}_jobs_route{S}"] = m.jobs.get((k, S), 0)
    res["robots_util_mean"] = _mean(utils)
    return res


def simulate(params: Params, seed: int, t_end: float, t_warmup: float = 0.0) ->
tuple[dict, list[float]]:
    m = Model(params, seed=seed).run(t_end, t_warmup)
    return m.results(), m.sojourn_times()
```

ДОДАТОК В. ЛІСТИНГ КОДУ ЕКСПЕРИМЕНТАЛЬНОГО ДОСЛІДЖЕННЯ

Наведено модулі пакета `fms.experiments`, які виконують серії експериментів з таблиці 4.1. Команди для їх запуску наведено в підрозділі 4.8.

В.1 Модуль common.py – спільні засоби для серій прогонів

```
from __future__ import annotations

import csv
import json
import math
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

import numpy as np
from scipy import stats

from ..model import simulate
from ..params import DEDICATED, NEAREST, Params

KEY_OUTPUTS = (
    "throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
    "M1_occupied", "M1_processing", "M1_blocked", "M2_occupied", "M2_processing",
"M2_blocked",
    "robot1_util", "robot2_util", "robot3_util", "robots_util_mean",
    "M1_robot_waiting", "M2_robot_waiting", "empty_route1_mean", "empty_route2_mean",
"empty_route3_mean",
)

STRATEGY_LABEL = {DEDICATED: "1) закріплені", NEAREST: "2) найближчий"}


def variants(strategy: str | None) -> list[str]:
    return [strategy] if strategy else [DEDICATED, NEAREST]


def _one(args):
    params, seed, t_end, t_warmup, keep_sojourn = args
    res, soj = simulate(params, seed, t_end, t_warmup)
    return res, (soj if keep_sojourn else None)


def run_batch(tasks: list[tuple], jobs: int = 1) -> list[tuple[dict, list[float] |
None]]:
    if jobs > 1 and len(tasks) > 1:
        with ProcessPoolExecutor(max_workers=jobs) as ex:
            return list(ex.map(_one, tasks, chunksize=max(1, len(tasks) // (4 * jobs))))
    return [_one(t) for t in tasks]


def replicate(params: Params, n: int, t_end: float, t_warmup: float,
              seed0: int = 1, jobs: int = 1, keep_sojourn: bool = False):
    tasks = [(params, seed0 + i, t_end, t_warmup, keep_sojourn) for i in range(n)]
    out = run_batch(tasks, jobs)
    results = [r for r, _ in out]
    sojourns = [s for _, s in out] if keep_sojourn else None
    return results, sojourns


def summarize(values, beta: float = 0.95) -> dict:
    x = np.asarray([v for v in values if not (isinstance(v, float) and math.isnan(v))],
dtype=float)
    n = len(x)
    if n == 0:
        return {"n": 0, "mean": math.nan, "std": math.nan, "ci": math.nan}
    mean = float(x.mean())
    std = float(x.std(ddof=1)) if n > 1 else math.nan
    ci = float(stats.t.ppf((1 + beta) / 2, n - 1) * std / math.sqrt(n)) if n > 1 else
math.nan
    return {"n": n, "mean": mean, "std": std, "ci": ci}


def out_dir(path: str | Path) -> Path:
```

```
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p


def write_csv(path: Path, rows: list[dict]) -> None:
    if not rows:
        return
    keys: list[str] = []
    for r in rows:
        for k in r:
            if k not in keys:
                keys.append(k)
    with path.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)


def write_json(path: Path, data) -> None:
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2, default=float),
encoding="utf-8")


def fmt(v, digits: int = 3) -> str:
    if isinstance(v, float):
        if math.isnan(v):
            return "—"
        return f"{v:.{digits}f}"
    return str(v)


def print_table(rows: list[dict], columns: list[str], headers: list[str] | None = None) -
> None:
    headers = headers or columns
    cells = [[fmt(r.get(c, "")) for c in columns] for r in rows]
    widths = [max(len(h), *(len(c[i]) for c in cells)) if cells else len(h) for i, h in
enumerate(headers)]
    print("  ".join(h.ljust(w) for h, w in zip(headers, widths)))
    print("  ".join("-" * w for w in widths))
    for c in cells:
        print("  ".join(v.ljust(w) for v, w in zip(c, widths)))
```

В.2 Модуль verification.py – верифікація моделі

```
from __future__ import annotations

from ..params import Params
from ..rng import const, exp, norm, unif
from .common import replicate, summarize

SCENARIOS = [
    ("базовий", {}, "—"),
    ("рідше надходження exp(80)", {"arrival": exp(80.0)},
     "пропускна ≈ 45/год, черга A мала, завантаження ↓"),
    ("частіше надходження exp(30)", {"arrival": exp(30.0)},
     "перевантаження: черга A росте, пропускна обмежена M2"),
    ("швидший M2 exp(50)", {"m2_time": exp(50.0)},
     "M2 не вузьке місце: пропускна ≈ 90/год, час проходження ↓"),
    ("повільніший M1 norm(120,10)", {"m1_time": norm(120.0, 10.0)},
     "M1 стає вузьким місцем, зайнятість M1 ≈ 1"),
    ("M2 на 6 місць", {"m2_capacity": 6},
     "пропускна ≈ 90/год, блокування M1 зникає"),
    ("швидке захоплення unif(2,0.5)", {"handling": unif(2.0, 0.5)},
     "завантаження роботів ↓, пропускна ↑"),
    ("переїзди ×3", "travel3",
     "завантаження роботів ↑, час проходження ↑"),
    ("детерміновані часи, exp→const(300)", {"arrival": const(300.0), "m1_time":
const(60.0),
                                            "m2_time": const(100.0), "handling":
const(8.0)},
     "варіант 1: кожна деталь 226 с; варіант 2: 226, 232, 239 с, далі 244 с (226 +
холостий хід W -> A)"),
]

COLUMNS = ["throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
           "M1_occupied", "M2_occupied", "M2_processing", "robots_util_mean", "wip_end"]
INPUT_COLUMNS = ["in_arrival", "in_m1", "in_m2", "in_handling", "in_m2_capacity",
"in_travel"]


def inputs_of(p: Params, travel: float) -> dict:
    return {"in_arrival": str(p.arrival), "in_m1": str(p.m1_time), "in_m2":
str(p.m2_time),
            "in_handling": str(p.handling), "in_m2_capacity": p.m2_capacity, "in_travel":
f"×{travel:g}"}


def run(strategy: str, reps: int, t_end: float, t_warmup: float, jobs: int) ->
list[dict]:
    rows = []
    for name, change, expected in SCENARIOS:
        p = Params(strategy=strategy)
        travel = 3.0 if change == "travel3" else 1.0
        p = p.scaled_travel(travel) if change == "travel3" else p.with_(**change)
        results, _ = replicate(p, reps, t_end, t_warmup, jobs=jobs)
        row = {"scenario": name, "strategy": strategy, **inputs_of(p, travel),
"expected": expected}
        for c in COLUMNS:
            row[c] = summarize([r[c] for r in results])["mean"]
        rows.append(row)
    return rows
```

В.3 Модуль warmup.py – тривалість перехідного періоду

```
from __future__ import annotations

import math

from ..model import Model
from ..params import Params

METRICS = ("throughput_per_hour", "sojourn_mean", "buffer_mean", "M2_occupied",
"robots_util_mean")


def _snapshot(m: Model) -> dict:
    t = m.tcurr
    soj = [p.t_done - p.t_arrival for p in m.completed]
    busy = [sum(m.robot_time(k).values()) / t for k in sorted(m.robot_channels)]
    return {
        "t": t,
        "throughput_per_hour": m.completed_total / t * 3600,
        "sojourn_mean": sum(soj) / len(soj) if soj else math.nan,
        "buffer_mean": m.sum_LA / t,
        "M2_occupied": 1.0 - m.M2.share(t, ("free",)),
        "robots_util_mean": sum(busy) / len(busy),
        "buffer_now": m.A.occupied(),
        "buffer_max": m.max_LA,
    }


def observe(params: Params, seeds: list[int], t_end: float, step: float) -> dict[int,
list[dict]]:
    series = {}
    for seed in seeds:
        snaps: list[dict] = []
        Model(params, seed=seed).run(t_end, observe_every=step, observer=lambda m:
snaps.append(_snapshot(m)))
        series[seed] = snaps
    return series


def transient_time(series: dict[int, list[dict]], metric: str, eps: float) -> float:
    runs = list(series.values())
    n_points = min(len(r) for r in runs)
    T = math.nan
    for k in range(n_points - 1, -1, -1):
        vals = [r[k][metric] for r in runs]
        if any(math.isnan(v) for v in vals) or max(vals) - min(vals) >= eps:
            break
        T = runs[0][k]["t"]
    return T


def run(params: Params, n_runs: int, t_end: float, step: float, eps_rel: float):
    seeds = list(range(1, n_runs + 1))
    series = observe(params, seeds, t_end, step)
    rows = [{"strategy": params.strategy, "arrival_mean": params.arrival.mean, "seed": s,
**snap}
            for s, snaps in series.items() for snap in snaps]
    summary = []
    for metric in METRICS:
        final = [snaps[-1][metric] for snaps in series.values()]
        level = sum(final) / len(final)
        eps = eps_rel * abs(level)
        summary.append({
            "strategy": params.strategy, "arrival_mean": params.arrival.mean, "metric":
metric,
            "final_mean": level, "eps": eps, "T_transient": transient_time(series,
metric, eps),
            "growth_last_half": _growth(series, metric),
```

```
        })
    return rows, summary


def _growth(series: dict[int, list[dict]], metric: str) -> float:
    ratios = []
    for snaps in series.values():
        mid, last = snaps[len(snaps) // 2][metric], snaps[-1][metric]
        if mid and not math.isnan(mid):
            ratios.append(last / mid - 1.0)
    return sum(ratios) / len(ratios) if ratios else math.nan
```

В.4 Модуль replications.py – кількість прогонів

```
from __future__ import annotations

import math

from ..params import Params
from .common import replicate, summarize

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "M2_occupied",
"robots_util_mean")


def run(params: Params, pilot: int, t_end: float, t_warmup: float, eps_rel: float,
        beta: float, jobs: int) -> list[dict]:
    results, _ = replicate(params, pilot, t_end, t_warmup, jobs=jobs)
    rows = []
    for resp in RESPONSES:
        s = summarize([r[resp] for r in results], beta)
        eps = eps_rel * abs(s["mean"])
        sigma = s["std"]
        p_cheb = math.ceil(sigma ** 2 / (eps ** 2 * (1 - beta))) if eps > 0 else math.nan
        p_clt = math.ceil((1.959964 * sigma / eps) ** 2) if eps > 0 else math.nan
        rows.append({
            "strategy": params.strategy, "arrival_mean": params.arrival.mean, "response":
resp,
            "pilot_runs": pilot, "mean": s["mean"], "std": sigma, "eps": eps,
            "p_chebyshev": p_cheb, "p_clt": p_clt,
        })
    return rows
```

В.5 Модуль compare.py – порівняння способів закріплення роботів

```
from __future__ import annotations

import numpy as np
from scipy import stats

from ..params import DEDICATED, NEAREST, Params
from .common import KEY_OUTPUTS, replicate, summarize

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
             "M1_occupied", "M2_occupied", "robots_util_mean")


def anova(groups: list[np.ndarray], alpha: float = 0.05) -> dict:
    q = len(groups)
    p = len(groups[0])
    means = np.array([g.mean() for g in groups])
    grand = np.concatenate(groups).mean()
    s_f = p * ((means - grand) ** 2).sum()
    s_r = sum(((g - g.mean()) ** 2).sum() for g in groups)
    d_f_guide = s_f / (q * p - 1)
    d_r = s_r / (q * (p - 1))
    df1, df2 = q - 1, q * (p - 1)
    f_guide = d_f_guide / d_r if d_r > 0 else np.inf
    f_classic = (s_f / df1) / d_r if d_r > 0 else np.inf
    f_crit = stats.f.ppf(1 - alpha, df1, df2)
    return {
        "S_f": s_f, "S_r": s_r, "D_f(3.10)": d_f_guide, "D_r(3.11)": d_r,
        "F(3.10/3.11)": f_guide, "F_classic": f_classic, "F_crit": f_crit,
        "p_value": float(stats.f.sf(f_classic, df1, df2)),
        "significant": bool(f_classic > f_crit),
    }


def run(reps: int, t_end: float, t_warmup: float, jobs: int,
        base: Params | None = None) -> tuple[list[dict], list[dict], list[dict]]:
    base = base or Params()
    runs: dict[str, list[dict]] = {}
    for strategy in (DEDICATED, NEAREST):
        runs[strategy], _ = replicate(base.with_(strategy=strategy), reps, t_end,
t_warmup, jobs=jobs)

    summary = []
    for strategy, results in runs.items():
        row = {"strategy": strategy, "arrival_mean": base.arrival.mean}
        for k in KEY_OUTPUTS:
            s = summarize([r[k] for r in results])
            row[k] = s["mean"]
            row[k + "_ci"] = s["ci"]
        summary.append(row)

    tests = []
    for resp in RESPONSES:
        a = np.array([r[resp] for r in runs[DEDICATED]], dtype=float)
        b = np.array([r[resp] for r in runs[NEAREST]], dtype=float)
        res = {"response": resp, "arrival_mean": base.arrival.mean,
               "mean_dedicated": a.mean(), "mean_nearest": b.mean(), **anova([a, b])}
        diff = b - a
        t_stat, t_p = stats.ttest_rel(b, a)
        res.update({"paired_diff_mean": diff.mean(), "paired_diff_ci":
summarize(diff)["ci"],
                    "paired_t": float(t_stat), "paired_p": float(t_p)})
        tests.append(res)

    raw = [r for results in runs.values() for r in results]
    return summary, tests, raw
```

В.6 Модуль capacity.py – межа стійкості

```
from __future__ import annotations

from ..params import Params
from ..rng import exp
from .common import run_batch, summarize, variants

DEFAULT_ARRIVALS = (34, 36, 38, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 52, 55, 60,
70, 80)
RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
"buffer_q99",
             "wip_end", "M1_occupied", "M2_occupied", "M2_processing", "M2_blocked",
"robots_util_mean",
             "robot1_util", "robot2_util", "robot3_util")


def run(arrivals, strategy, reps: int, t_end: float, t_warmup: float, jobs: int,
        base: Params | None = None) -> list[dict]:
    base = base or Params()
    points = [(s, a) for s in variants(strategy) for a in arrivals]
    tasks = [(base.with_(strategy=s, arrival=exp(float(a))), seed, t_end, t_warmup,
False)
             for s, a in points for seed in range(1, reps + 1)]
    out = run_batch(tasks, jobs)
    rows = []
    for i, (s, a) in enumerate(points):
        results = [r for r, _ in out[i * reps:(i + 1) * reps]]
        row = {"strategy": s, "arrival_mean": a, "input_per_hour": 3600 / a}
        for k in RESPONSES:
            st = summarize([r[k] for r in results])
            row[k] = st["mean"]
            row[k + "_ci"] = st["ci"]
        row["output_input_ratio"] = row["throughput_per_hour"] / row["input_per_hour"]
        row["stable"] = row["output_input_ratio"] >= 0.98 and row["wip_end"] < 30
        rows.append(row)
    return rows


def max_stable(rows: list[dict]) -> list[dict]:
    out = {}
    for r in rows:
        key = r["strategy"]
        cur = out.setdefault(key, {"strategy": key,
                                   "max_stable_input_per_hour": 0.0,
"min_stable_arrival_mean": None,
                                   "max_throughput_per_hour": 0.0})
        cur["max_throughput_per_hour"] = max(cur["max_throughput_per_hour"],
r["throughput_per_hour"])
        if r["stable"] and r["input_per_hour"] > cur["max_stable_input_per_hour"]:
            cur["max_stable_input_per_hour"] = r["input_per_hour"]
            cur["min_stable_arrival_mean"] = r["arrival_mean"]
    return list(out.values())
```

В.7 Модуль distribution.py – розподіл часу проходження деталей

```
from __future__ import annotations

import math

import numpy as np
from scipy import stats

from ..params import Params
from .common import replicate

CANDIDATES = {
    "normal": (stats.norm, {}, 2),
    "exponential (shifted)": (stats.expon, {}, 2),
    "gamma (shifted)": (stats.gamma, {}, 3),
    "lognormal (shifted)": (stats.lognorm, {}, 3),
    "erlang/gamma (loc=0)": (stats.gamma, {"floc": 0.0}, 2),
}


def histogram(data: np.ndarray, bins: int) -> tuple[np.ndarray, np.ndarray]:
    counts, edges = np.histogram(data, bins=bins)
    return counts, edges


def chi_square(data: np.ndarray, dist, params: tuple, n_params: int, bins: int) -> dict:
    counts, edges = histogram(data, bins)
    cdf = dist.cdf(edges, *params)
    cdf[0], cdf[-1] = 0.0, 1.0
    probs = np.diff(cdf)
    n = len(data)
    obs, exp_ = [], []
    acc_o = acc_e = 0.0
    for o, p in zip(counts, probs):
        acc_o += o
        acc_e += n * p
        if acc_e >= 5:
            obs.append(acc_o)
            exp_.append(acc_e)
            acc_o = acc_e = 0.0
    if acc_e > 0 and exp_:
        obs[-1] += acc_o
        exp_[-1] += acc_e
    obs, exp_ = np.array(obs), np.array(exp_)
    chi2 = float(((obs - exp_) ** 2 / exp_).sum())
    df = len(obs) - 1 - n_params
    crit = float(stats.chi2.ppf(0.95, df)) if df > 0 else math.nan
    return {"chi2": chi2, "df": df, "chi2_crit_0.05": crit, "intervals": len(obs),
            "accepted": bool(df > 0 and chi2 < crit)}


def gamma_shifted_moments(data: np.ndarray) -> tuple[float, float, float]:
    mean, std = float(data.mean()), float(data.std(ddof=1))
    skew = max(float(stats.skew(data)), 1e-3)
    k = (2.0 / skew) ** 2
    theta = std / np.sqrt(k)
    loc = min(mean - k * theta, float(data.min()) - 1e-6)
    return k, loc, theta


def fit_gamma_shifted(data: np.ndarray) -> tuple:
    k0, loc0, theta0 = gamma_shifted_moments(data)
    k, loc, theta = stats.gamma.fit(data, k0, loc=loc0, scale=theta0)
    if k < 1.0 or loc >= data.min():
        return k0, loc0, theta0
    return k, loc, theta
```

```
def fit_all(data: np.ndarray, bins: int) -> list[dict]:
    rows = []
    for name, (dist, fixed, k) in CANDIDATES.items():
        if name.startswith("exponential"):
            params = (float(data.min()), float(data.mean() - data.min()))
        elif name == "gamma (shifted)":
            params = fit_gamma_shifted(data)
        else:
            params = dist.fit(data, **fixed)
        res = chi_square(data, dist, params, k, bins)
        rows.append({"law": name, "params": ", ".join(f"{v:.4g}" for v in params),
                     "params_raw": [float(v) for v in params], **res})
    rows.sort(key=lambda r: r["chi2"])
    return rows


def run(params: Params, reps: int, t_end: float, t_warmup: float, jobs: int,
        bins: int, thin: int, max_n: int = 0) -> dict:
    _, sojourns = replicate(params, reps, t_end, t_warmup, jobs=jobs, keep_sojourn=True)
    data = np.concatenate([np.asarray(s[::thin], dtype=float) for s in sojourns])
    if max_n and len(data) > max_n:
        data = data[np.linspace(0, len(data) - 1, max_n).astype(int)]
    counts, edges = histogram(data, bins)
    hist = [{"left": float(edges[i]), "right": float(edges[i + 1]), "count":
int(counts[i]),
             "freq": float(counts[i] / len(data))} for i in range(len(counts))]
    describe = {
        "n": int(len(data)), "mean": float(data.mean()), "std": float(data.std(ddof=1)),
        "cv": float(data.std(ddof=1) / data.mean()), "min": float(data.min()),
        "p50": float(np.quantile(data, 0.5)), "p90": float(np.quantile(data, 0.9)),
        "p95": float(np.quantile(data, 0.95)), "max": float(data.max()),
        "skew": float(stats.skew(data)),
    }
    return {"describe": describe, "histogram": hist, "fits": fit_all(data, bins), "data":
data}
```

В.8 Модуль improvement.py – пробні запуски пропозицій

```
from __future__ import annotations

from ..params import DEDICATED, NEAREST, Params
from .common import replicate, summarize

SCENARIOS = [
    ("базова система", {}, {}),
    ("4 місця на M2", {"m2_capacity": 4}, {}),
    ("4-й робот на маршруті 1 (A → M1)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
1)}),
    ("4-й робот на маршруті 2 (M1 → M2)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
2)}),
    ("4-й робот на маршруті 3 (M2 → W)", {"n_robots": 4}, {"dedicated_routes": (1, 2, 3,
3)}),
    ("4 місця на M2 і 4-й робот на маршруті 2", {"m2_capacity": 4, "n_robots": 4},
     {"dedicated_routes": (1, 2, 3, 2)}),
]

RESPONSES = ("throughput_per_hour", "sojourn_mean", "buffer_max", "buffer_mean",
"wip_end",
             "M1_occupied", "M2_occupied", "M2_blocked", "robots_util_mean")


def run(reps: int, t_end: float, t_warmup: float, jobs: int, base: Params | None = None)
-> list[dict]:
    base = base or Params()
    rows = []
    seen: set[tuple] = set()
    for name, change, dedicated_change in SCENARIOS:
        for strategy in (DEDICATED, NEAREST):
            params = base.with_(strategy=strategy, **change,
                                **(dedicated_change if strategy == DEDICATED else {}))
            key = (strategy, params.m2_capacity, params.n_robots,
params.dedicated_routes)
            if key in seen:
                continue
            seen.add(key)
            results, _ = replicate(params, reps, t_end, t_warmup, jobs=jobs)
            input_per_hour = 3600 / base.arrival.mean
            label = name if strategy == DEDICATED else name.split(" на маршруті")[0]
            row = {"scenario": label,
                   "strategy": strategy, "m2_capacity": params.m2_capacity, "n_robots":
params.n_robots,
                   "arrival_mean": base.arrival.mean, "input_per_hour": input_per_hour}
            for k in RESPONSES:
                st = summarize([r[k] for r in results])
                row[k] = st["mean"]
                row[k + "_ci"] = st["ci"]
            row["output_input_ratio"] = row["throughput_per_hour"] / input_per_hour
            row["stable"] = row["output_input_ratio"] >= 0.98 and row["wip_end"] < 30
            rows.append(row)
    return rows
```